



廈門大學

《计算机组成原理》 课程实验报告

姓名：马鑫

学院：信息学院

系：软件工程系

专业：数字媒体技术

学号：37220232203780

2026 年 5 月

第 5 次实验 CPU 设计 (I)

1. 实验目的

- (1) 掌握单总线结构 MIPS 处理器（硬布线控制器、微程序控制器）、单周期 MIPS 处理器（硬布线控制器）、单周期 RISC-V 处理器（硬布线控制器）设计的基本原理。
- (2) 对已有的 5 条指令单总线结构 MIPS 处理器（微程序控制器）进行修改，增加 1 条 add 指令（共 6 条指令）。
- (3) 鼓励同学们对已有的 5 条指令单总线结构 MIPS 处理器（硬布线控制器）进行修改，增加 1 条 add 指令（共 6 条指令）。

2. 实验环境

- (1) Windows 系统下运行 Logisim 软件（需安装 JDK）。
- (2) MARS 4.5、RARS 1.5 汇编工具。

3. 实验内容

3.1 验证实验

- (1) 单总线结构 MIPS 处理器（硬布线控制器）（5 条指令）

在单总线结构 MIPS 处理器（硬布线控制器）的数据通路上运行程序：

第一步：测试 5 条指令的程序： test1.asm

```

#单总线结构MIPS处理器测试程序                                test1.asm
#测试5条指令 (lw、sw、beq、addi、slt)
#程序执行完 RAM存储器100、101、102单元的内容分别为：8、9、1

main:
    addi $s0,$zero,8          #s0=8
    sw $s0,1024($zero)        #s0=8 保存到(1024, 即100单元)

    addi $s0,$zero,9          #s0=9
    sw $s0,1028($zero)        #s0=9 保存到(1028, 即101单元)

    lw $s1,1024($zero)         #s1=8
    lw $s2,1028($zero)         #s2=9

    slt $s3,$s1,$s2           # s1 < s2    s3=1

    beq $s1,$s2,loop1         #s1不等于s2  执行下一条指令

    sw $s3,1032($zero)         #s3=1 保存到(1032, 即102单元)

    beq $zero,$zero,loop2      #转loop2

loop1:
    addi $s4,$zero,2           #s4=2
    sw $s4,1032($zero)         #s4=2 保存到(1032, 即102单元)
                                这条指令不会执行到
                                这条指令不会执行到

loop2:
    beq $zero,$zero,loop2      #转loop2      死循环

```

行 7, 列 49 | 1,007 个字符 | 纯文本 | 100% | Unix (LF) | ANSI

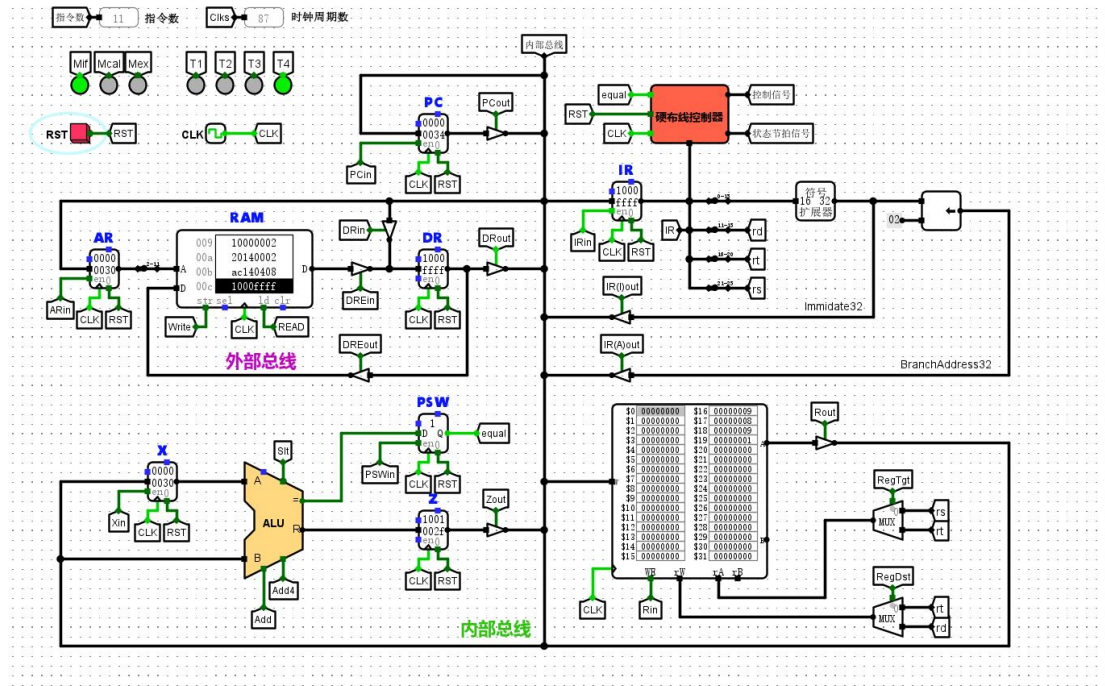
第二步：在单总线结构 MIPS 处理器（硬布线控制器）的数据通路上运行测试程序（test1.hex）：

将 test1.hex 装入到 RAM 中

Logsim: 1 / 1 近世代数综合

编辑	工程	电路仿真	窗口	帮助
000	20100008	ac100400	20100009	ac100404
010	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000

运行程序



观看 RAM 存储器 100~102 存储单元内容正确

```

0e0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
0f0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
100 00000008 00000009 00000001 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
120 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0

```

第三步：运行 5 个数的排序程序 sort3_mips_bus.hex 和 sort4_mips_bus.hex

将 sort3_mips_bus.hex 装入 RAM 存储器中

电路仿真	窗口	帮助
000	20100003	ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0274402a 11000002
010	ae330000	ae140000 2231fffc 12110001 1000fff7 22100004 20110410 12110001 1000fff3 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

运行后观看 RAM 存储器 100~104 存储单元按降序排序

```

0e0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
0f0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
100 00000005 00000004 00000003 00000002 00000001 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
120 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0
130 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0

```

将 sort4_mips_bus.hex 装入 RAM 存储器中

电路仿真	窗口	帮助
000	20100003	ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0293402a 11000002
010	ae330000	ae140000 2231fffc 12110001 1000fff7 22100004 20110410 12110001 1000fff3 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

运行后观看 RAM 存储器 100~104 存储单元按升序排序

```

0f0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 000
100 00000001 00000002 00000003 00000004 00000005 00000000 00000000 000
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 000
120 00000000 00000000 00000000 00000000 00000000 00000000 00000000 000

```

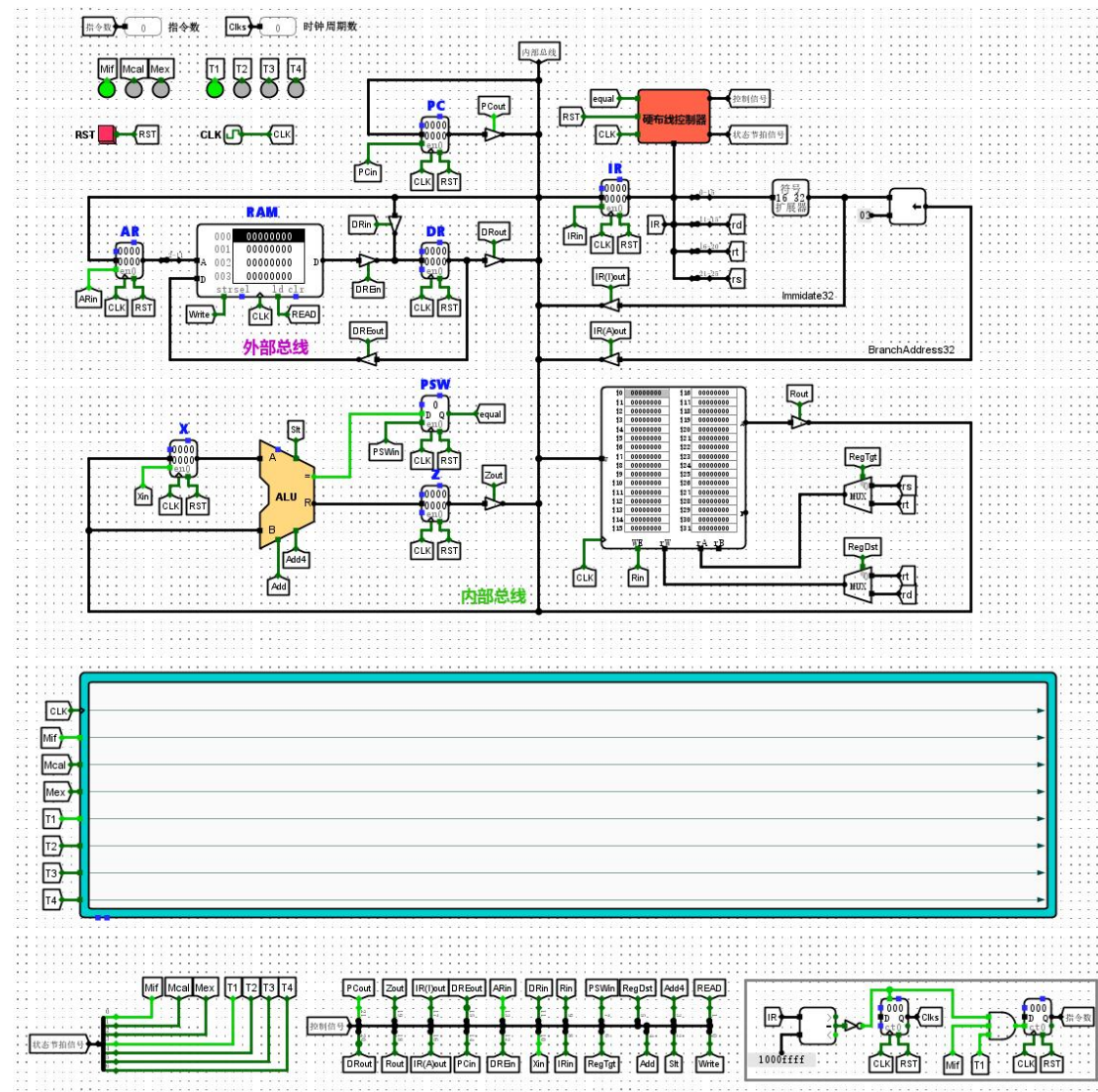
请问：在该单总线结构 MIPS 处理器（硬布线控制器）的数据通路上，能够运行求累加和程序、 计算费波那契数列程序吗？为什么？

答：不能。该数据通路仅支持 addi 立即数加法指令，缺少 add 寄存器加法指令，无法实现变量之间的加法。

请分析单总线结构 MIPS 处理器（硬布线控制器）的电路原理：

• 包括：数据通路、硬布线控制器、指令译码器、状态机、输出函数、硬布线控制器组合逻辑单元、算术逻辑单元 ALU 等电路。

答：



数据通路：

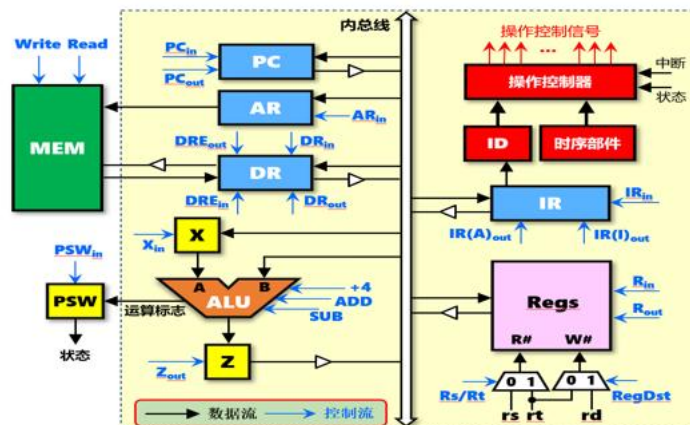


图6.8 单总线结构的计算机框图

参考教材图 6.8，包括以下功能模块：

PC：程序计数器；

AR：存储器地址寄存器；

DR：存储器数据存储器；

X、Z：暂存器；

ALU：算术逻辑单元；

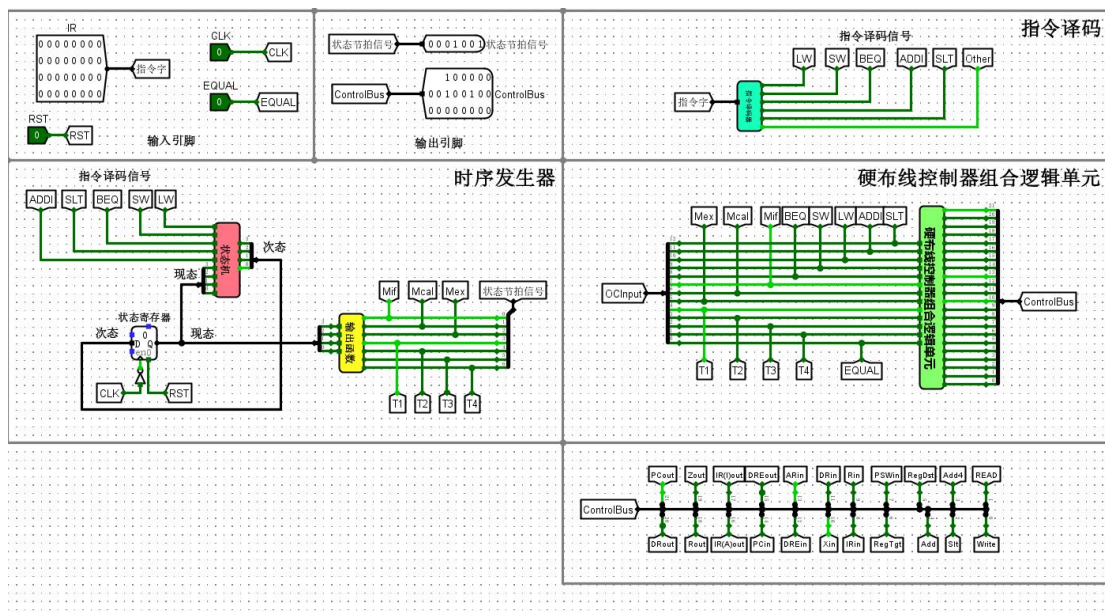
PSW：程序状态字寄存器；

IR：指令寄存器；

ID：指令译码器；

Regs：寄存器堆；

MEM：存储器。



硬布线控制器：

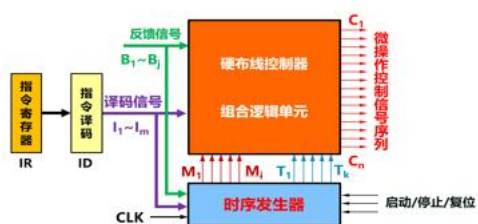
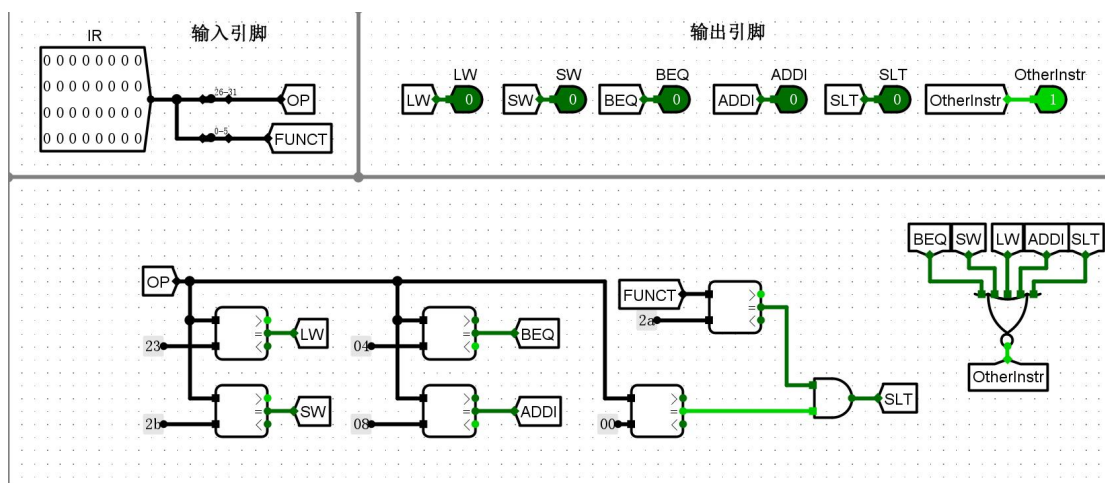
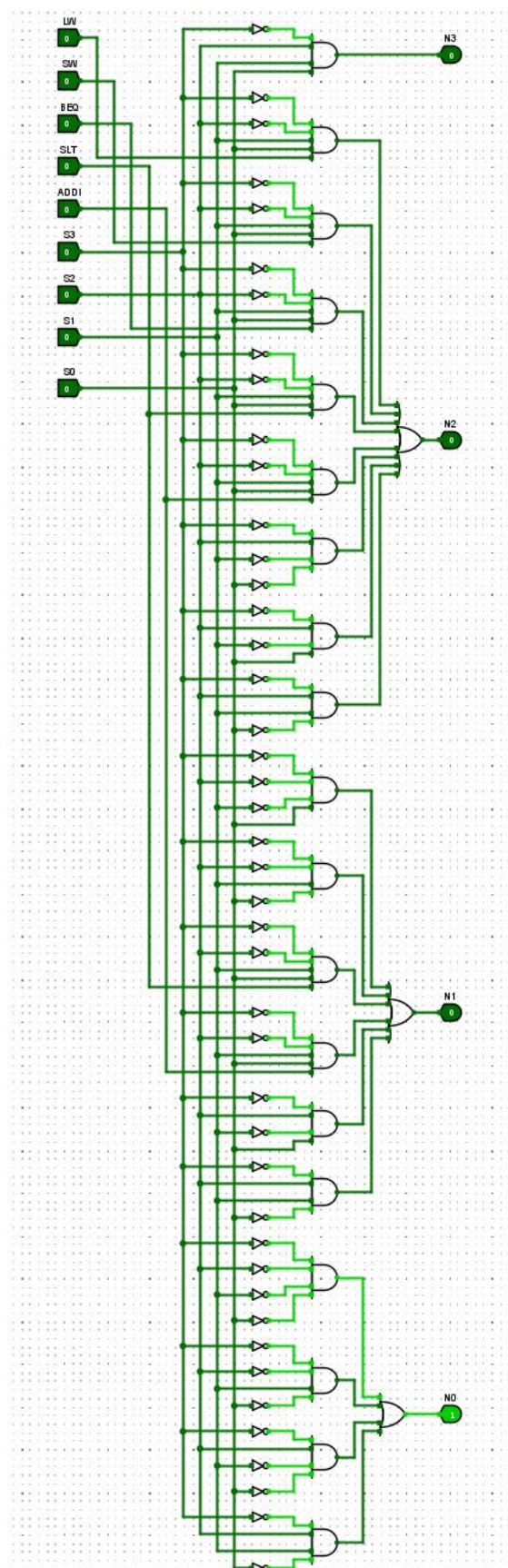


图6.43 传统三级时序硬布线控制器模型

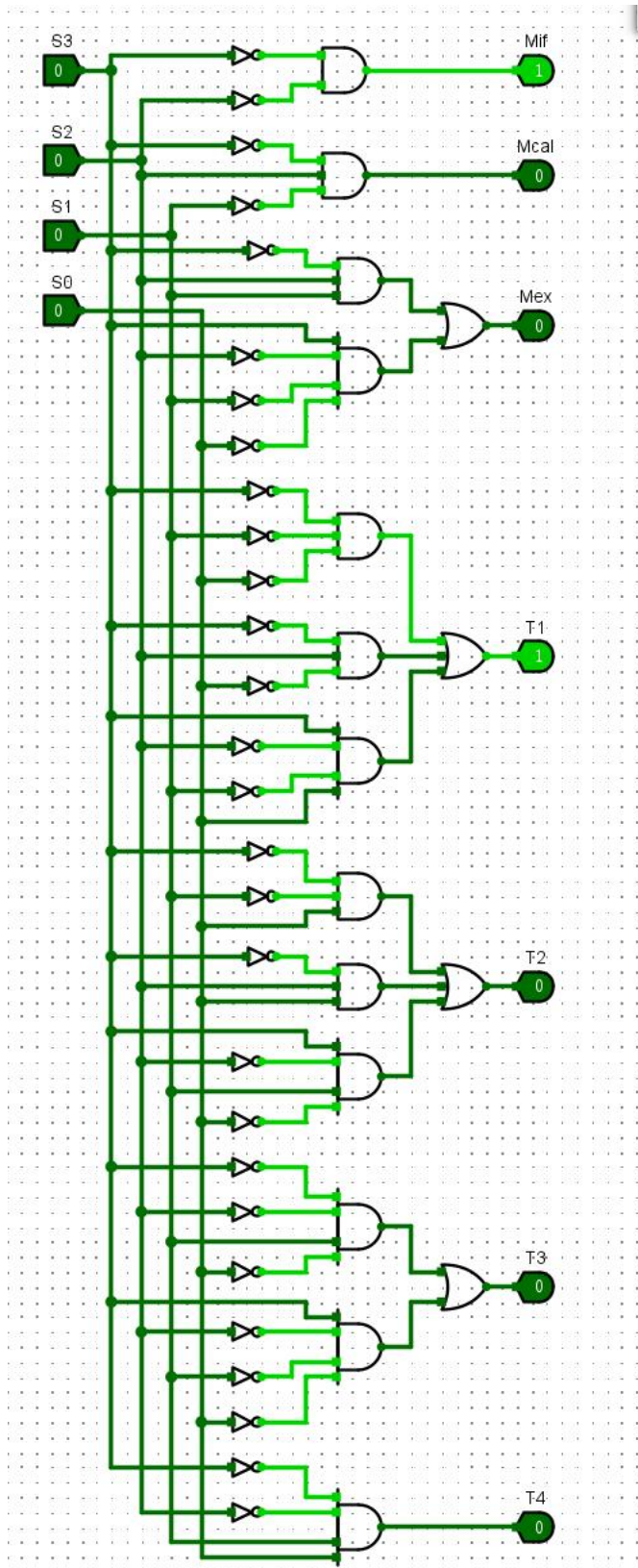
参考教材图 6.43。具体模块见后文。



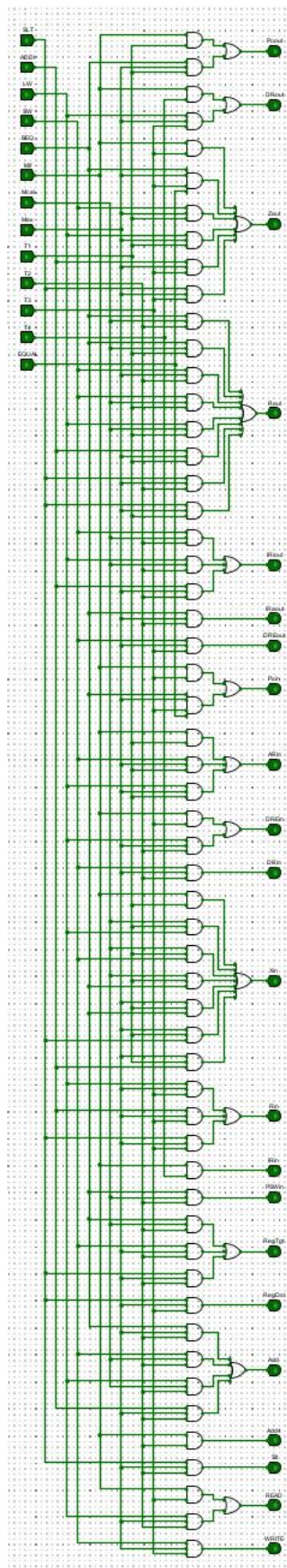
指令译码器：通过多个比较器将 32 位指令分为高 6 位 FUNCT 和低 6 位 OP，从而翻译为特定的指令，OP 的值 0x23 为 LW，0x2b 为 SW，0x04 为 BEQ，0x08 为 ADDI，OP 为 0x00 且 FUNCT 为 0x2a 为 SLT，其他均归为 OtherInstr。



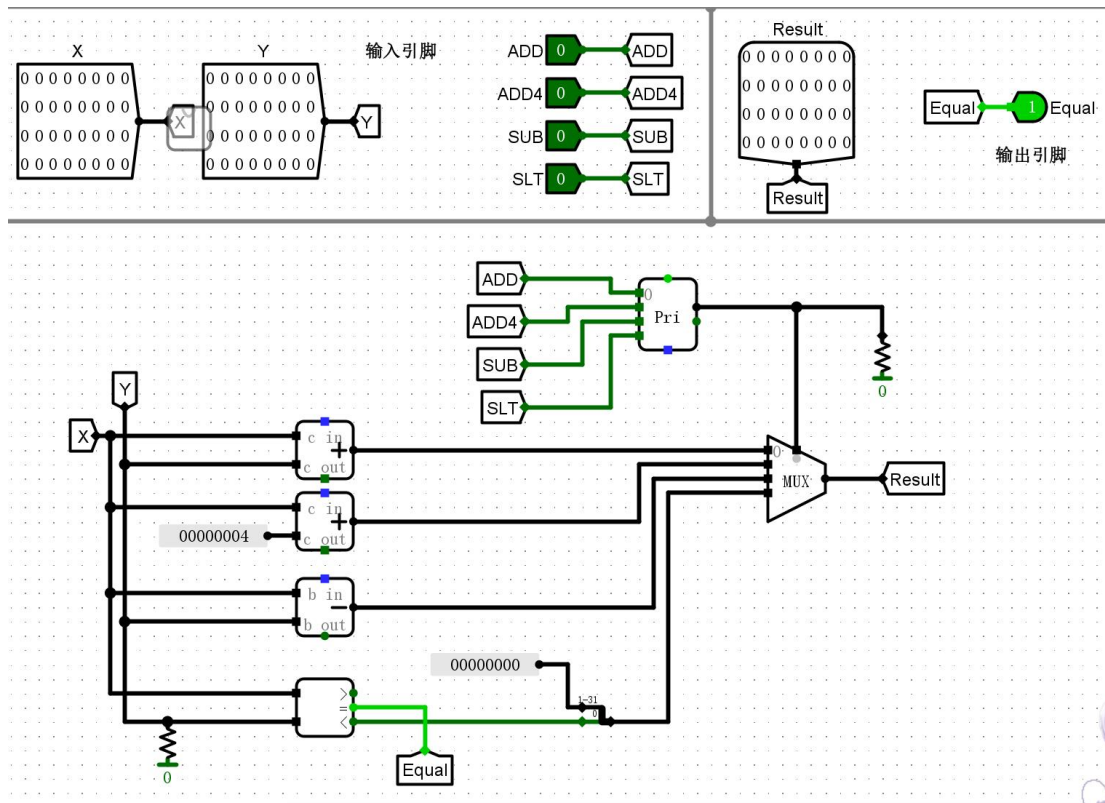
状态机：根据现态信号 S0-S3 和指令信号，推出次态。



输出函数：根据 4 个现态信号，输出对应的状态节拍信号。



硬布线控制器组合逻辑单元：把信号与指令译码信号合在一起进行逻辑运算，输出总线控制信号。



算术逻辑单元 ALU：支持 ADD、ADD4、SUB、SLT 四种运算的结果输出，先用加法器、减法器、比较器等获得结果，再根据多路选择器选择输出的结果。

(2) 单总线结构 MIPS 处理器（微程序控制器）（5 条指令）

在单总线结构 MIPS 处理器（微程序控制器）的数据通路上运行程序：

- test1.hex、 sort3_mips_bus.hex、 sort4_mips_bus.hex

```
test1.asm  git_commit# .env  NTEUpdate.l  test1.i  X
文件  编辑  查看

#单总线结构MIPS处理器测试程序                                test1.asm
#测试5条指令 (lw、sw、beq、addi、slt)
#程序执行完 RAM存储器100、101、102单元的内容分别为: 8、9、1

main:
    addi $s0,$zero,8          #s0=8
    sw $s0,1024($zero)        #s0=8 保存到(1024, 即100单元)

    addi $s0,$zero,9          #s0=9
    sw $s0,1028($zero)        #s0=9 保存到(1028, 即101单元)

    lw $s1,1024($zero)         #s1=8
    lw $s2,1028($zero)         #s2=9

    slt $s3,$s1,$s2           # s1 < s2    s3=1

    beq $s1,$s2,loop1         #s1不等于s2  执行下一条指令

    sw $s3,1032($zero)         #s3=1 保存到(1032, 即102单元)

    beq $zero,$zero,loop2      #转loop2

loop1:
    addi $s4,$zero,2           #s4=2
    sw $s4,1032($zero)         #s4=2 保存到(1032, 即102单元)
                                这条指令不会执行到
                                这条指令不会执行到

loop2:
    beq $zero,$zero,loop2      #转loop2    死循环
```

```

#MIPS程序  sort3 mips bus.asm  排序程序（降序排列，从大到小）  仅使用5条指令
(lw、sw、beq、addi、slt)

#先将5个数（3、1、5、2、4）存放的地址为1024开始的RAM存储器中，然后对这5个数进行排序，排序好的5个数
（5、4、3、2、1）仍然放在地址为1024开始的RAM存储器中

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改） 保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改） 保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改） 保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改） 保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改） 保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024        # $s0=1024          排序区间开始地址
    addi $s1,$zero,1040        # $s1=1040=1024+5*4-4  排序区间结束地址  5个数

sort loop:
    lw $s3,0($s0)              # $s3=($s0)
    lw $s4,0($s1)              # $s4=($s1)
    slt $t0,$s3,$s4            #如果 $s3 < $s4, 则置 $t0=1; 否则, 置 $t0=0  降序排序  从大到小
    beq $t0,$zero,sort next    #如果 $t0=0, 则转sort next
    sw $s3,0($s1)              #交换($s0)和($s1)
    sw $s4,0($s0)              #交换($s0)和($s1)

sort next:
    addi $s1,$s1,-4            # $s1-4 -> $s1
    beq $s0,$s1,loop1          #如果 $s0=$s1, 则转loop1
    beq $zero,$zero,sort loop  #转sort loop

loop1:
    addi $s0,$s0,4             # $s0+4 -> $s0
    addi $s1,$zero,1040        # $s1=1040=1024+5*4-4  排序区间结束地址  5个数
    beq $s0,$s1,loop2          #如果 $s0=$s1, 则转loop2
    beq $zero,$zero,sort loop  #转sort loop

loop2:
    beq $zero,$zero,loop2      #转loop2  死循环

```

```

#MIPS程序  sort4 mips bus.asm  排序程序（升序排列，从小到大）  仅使用5条指令
(lw、sw、beq、addi、slt)

#先将5个数（3、1、5、2、4）存放到地址为1024开始的RAM存储器中，然后对这5个数进行排序，排序好的5个数
（1、2、3、4、5）仍然放在地址为1024开始的RAM存储器中

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改） 保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改） 保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改） 保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改） 保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改） 保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024       # $s0=1024
    addi $s1,$zero,1040       # $s1=1040=1024+5*4-4  排序区间开始地址  排序区间结束地址  5个数

sort loop:
    lw $s3,0($s0)             # $s3=($s0)
    lw $s4,0($s1)             # $s4=($s1)
    slt $t0,$s4,$s3           #如果 $s4 < $s3, 则置 $t0=1; 否则, 置 $t0=0  升序排序  从小到大
    beq $t0,$zero,sort next   #如果 $t0=0, 则转sort next
    sw $s3,0($s1)             #交换($s0)和($s1)
    sw $s4,0($s0)             #交换($s0)和($s1)

sort next:
    addi $s1,$s1,-4           # $s1-4 -> $s1
    beq $s0,$s1,loop1         #如果 $s0=$s1, 则转loop1
    beq $zero,$zero,sort loop #转sort loop

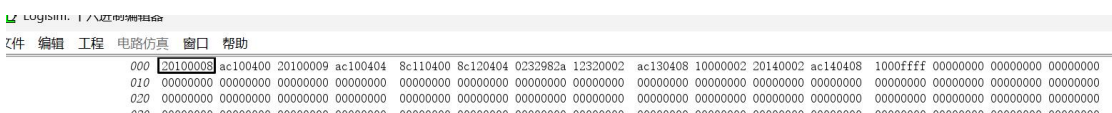
loop1:
    addi $s0,$s0,4            # $s0+4 -> $s0
    addi $s1,$zero,1040       # $s1=1040=1024+5*4-4  排序区间结束地址  5个数
    beq $s0,$s1,loop2         #如果 $s0=$s1, 则转loop2
    beq $zero,$zero,sort loop #转sort loop

loop2:
    beq $zero,$zero,loop2     #转loop2  死循环

```

- 具体步骤与单总线结构 MIPS 处理器（硬布线控制器）相同。
- 将.hex 文件装入到 RAM 中，按 RST 按钮，再按 Ctrl+K，等到 RAM 存储器中的内容不动，再按 Ctrl+K。

装入 test1.hex



装入 sort3_mips_bus.hex

电路仿真	窗口	帮助
000	20100003	ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0274402a 11000002
010	ae330000	ae140000 2231ffff 12110001 1000ffff 22100004 20110410 12110001 1000fff3 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

装入 sort4_mips_bus.hex

电路仿真	窗口	帮助
000	20100003	ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0293402a 11000002
010	ae330000	ae140000 2231ffff 12110001 1000ffff 22100004 20110410 12110001 1000fff3 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

- 观看 RAM 存储器中的结果是否正确？

运行完 test1.hex 后，观看 RAM 存储器 100~102 存储单元内容正确。

```
0f0 00000000 00000000 00000000 00000000 00000000
100 00000008 00000009 00000001 00000000 00000000
110 00000000 00000000 00000000 00000000 00000000
```

运行完 sort3_mips_bus.hex 后，观看 RAM 存储器 100~104 存储单元按降序排序

```
0f0 00000000 00000000 00000000 00000000 00000000 00
100 00000005 00000004 00000003 00000002 00000001 00
110 00000000 00000000 00000000 00000000 00000000 00
```

运行完 sort4_mips_bus.hex 后，观看 RAM 存储器 100~104 存储单元按升序排序

```
0f0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
100 00000001 00000002 00000003 00000004 00000005 00000000 00000000 00000000 00000000
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

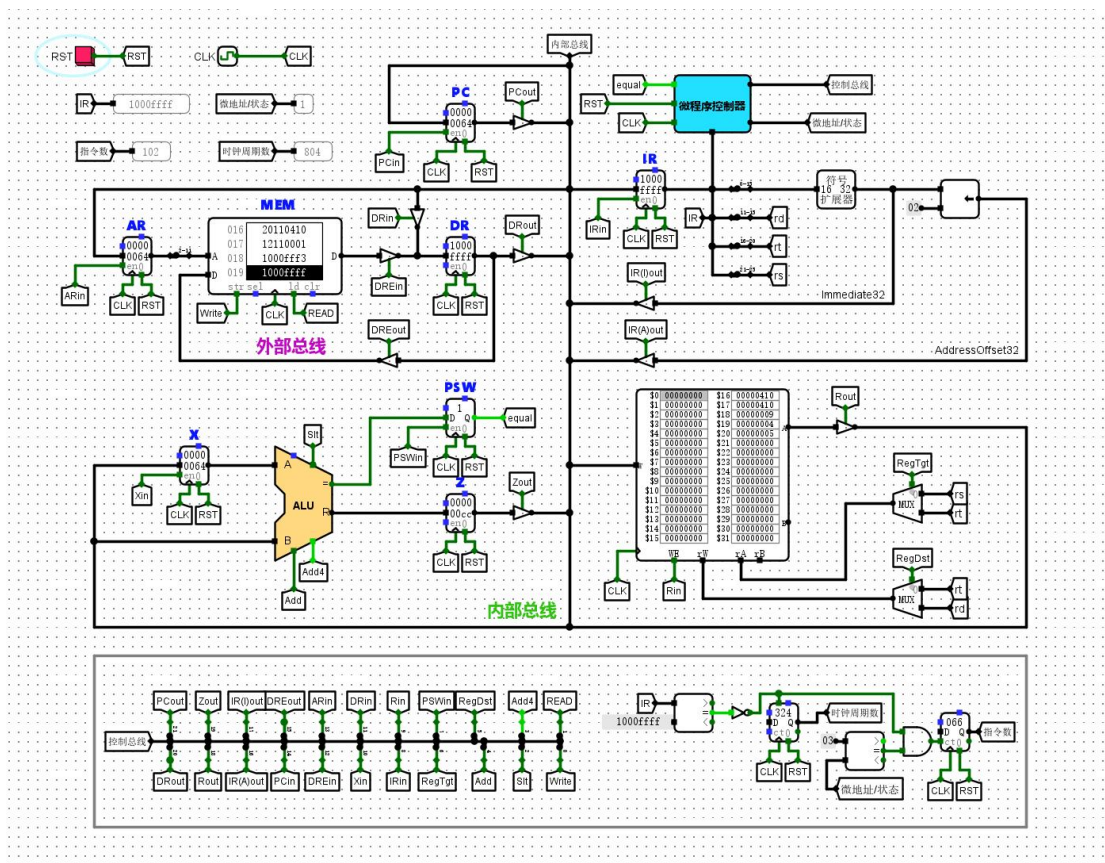
请问：在该单总线结构 MIPS 处理器（微程序控制器）的数据通路上，能够运行求累加和程序、计算费波那契数列程序吗？为什么？

答：不能。该数据通路仅支持 addi 立即数加法指令，缺少 add 寄存器加法指令，无法实现变量之间的加法。

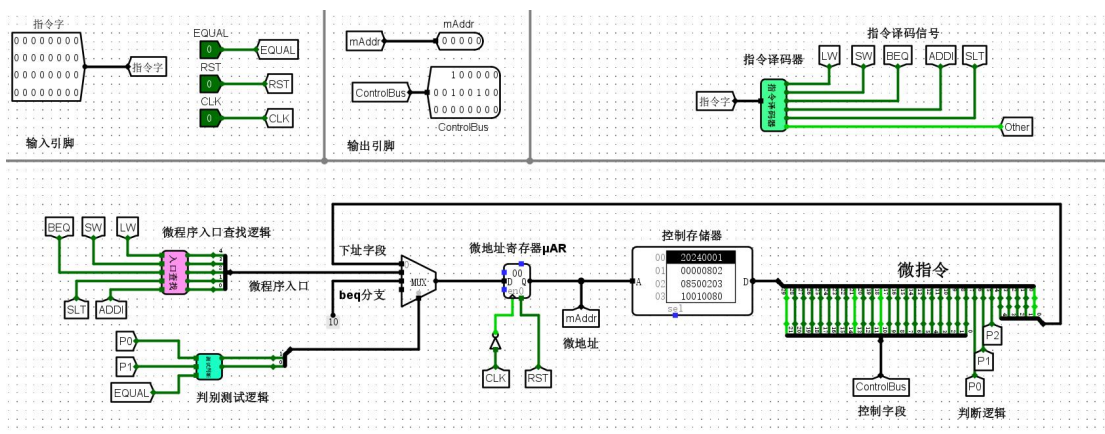
请分析单总线结构 MIPS 处理器（微程序控制器）的电路原理和微程序：

- 包括：数据通路、微程序控制器（下址字段法）、微程序入口查找逻辑、判别测试逻辑、控制存储器中的微程序（25 条微指令）等。

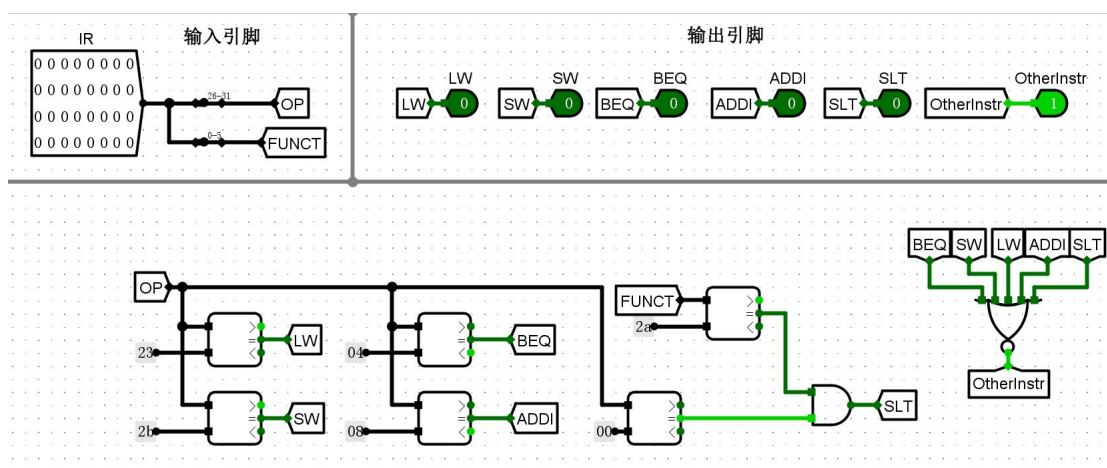
答：



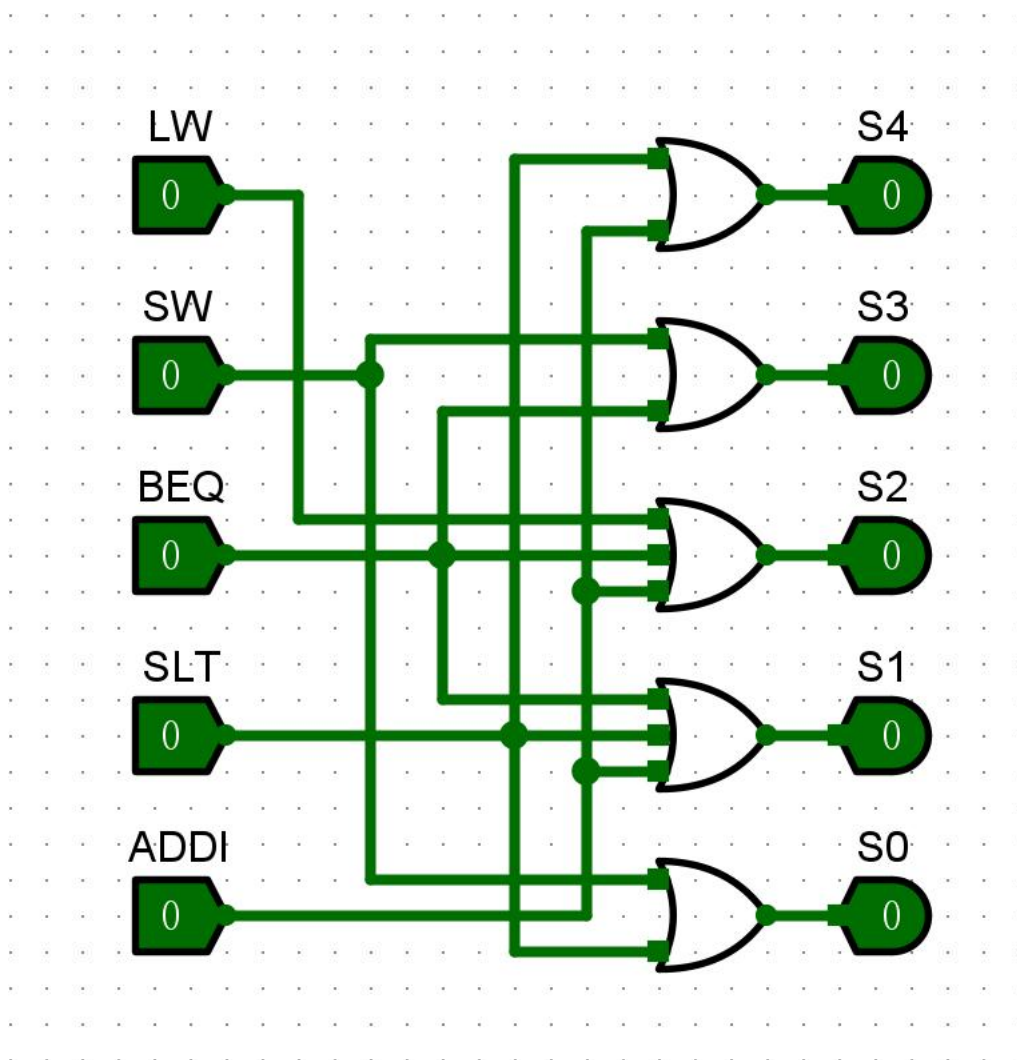
数据通路：与硬布线控制器基本相同。



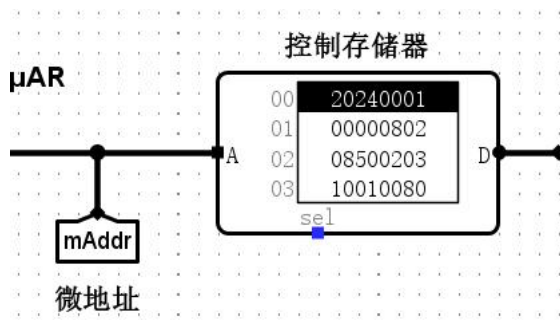
微程序控制器：具体模块见后文。



指令译码器：同硬布线控制器。



微程序入口查找逻辑电路：根据指令译码信号，逻辑运算得到微程序入口地址值（S4S3S2S1S0），具体为 LW=1，输出 4，SW=1，输出 9，BEQ=1，输出 14，SLT=1，输出 19，ADDI=1，输出 22。



控制存储器中的微程序: 用只读存储器用来存放全部指令系统的微程序, 通过输入地址, 输出相应的微程序。

(3) 单周期 MIPS 处理器 (硬布线控制器) (24 条指令)

测试程序: 包括求累加和程序、计算费波那契数列程序、排序程序 (降序、升序)

- 先在指令存储器中装入.hex 文件, 按 Ctrl+K, 再按 “总复位” 按钮, 指令存储器中的内容停止后, 按 Ctrl+K, 观看数据存储器中的内容是否正确?

```

#MIPS程序 求累加和          sum mips.asm
#求累加和: 1+2+.....+n, n的值为10 (可以改变), 累加和的结果存放到地址为0的数据存储器中

main:
    addi $s0,$zero,10        # n=10 -> s0
    addi $s1,$zero,1         # 1 -> s1
    addi $s2,$zero,1         # 1 -> s2
    addi $s3,$zero,0         # 0 -> s3
loop:
    add $s3,$s3,$s1          # s3+s1 -> s3
    beq $s1,$s0,finish       # 如果s1=s0, 则转finish
    add $s1,$s1,$s2          # s1+s2 -> s1
    j loop
finish:
    sw $s3,0($zero)          # s3 存到地址为0的存储单元中

    addi $v0,$zero,10        # 10号系统调用
    syscall                  # 程序退出
  
```

sum_mips

```
Jdate.l | test1.asm | sort3_mips.t | sort4_mips.t | sum_mips.as | fib_m | ×
文件 编辑 查看

#MIPS程序  计算费波那契数列                                fib mips.asm
#n=10 (可以改变), 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 保存在地址为
0、4、8、12、...的数据存储器中

main:
    addi $s2,$zero,10                                # $s2=n=10

    addi $s1,$zero,0                                # $s1=0
    sw $s1,0($zero)                                # 0 -> (0)

    addi $s1,$zero,1                                # $s1=1
    sw $s1,4($zero)                                # 1 -> (4)

    sw $s1,8($zero)                                # 1 -> (8)

    addi $s3,$zero,2                                # $s3 = 2
    addi $s4,$zero,1                                # $s4 = 1
    addi $s5,$zero,1                                # $s5 = 1

loop:
    add $s6,$s4,$s5                                # $s4+$s5 -> $s6
    addi $s4,$s5,0                                # $s5 -> $s4
    addi $s5,$s6,0                                # $s6 -> $s5
    addi $s3,$s3,1                                # $s3+1 -> $s3
    add $s7,$s3,$s3                                # $s3+$s3 -> $s7
    add $s0,$s7,$s7                                # $s7+$s7 -> $s0
    sw $s6,0($s0)                                # $s6 -> 0($s0)
    beq $s2,$s3,end                                # if $s2 = $s3 goto end
    j loop                                        # goto loop

end:
    addi $v0,$zero,10                                # 10号系统调用
    syscall                                        # 程序退出
```

fib_mips

```

#MIPS程序  sort3 mips bus.asm  排序程序（降序排列，从大到小）  仅使用5条指令
(lw、sw、beq、addi、slt)

#先将5个数（3、1、5、2、4）存放到地址为0开始的数据存储器中，然后对这5个数进行排序。

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改）保存到(0)
    sw $s0,0($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改）保存到(4)
    sw $s0,4($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改）保存到(8)
    sw $s0,8($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改）保存到(12)
    sw $s0,12($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改）保存到(16)
    sw $s0,16($zero)

    addi $s0,$zero,0          #s0=0
    addi $s1,$zero,16         #s1=16=5*4-4  排序区间开始地址  5个数

sort loop:
    lw $s3,0($s0)             #s3=(s0)
    lw $s4,0($s1)             #s4=(s1)
    slt $t0,$s3,$s4           #如果s3<s4, 则置t0=1; 否则, 置t0=0  降序排序  从大到小
    beq $t0,$zero,sort next   #如果t0=0, 则转sort next
    sw $s3,0($s1)             #交换(s0)和(s1)
    sw $s4,0($s0)             #交换(s0)和(s1)

sort next:
    addi $s1,$s1,-4           #s1-4 -> s1
    beq $s0,$s1,loop1         #如果s0=s1, 则转loop1
    beq $zero,$zero,sort loop #转sort loop

loop1:
    addi $s0,$s0,4            #s0+4 -> s0
    addi $s1,$zero,16         #s1=16=5*4-4  排序区间结束地址  5个数
    beq $s0,$s1,loop2         #如果s0=s1, 则转loop2
    beq $zero,$zero,sort loop #转sort loop

loop2:
    beq $zero,$zero,loop2     #转loop2  死循环

```

sort3_mips

```

#MIPS程序  sort4 mips bus.asm  排序程序 (升序排列, 从小到大)  仅使用5条指令
(lw、sw、beq、addi、slt)

#先将5个数 (3、1、5、2、4) 存放到地址为0开始的数据存储器中, 然后对这5个数进行排序。

.text

main:
    addi $s0,$zero,3          #第1个数=3 (可以修改) 保存到(0)
    sw $s0,0($zero)
    addi $s0,$zero,1          #第2个数=1 (可以修改) 保存到(4)
    sw $s0,4($zero)
    addi $s0,$zero,5          #第3个数=5 (可以修改) 保存到(8)
    sw $s0,8($zero)
    addi $s0,$zero,2          #第4个数=2 (可以修改) 保存到(12)
    sw $s0,12($zero)
    addi $s0,$zero,4          #第5个数=4 (可以修改) 保存到(16)
    sw $s0,16($zero)

    addi $s0,$zero,0          # $s0=0
    addi $s1,$zero,16         # $s1=16=5*4-4  排序区间开始地址  排序区间结束地址  5个数

sort loop:
    lw $s3,0($s0)             # $s3=($s0)
    lw $s4,0($s1)             # $s4=($s1)
    slt $t0,$s4,$s3           #如果 $s4 < $s3, 则置 $t0=1; 否则, 置 $t0=0  升序排序  从小到大
    beq $t0,$zero,sort next   #如果 $t0=0, 则转 sort next
    sw $s3,0($s1)             #交换 ($s0)和 ($s1)
    sw $s4,0($s0)             #交换 ($s0)和 ($s1)

sort next:
    addi $s1,$s1,-4           # $s1-4 -> $s1
    beq $s0,$s1,loop1         #如果 $s0=$s1, 则转 loop1
    beq $zero,$zero,sort loop #转 sort loop

loop1:
    addi $s0,$s0,4            # $s0+4 -> $s0
    addi $s1,$zero,16         # $s1=16=5*4-4  排序区间结束地址  5个数
    beq $s0,$s1,loop2         #如果 $s0=$s1, 则转 loop2
    beq $zero,$zero,sort loop #转 sort loop

loop2:
    beq $zero,$zero,loop2     #转 loop2  死循环

```

sort4_mips

```

编辑 工程 电路仿真 窗口 帮助
000 20100003 20110001 20120001 20130000 02719820 12300002 02328820 08000c04 ac130000 2002000a 0000000c 00000000 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 sum_mips.hex

```

十六进制编辑器
工程 电路仿真 窗口 帮助
000 00000037 00000000 00000000 00000000 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

运行后累加和存储在地址为 0 的数据存储器中

皇	电路仿真	窗口	帮助
000	2012000a	20110000	ac110000 20110001 ac110004 ac110008 20130002 20140001 20150001 0295b020 22b40000 22d50000 22730001 0273b820 02f78020 ae160000
010	12530001	08000c09	2002000a 0000000c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

载入 fib_mips.hex

im: 十六进制编辑器

目工程电路仿真窗口帮助

000	00000000	00000001	00000001	00000002	00000003	00000005	00000008	0000000d	00000015	00000022	00000037	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
030	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

运行后计算好的 11 个数（0、1、1、2、3、5、8、13、21、34、……）保存在地址为 0、4、8、12、...的数据存储器中

皇	电路仿真	窗口	帮助
000	20100003	ac100000	20100001 ac100004 20100005 ac100008 20100002 ac10000c 20100004 ac100010 20100000 20110010 8e130000 8e340000 0274402a 11000002
010	ae330000	ae140000	2231ffff 12110001 1000ffff 22100004 20110010 12110001 1000ffff 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

载入 sort3_mips_bus.hex

gism: 十六进制编辑器

编辑 工程 电路仿真 窗口 帮助

000	00000028	00000005	00000004	00000002	00000001	00000000	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

运行后，5 个数在地址为 0 开始的数据存储器中降序排序

皇	电路仿真	窗口	帮助
000	20100003	ac100000	20100001 ac100004 20100005 ac100008 20100002 ac10000c 20100004 ac100010 20100000 20110010 8e130000 8e340000 0293402a 11000002
010	ae330000	ae140000	2231ffff 12110001 1000ffff 22100004 20110010 12110001 1000ffff 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

载入 sort4_mips_bus.hex

m: 十六进制编辑器

目 工程 电路仿真 窗口 帮助

000	00000001	00000002	00000003	00000004	00000005	00000000	00000000	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

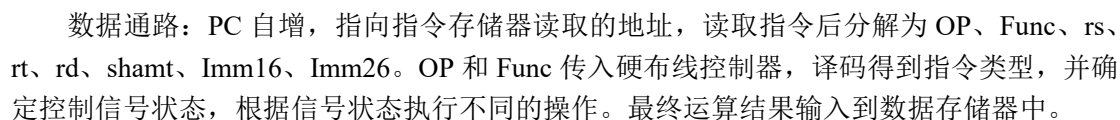
运行后，5 个数在地址为 0 开始的数据存储器中升序排序

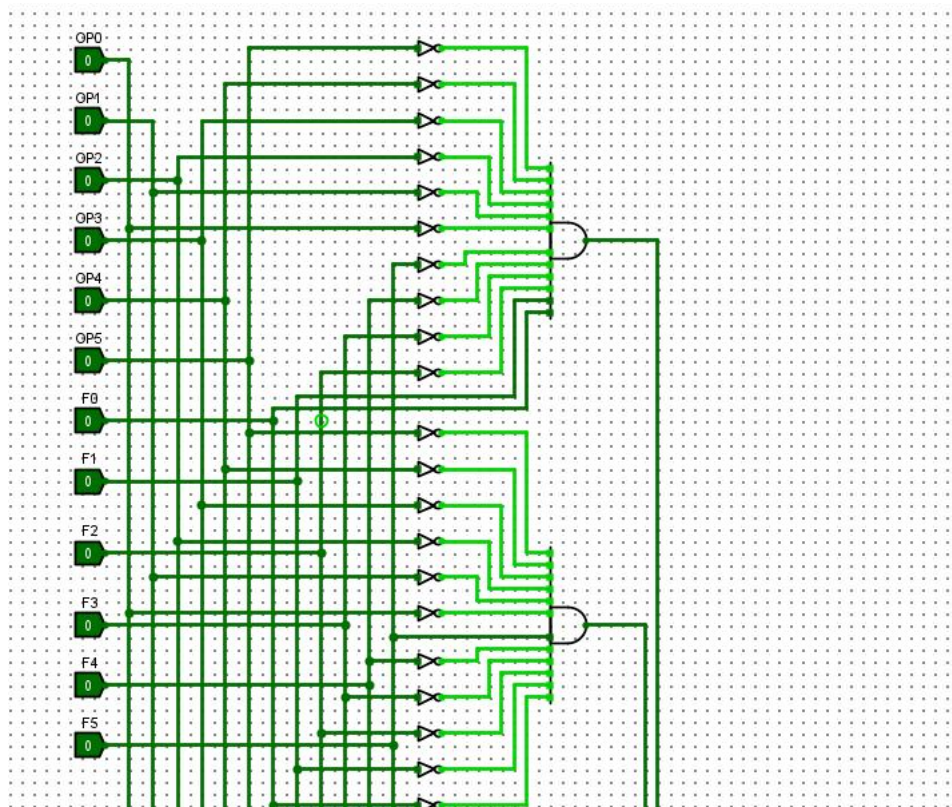
请问：该单周期 MIPS 处理器的控制器能不能采用微程序控制器的方法设计？为什么？

答：不能。微程序取指令、指令译码、取数过程就要消耗一个时钟周期，而单周期处理器的所有指令在一个时钟周期内完成。

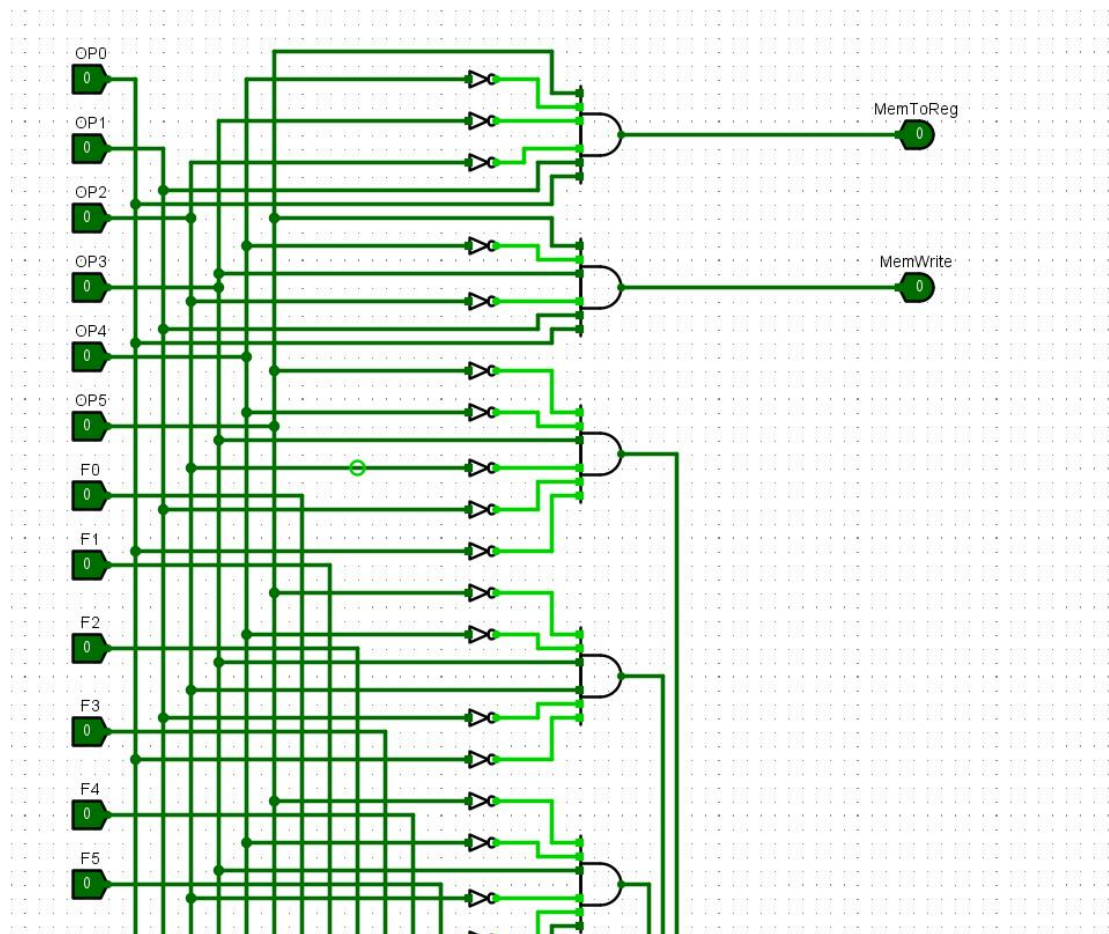
请分析单周期 MIPS 处理器（硬布线控制器）的电路原理：

- 答:





运算器控制器：根据 6 位 OP 和 Func 字段，经过逻辑运算得出 4 位 AluOP。



控制信号生成：根据 6 位 OP 和 Func 字段，经过逻辑运算得出各控制信号的电平。

编写测试程序，对 24 条指令的功能进行测试。

答：编写 test.asm，编译成 hex 文件后，逐步运行进行验证

```

.data
    test_data: .word 0x12345678, 0xabcdef01, 0x00000000

.text
.globl main
main:
    # 建立 RAM 的基地址指针, 令 $gp = 0x00000000
    addiu $gp, $0, 0

    # -----
    # 1. 初始化与立即数加法 (测试指令 10: addiu)
    # -----
    addiu $s0, $0, 10
    sw    $s0, 8($gp)    # 落盘验证 -> RAM[0x08] = 0x0000000A

    addiu $s1, $0, 20

    addiu $s2, $0, -5
    sw    $s2, 12($gp)   # 落盘验证 -> RAM[0x0C] = 0xFFFFFFF8 (负数符号扩展)

```

工程 电路仿真 窗口 帮助

```

000 12345678 abcdef01 00000000 000
010 00000000 00000000 00000000 000

```

手动初始化数据存储器

工程 电路仿真 窗口 帮助

```

000 12345678 abcdef01 0000000a ffffffff 00000000 00000000 (
010 00000000 00000000 00000000 00000000 00000000 00000000 (
020 00000000 00000000 00000000 00000000 00000000 00000000 (
030 00000000 00000000 00000000 00000000 00000000 00000000 (

```

addiu, sw 得到验证

```

# -----
# 2. 算术运算指令测试 (测试指令 1: add, 9: addu, 3: addi, 16: sub)
# -----
add    $t0, $s1, $s0
sw     $t0, 16($gp)    # 落盘验证 -> RAM[0x10] = 0x0000001E (30)

addu   $t1, $s1, $s2
sw     $t1, 20($gp)    # 落盘验证 -> RAM[0x14] = 0x0000000F (15)

addi   $t2, $s0, -15
sw     $t2, 24($gp)    # 落盘验证 -> RAM[0x18] = 0xFFFFFFF8 (-5)

sub     $t3, $s1, $s0
sw     $t3, 28($gp)    # 落盘验证 -> RAM[0x1C] = 0x0000000A (10)

```

制编辑器

电路仿真 窗口 帮助

```
000 12345678 abcdef01 0000000a ffffffff 0000001e 0000000f ffffffff 0000000a
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

add,addu,addi,sub 得到验证

```
# -----
# 3. 逻辑运算指令测试 (测试指令 11: and, 12: andi, 17: or, 18: ori, 19: nor)
# -----
addiu $s3, $0, 0x5555
addiu $s4, $0, 0xaaaa

and $t4, $s3, $s4
sw $t4, 32($gp) # 落盘验证 -> RAM[0x20] = 0x00000000

andi $t5, $s3, 0x00ff
sw $t5, 36($gp) # 落盘验证 -> RAM[0x24] = 0x00000055 (零扩展)

or $t6, $s3, $s4
sw $t6, 40($gp) # 落盘验证 -> RAM[0x28] = 0x0000FFFF

ori $t7, $s3, 0xff00
sw $t7, 44($gp) # 落盘验证 -> RAM[0x2C] = 0x0000FF55 (零扩展)

nor $t8, $t4, $0
sw $t8, 48($gp) # 落盘验证 -> RAM[0x30] = 0xFFFFFFFF
```

工程 电路仿真 窗口 帮助

```
000 12345678 abcdef01 0000000a ffffffff 0000001e 0000000f ffffffff 0000000a 00000000 00000055 0000ffff 0000ff55 ffffffff
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

and,andi,or,ori,nor 得到验证

```
# -----
# 4. 移位运算指令测试 (测试指令 13: sll, 14: srl, 15: sra)
# -----
addiu $s5, $0, 2
sll $t0, $s5, 2
sw $t0, 52($gp) # 落盘验证 -> RAM[0x34] = 0x00000008

srl $t1, $t0, 1
sw $t1, 56($gp) # 落盘验证 -> RAM[0x38] = 0x00000004

sra $t2, $s2, 2 # $s2 是 -5 (0xffffffffb)
sw $t2, 60($gp) # 落盘验证 -> RAM[0x3C] = 0xFFFFFFF0 (-2, 算术右移符号保持)
```

30111_1_1\AZL\中\2019\01\01\01\01

编辑 工程 电路仿真 窗口 帮助

```
000 12345678 abcdef01 0000000a ffffffff 0000001e 0000000f ffffffff 0000000a 00000000 00000055 0000ffff 0000ff55 ffffffff 00000008 00000004 fffffffe
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

sll,srl,sra 得到验证

```
# -----  
# 5. 比较置位指令测试 (测试指令 2: slt, 20: sltu, 21: slti)  
# -----  
slt $t3, $s2, $s0    # -5 < 10 成立  
sw  $t3, 64($gp)     # 落盘验证 -> RAM[0x40] = 0x00000001  
  
sltu $t4, $s2, $s0    # 无符号 0xffffffffb < 0x0000000a 不成立  
sw  $t4, 68($gp)     # 落盘验证 -> RAM[0x44] = 0x00000000  
  
slti $t5, $s2, 0      # -5 < 0 成立  
sw  $t5, 72($gp)     # 落盘验证 -> RAM[0x48] = 0x00000001
```

程 电路仿真 窗口 帮助

000	12345678	abcdef01	0000000a	fffffffb	0C
010	00000001	00000000	00000001	00000000	0C
020	00000000	00000000	00000000	00000000	0C
030	00000000	00000000	00000000	00000000	0C
...	...	...	...	...	...

slt,sltu,slti 得到验证

```

# -----
# 6. 访存与分支跳转组合验证 (测试指令 4: lw, 6: beq, 7: bne, 22: j, 23: jal, 24: jr)
# -----
# 重新用 lw 读取最初的 RAM 数据来做分支判断, 顺便测试 lw 指令
lw  $a0, 0($gp)    # $a0 = 0x12345678
lw  $a1, 0($gp)    # $a1 = 0x12345678
addiu $a2, $0, 0    # $a2 = 0

beq  $a0, $a1, test bne # 如果 lw 和 beq 正常, 必跳转
beq  $0, $0, error    # 没跳走说明出错, 卡死

test bne:
    bne  $a0, $a2, test j # 如果 bne 正常, 必跳转
    beq  $0, $0, error

test j:
    j    test jal        # 如果 j 正常, 必跳转
    beq  $0, $0, error

test jal:
    jal  target sub      # 调用子程序, 测试 jal

# 子程序成功返回后到这里
addiu $v1, $0, 99        # 终极全通标记 99 (0x63)
sw  $v1, 76($gp)        # 落盘验证 -> RAM[0x4C] = 0x00000063
j    finish

target sub:
    jr  $ra              # 测试 jr 返回

```

十六进制编辑器

工程 电路仿真 窗口 帮助

000	12345678	abcdef01	0000000a	fffffffb	0C
010	00000001	00000000	00000001	00000063	0C
020	00000000	00000000	00000000	00000000	0C

lw,beq,bne,j,jal,jr 得到验证

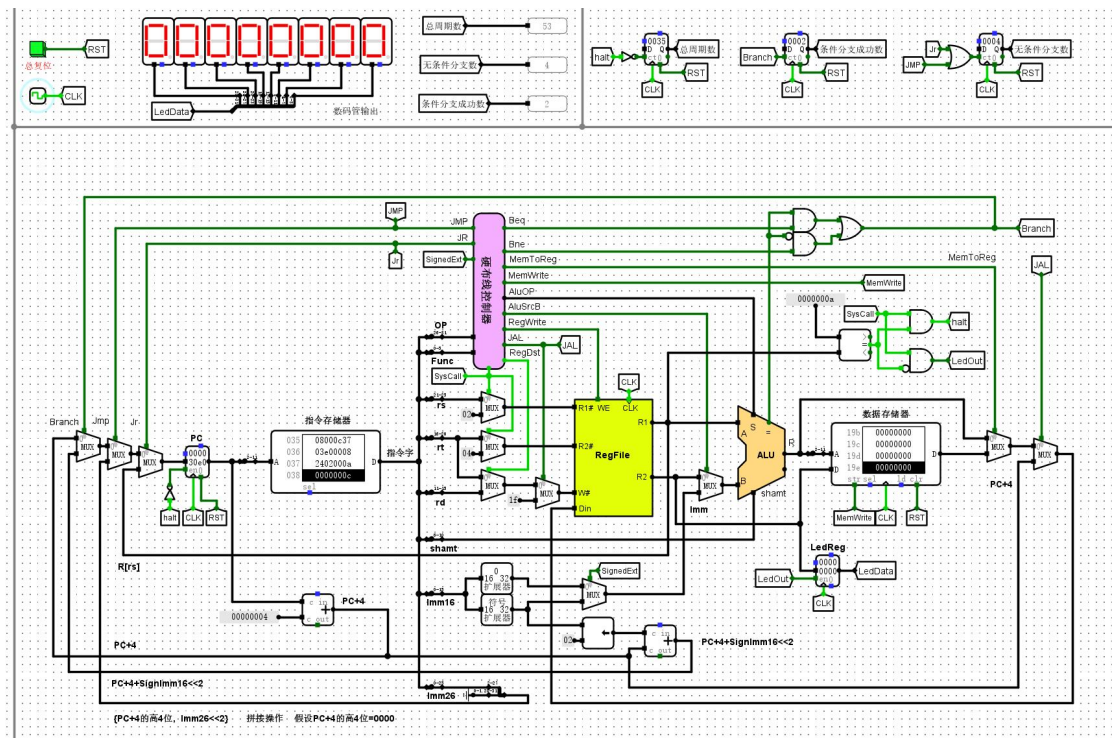
```

# -----
# 7. 终点与错误陷阱 (测试指令 8: syscall)
# -----

finish:
    addiu $v0, $0, 10
    syscall    # 顺利停机!

error:
    # 任何一步跳转失败, 或者逻辑算错导致分支走错, 都会死锁在这里。
    # 此时 RAM[0x4C] 将永远是初始的 0x00000000, 不会变成 0x63。
    beq  $0, $0, error

```



顺利停机，syscall 得到验证

(4) 单周期 RISC-V 处理器（硬布线控制器）（9 条指令）

测试程序：包括求累加和程序、计算费波那契数列程序、排序程序（降序、升序）

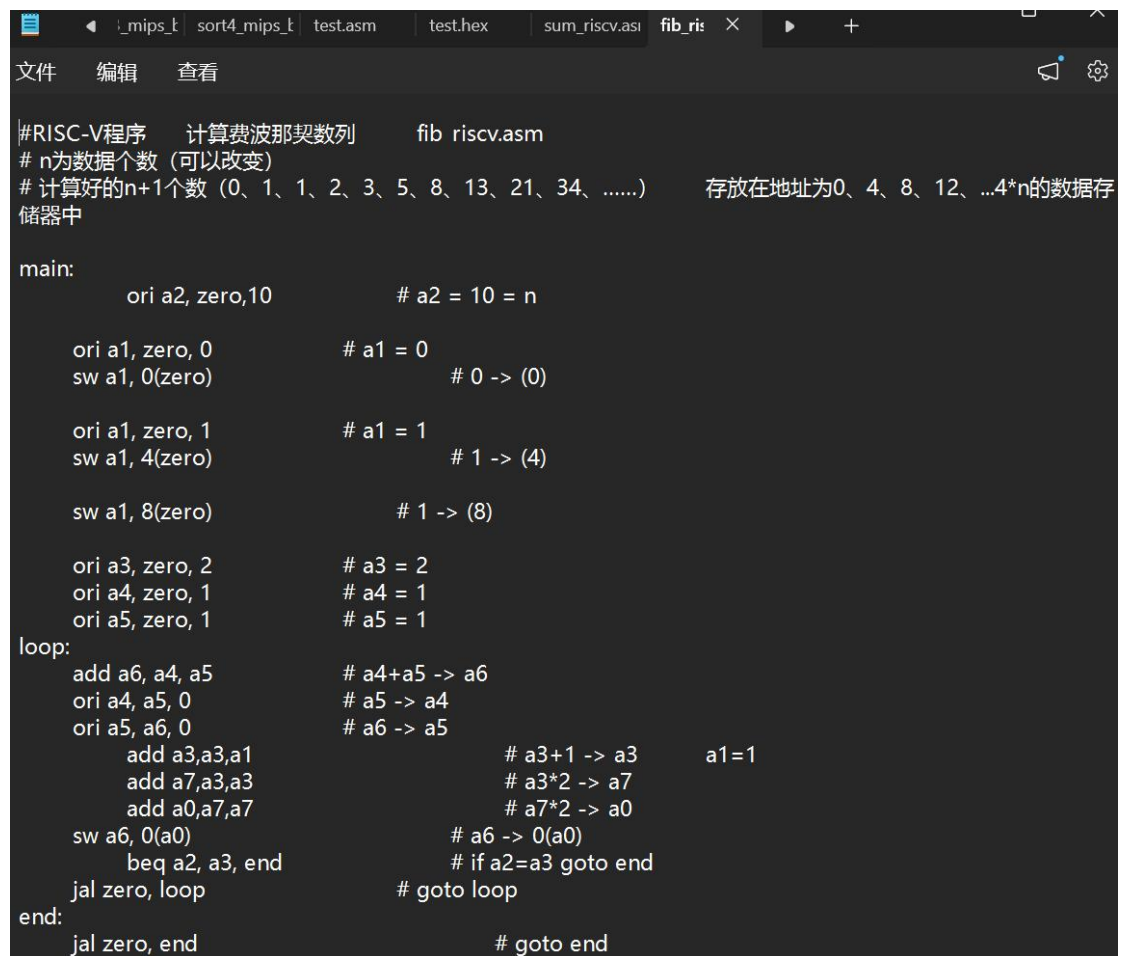
- 先在指令存储器中装入.hex 文件，按 Ctrl+K，再按“复位”按钮，指令存储器中的内容停止后，按 Ctrl+K，观看数据存储器中的内容是否正确？

```

# RISC-V计算累加和程序      sum riscv.asm      result=1+2+...+n
# 结果result存放在地址为0的数据存储器中

main:
    ori a3,zero,10          # a3=10=n
    ori a5, zero, 1          # a5=1
    ori a2, zero, 1          # a2=1
    ori a4, zero, 0          # a4=0
loop:
    add a4, a4, a5           # a4+a5 -> a4
    beq a5, a3, finish       # 判断a5=a3?      若a5=a3=n, 则跳出循环
    add a5, a5, a2           # a5+a2 -> a5
    jal zero, loop           # 无条件跳转到loop执行
finish:
    sw a4, 0(zero)           # a4 -> (0)        将累加结果result保存到存储器中
end:
    jal zero, end            # 无条件跳转到end执行
  
```

sum_riscv



```
#RISC-V程序  计算斐波那契数列      fib_riscv.asm
# n为数据个数 (可以改变)
# 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....)      存放在地址为0、4、8、12、...4*n的数据存储器中

main:
    ori a2, zero, 10          # a2 = 10 = n

    ori a1, zero, 0           # a1 = 0
    sw a1, 0(zero)             # 0 -> (0)

    ori a1, zero, 1           # a1 = 1
    sw a1, 4(zero)             # 1 -> (4)

    sw a1, 8(zero)             # 1 -> (8)

    ori a3, zero, 2           # a3 = 2
    ori a4, zero, 1           # a4 = 1
    ori a5, zero, 1           # a5 = 1

loop:
    add a6, a4, a5             # a4+a5 -> a6
    ori a4, a5, 0              # a5 -> a4
    ori a5, a6, 0              # a6 -> a5
    add a3, a3, a1             # a3+1 -> a3      a1=1
    add a7, a3, a3             # a3*2 -> a7
    add a0, a7, a7             # a7*2 -> a0
    sw a6, 0(a0)               # a6 -> 0(a0)
    beq a2, a3, end            # if a2=a3 goto end
    jal zero, loop             # goto loop

end:
    jal zero, end              # goto end
```

fib_riscv

```

#RISC-V冒泡方法排序程序      sort3_riscv.asm      将5个数据进行排序（从大到小，降序排列）
# 5个数据由程序设置存放在地址为0、4、8、12、16的数据存储器中
# 排列好的数据仍然放在地址为0、4、8、12、16的数据存储器中

main:
    ori  a2,zero,5          #a2=5

    ori  a1, zero, 3         # a1 = 3
    sw   a1, 0(zero)         # 3 -> (0)
    ori  a1, zero, 1         # a1 = 1
    sw   a1, 4(zero)         # 1 -> (4)
    ori  a1, zero, 5         # a1 = 5
    sw   a1, 8(zero)         # 5 -> (8)
    ori  a1, zero, 2         # a1 = 2
    sw   a1, 12(zero)        # 2 -> (12)
    ori  a1, zero, 4         # a1 = 4
    sw   a1, 16(zero)        # 4 -> (16)

    ori  a4,zero,1
    ori  a5,zero,4
    ori  a6,zero,-1
loop1:
    beq  a2,a4,finish
    ori  a3,zero,1
    ori  a7,zero,0
    ori  s8,zero,4
loop2:
    sltu s11,a3,a2
    beq  s11,zero,loop3
    lw   s9,0(a7)
    lw   s10,0(s8)
    sltu s11,s10,s9          #降序排列，从大到小
    beq  s11,a4,loop4
    sw   s10,0(a7)
    sw   s9,0(s8)
    jal  zero,loop4
loop3:
    add  a2,a2,a6
    jal  zero,loop1
loop4:
    add  a3,a3,a4
    add  a7,a7,a5
    add  s8,s8,a5
    jal  zero,loop2
finish:
    jal  zero,finish

```

sort3_riscv

```

#RISC-V冒泡方法排序程序      sort4 riscv.asm      将5个数据进行排序（从小到大，升序排列）
# 5个数据由程序设置存放在地址为0、4、8、12、16的数据存储器中
# 排列好的数据仍然放在地址为0、4、8、12、16的数据存储器中

main:
    ori    a2,zero,5          #a2=5

    ori    a1, zero, 3        # a1 = 3
    sw     a1, 0(zero)        # 3 -> (0)
    ori    a1, zero, 1        # a1 = 1
    sw     a1, 4(zero)        # 1 -> (4)
    ori    a1, zero, 5        # a1 = 5
    sw     a1, 8(zero)        # 5 -> (8)
    ori    a1, zero, 2        # a1 = 2
    sw     a1, 12(zero)       # 2 -> (12)
    ori    a1, zero, 4        # a1 = 4
    sw     a1, 16(zero)       # 4 -> (16)

    ori    a4,zero,1
    ori    a5,zero,4
    ori    a6,zero,-1

loop1:
    beq    a2,a4,finish
    ori    a3,zero,1
    ori    a7,zero,0
    ori    s8,zero,4

loop2:
    sltu   s11,a3,a2
    beq    s11,zero,loop3
    lw     s9,0(a7)
    lw     s10,0(s8)
    sltu   s11,s9,s10          #升序排列，从小到大
    beq    s11,a4,loop4
    sw     s10,0(a7)
    sw     s9,0(s8)
    jal    zero,loop4

loop3:
    add    a2,a2,a6
    jal    zero,loop1

loop4:
    add    a3,a3,a4
    add    a7,a7,a5
    add    s8,s8,a5
    jal    zero,loop2

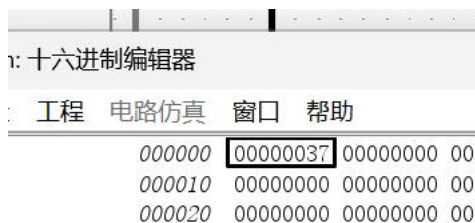
finish:
    jal    zero,finish

```

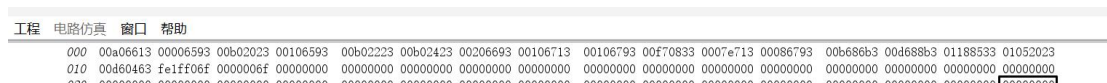
sort4_riscv

呈	电路仿真	窗口	帮助
000	00a06693	00106793	00106613 00006713 00f70733 00d78663 00c787b3 ff5ff06f 00e02023 0000006f 00000000 00000000
010	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

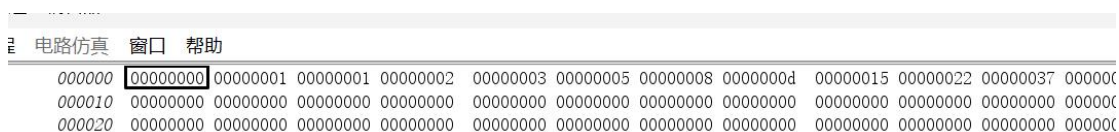
载入 sum_riscv.hex



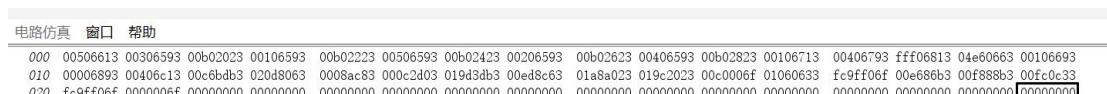
运行后累加和结果存放在地址为 0 的数据存储器中



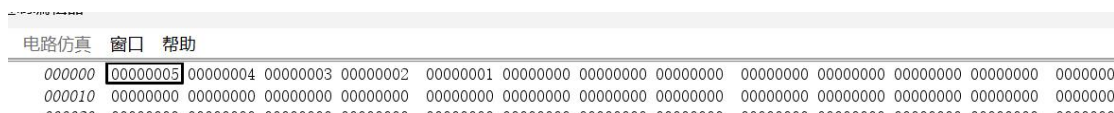
载入 fib_riscv.hex



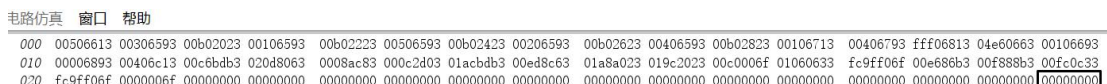
运行后计算好的 11 个数（0、1、1、2、3、5、8、13、21、34、……）存放在地址为 0、4、8、12、...40 的数据存储器中



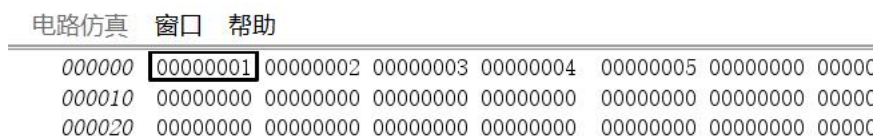
载入 sort3_riscv.hex



运行后 0、4、8、12、16 数据存储器中的数据降序排序



载入 sort4_riscv.hex



运行后 0、4、8、12、16 数据存储器中的数据升序排序

请问：该单周期 RISC-V 处理器的控制器能不能采用微程序控制器的方法设计？为什么

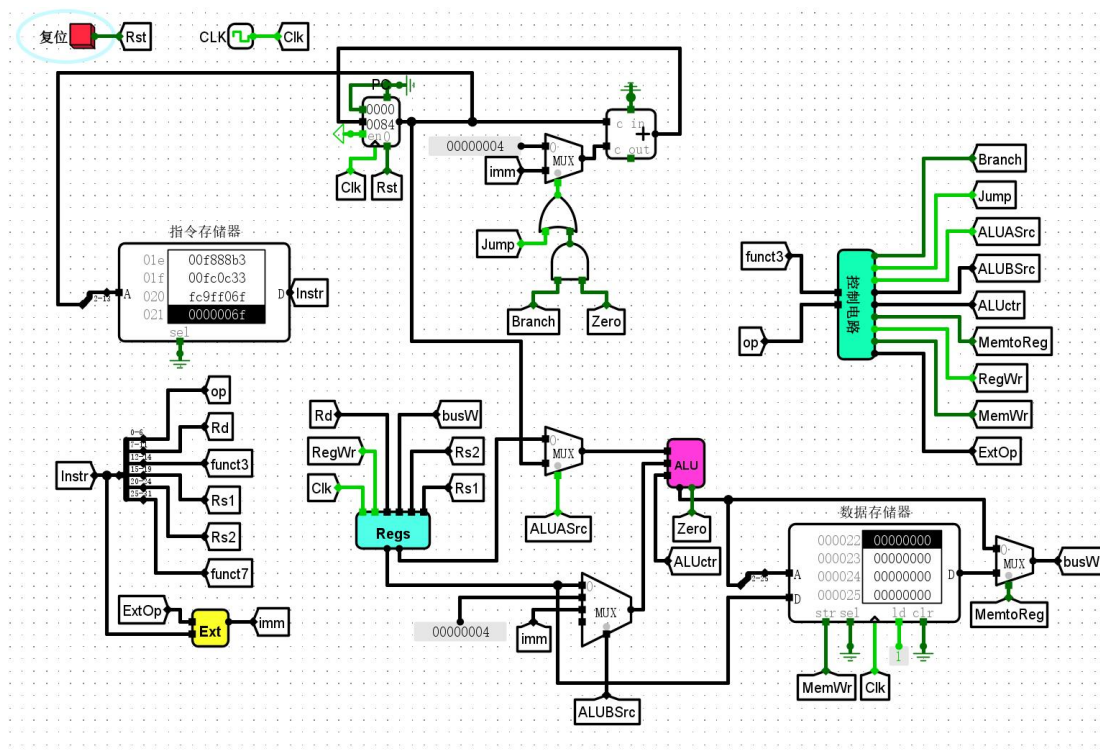
么？

答：不能。原因同 mips 处理器类似，使用微程序控制器均会增加时钟周期数。

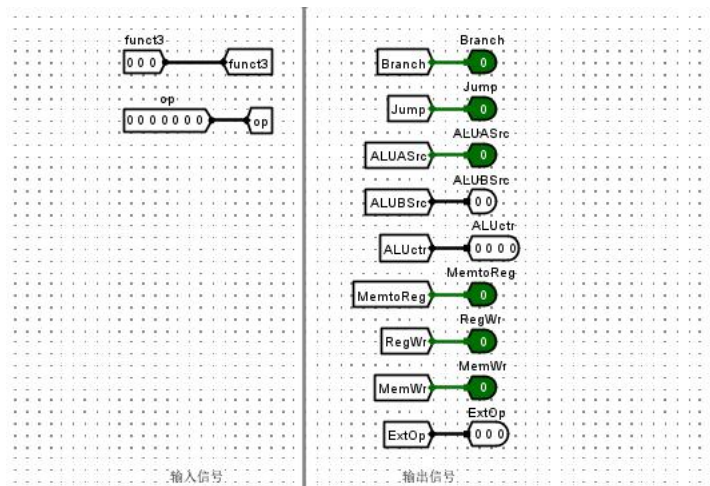
请分析单周期 RISC-V 处理器（硬布线控制器）的电路原理：

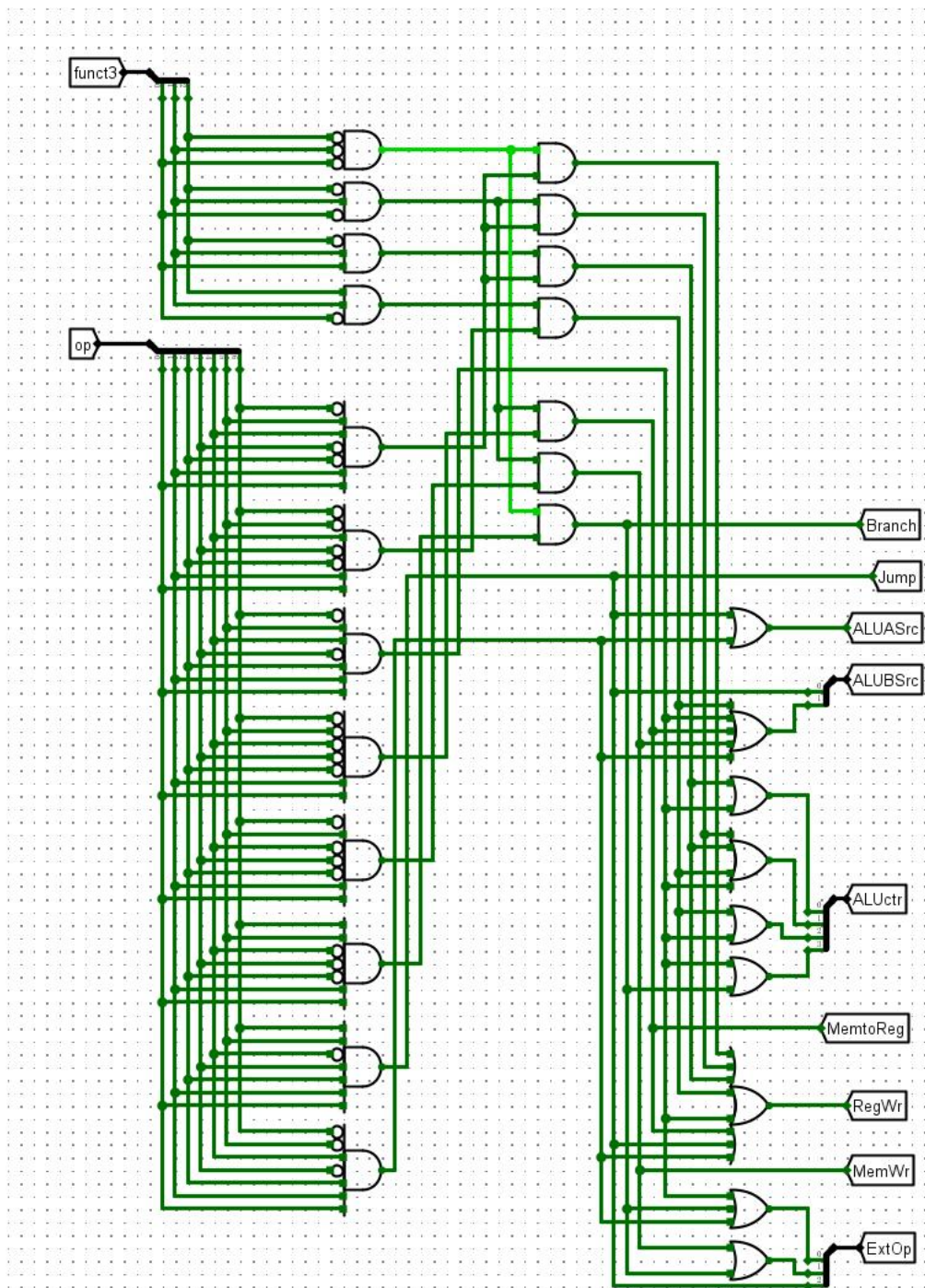
- 包括：数据通路、硬布线控制器、立即数扩展部件 Ext、算术逻辑单元 ALU 等电路。

答：

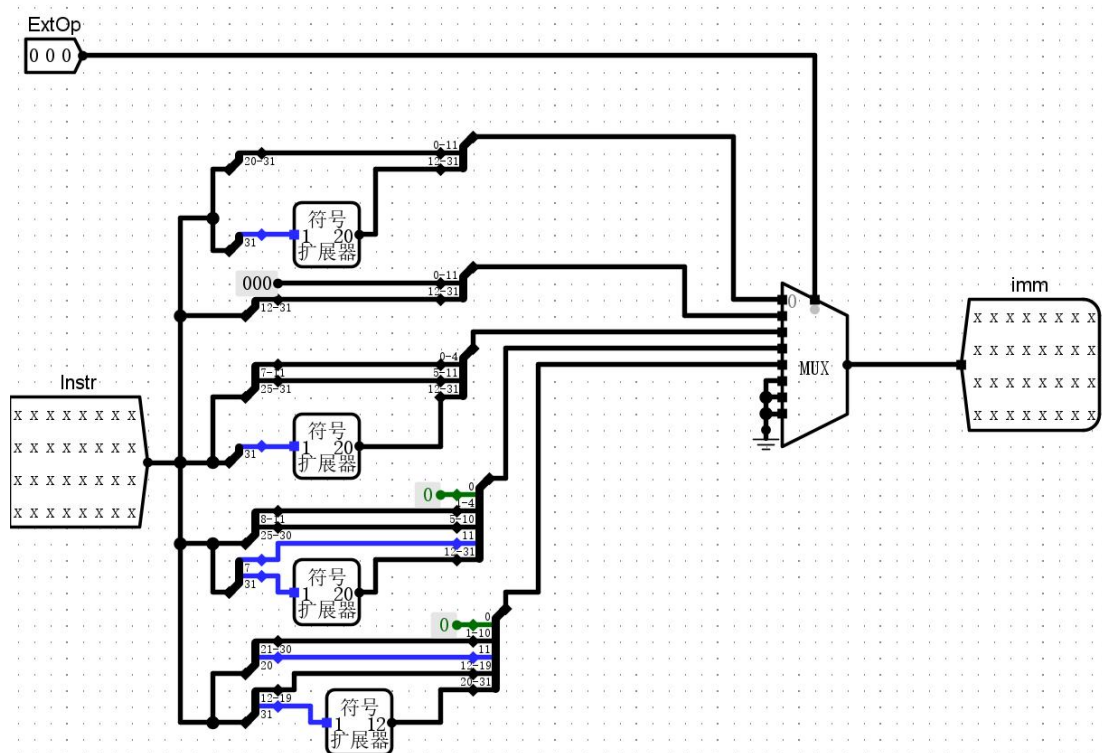


数据通路：PC 用寄存器实现，每轮自增 $PC=PC+4$ ，从指令存储器中取指令后存在 Instr 中，分解为 op, Rd, funct3, Rs1, Rs2, funct7, 并进行立即数扩展。op 和 funct3 输入给控制电路输出指令引脚。寄存器堆输入 2 个度出口的寄存器编号、1 个写入口的寄存器编号、写入数据，输出 2 两个读出口数据。读出数据经过多路选择器和 ALU 的运算后存入数据存储器中。

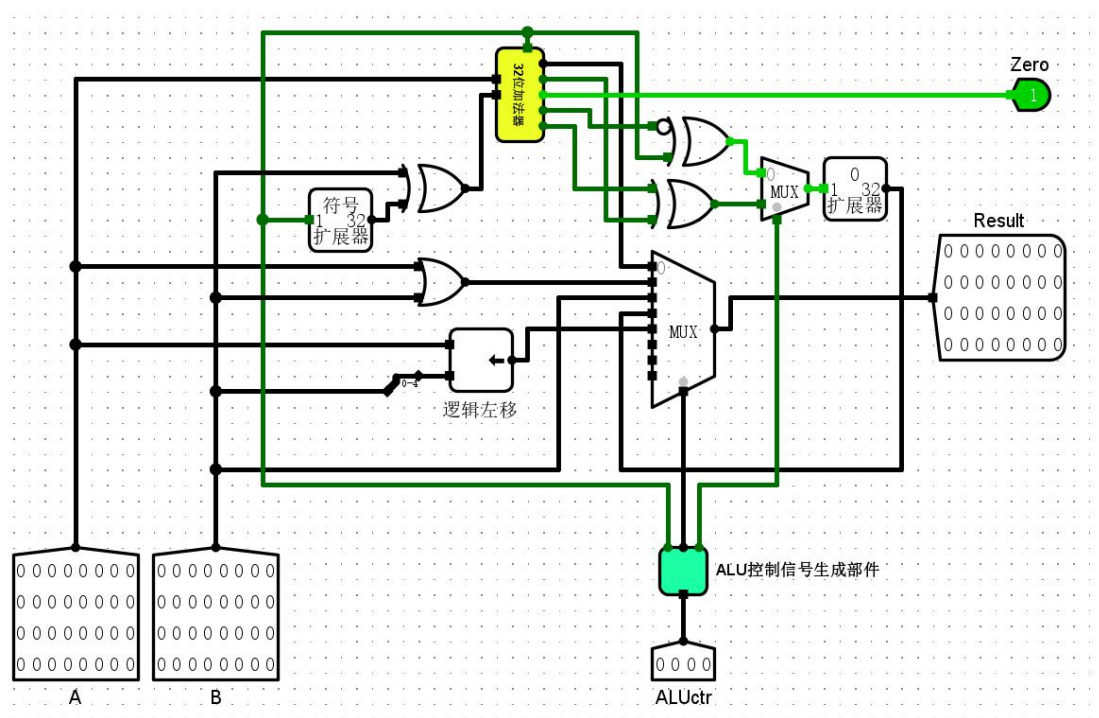




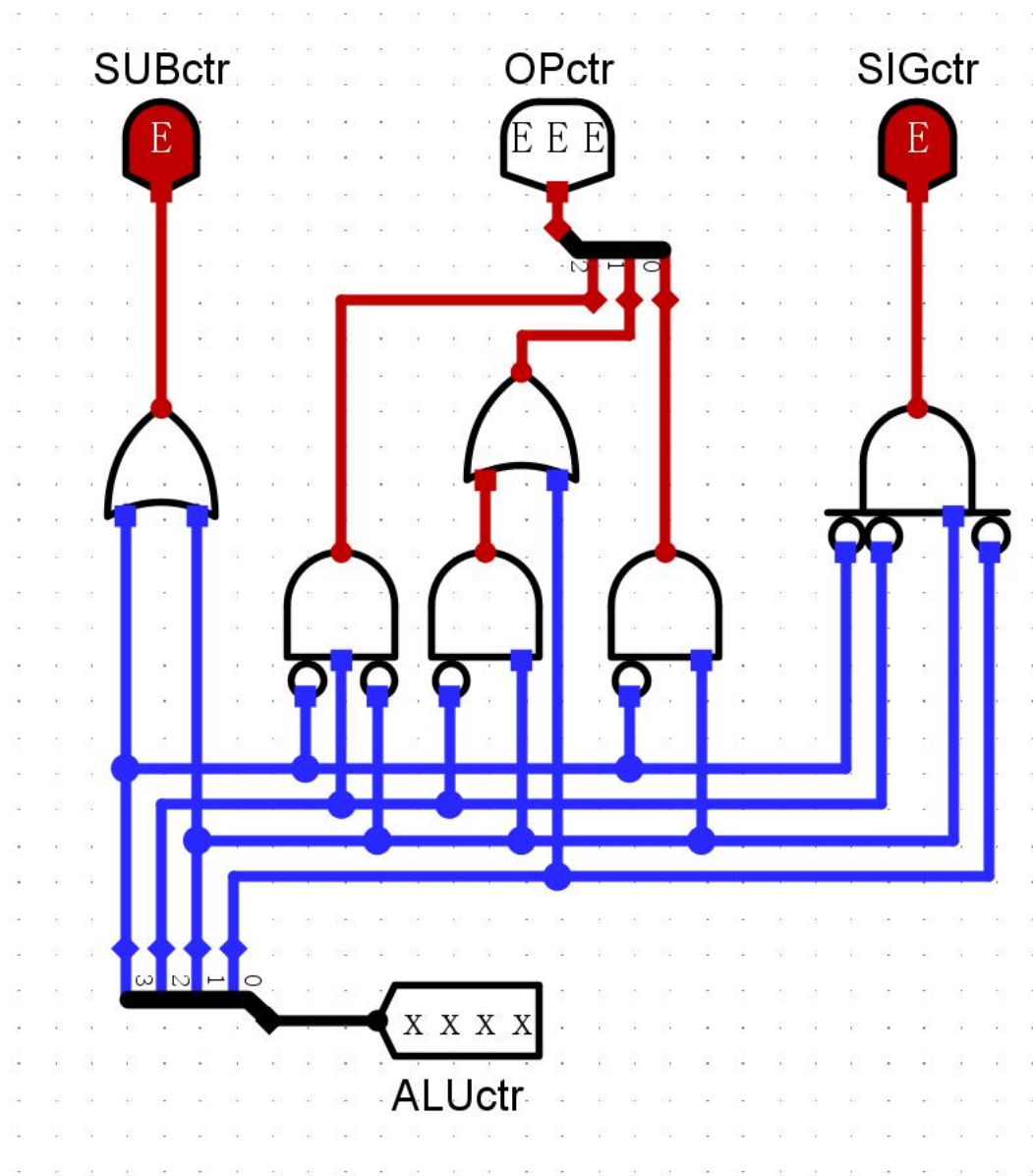
控制电路：输入 3 为的 funct3 和 7 位的 op，经过门电路运算后输出 9 种控制信号。



立即数扩展电路 EXT：根据 ExtOp 控制信号对输入的 instr 立即数进行扩展。



算术逻辑单元 ALU：



ALU 控制信号生成部件通过输入 4 位 ALUctr 获得 SUBctr, SIGctr, OPctr 三个信号。

SUBctr 控制 ALU 执行加法计算还是减法计算, SIGctr 控制 ALU 执行带符号整数比较小于置 1 还是无符号整数比较小于置 1, OPctr 作为多路选择器的选择信号, 控制选择哪种运算的结果输出, 包括加、按位或、操作数 B 选择和小于置 1 四种运算。

除完成实验课件要求内容外 (实验环节基本要求), 需回答下述问题 (实验报告采分点):

(1) 针对 test1.asm 程序段

```
loop1:
    addi $s4,$zero,2      #s4=2
    sw $s4,1032($zero)    #s4=2 保存到(1032, 即102单元)
```

这条指令不会执行到
这条指令不会执行到

为何上述两条指令不会执行到?

答：

```
lw $s1,1024($zero)    # $s1=8
lw $s2,1028($zero)    # $s2=9
```

取数指令令 $s1=8, s2=9$ 。

```
beq $s1,$s2,loop1    # $s1不等于$s2    执行下一条指令
```

当 $s1$ 等于 $s2$ 时，会跳进 $loop1$ 中，明显这里不相等，跳过指令。

```
beq $zero,$zero,loop2    # 转loop2
```

跳进 $loop2$ 中。

```
loop2:
    beq $zero,$zero,loop2    # 转loop2    死循环
```

$loop2$ 是死循环，程序会卡在这里，从而不会运行 $loop1$ 的指令。

(2) 为何安排死循环？若不采用死循环，针对此种情况，MIPS 架构还可以采用什么方式？（课外扩展）

```
loop2:
    beq $zero,$zero,loop2    # 转loop2    死循环
```

答：代码执行完后，程序不会自动停止运行，PC 会继续+1，将会运行未知指令，死循环能够作为程序运行完毕的标志，并保护目标存储单元免受未知程序的干扰。

使用系统调用终止程序：

```
addi $v0,$zero,10
```

```
syscall
```

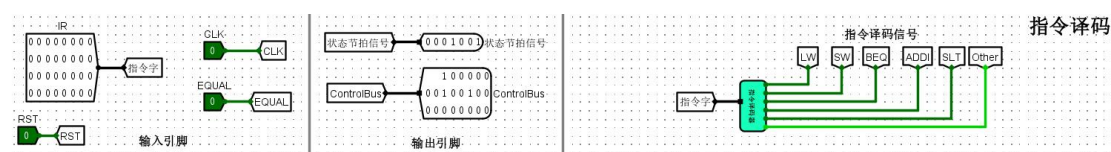
使用 `break` 或 `trap` 指令触发异常。

(3) 在该单总线结构 MIPS 处理器（硬布线控制器）的数据通路上，能够运行求累加和程序、计算费波那契数列程序吗？为什么？

答：不能。该数据通路仅支持 `addi` 立即数加法指令，缺少 `add` 寄存器加法指令，无法实现变量之间的加法。

(4) 针对单总线结构 MIPS 处理器（硬布线控制器）的下述电路进行分析。

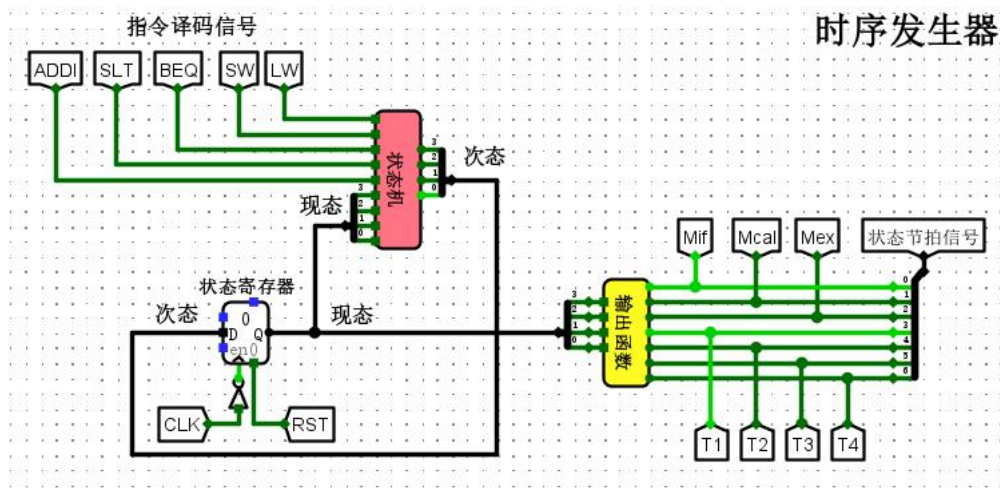
a) 硬布线控制器



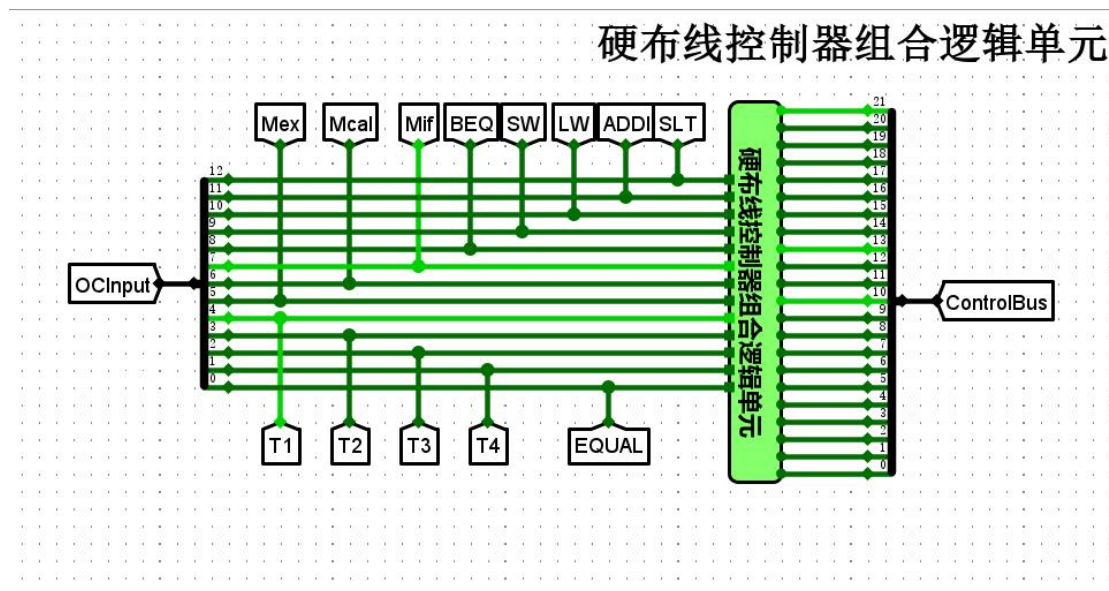
输入引脚：接受来自 IR 的 32 位指令字、CLK、EQUAL、RST。

输出引脚：输出 7 位状态节拍信号和 22 位 ControlBus。

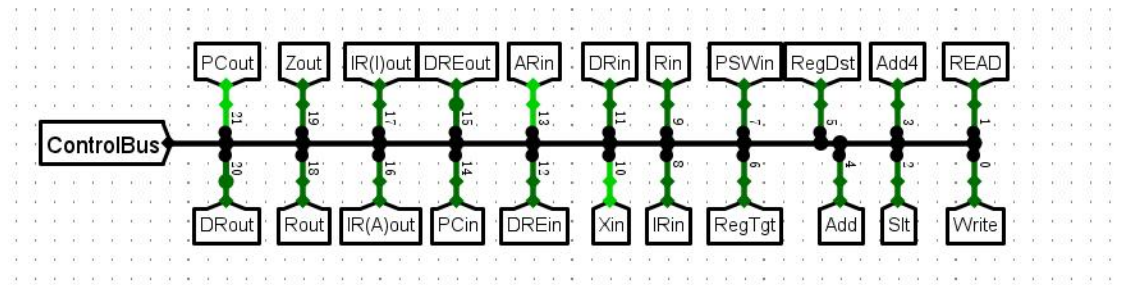
指令译码：将 IR 中的操作码解密，识别出当前是什么指令，电路支持的指令为 LW、SW、BEQ、ADDI、SLT 和 Other。



时序发生器：状态寄存器使用 D 触发器，保存现态，状态机模块根据当前状态和指令信号，计算次态，输出函数将现态解码为具体的状态节拍信号，以及微操作阶段标志。

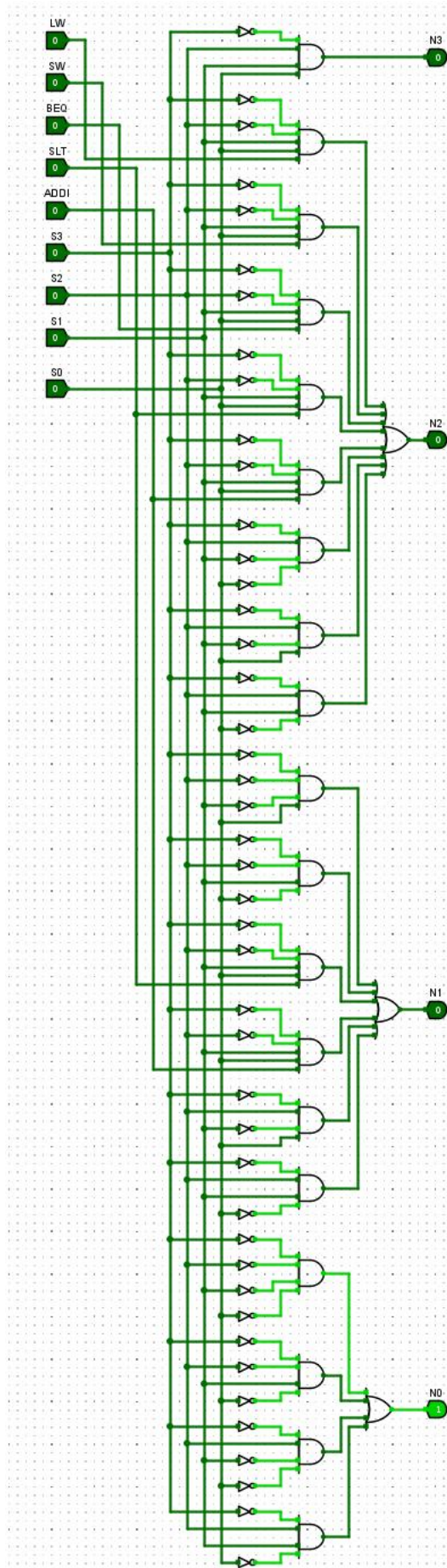


组合逻辑单元：根据时序状态信号、指令译码信号和 EQUAL 条件标志，输出一根控制总线，用来决定哪些微操作控制信号需要置 1。



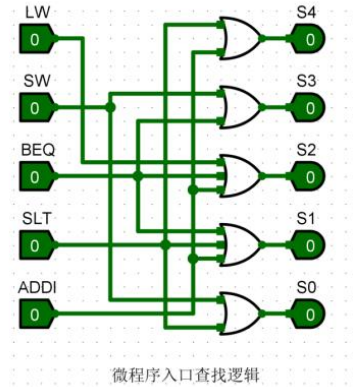
总线分配：将 22 位微操作信号分流到具体的 CPU 内部控制引脚上。

b) 状态机



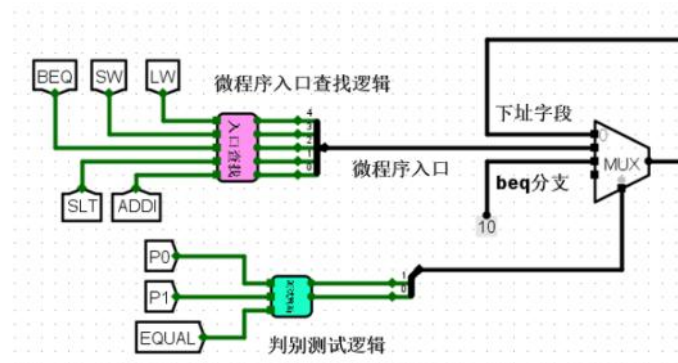
(5) 分析单总线结构 MIPS 处理器（微程序控制器）的下述电路原理

a) 微程序入口查找逻辑



L	S	BE	SL	AD	S	S	S	S	S
W	W	Q	T	DI	4	3	2	1	0
1					0	0	1	0	0
	1				0	1	0	0	1
		1			0	1	1	1	0
			1		1	0	0	1	1
				1	1	0	1	1	0

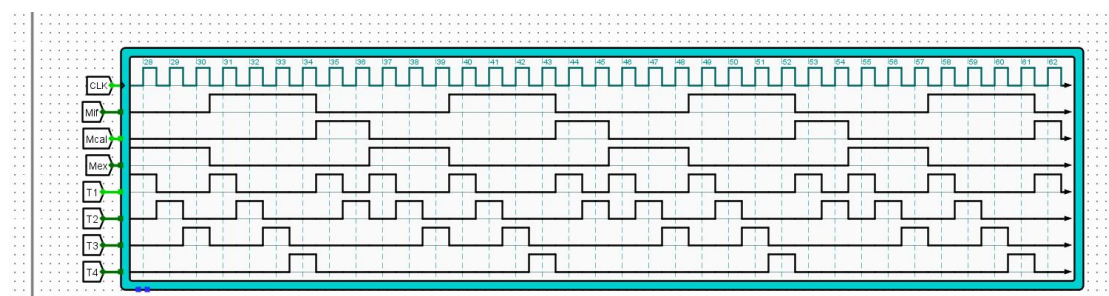
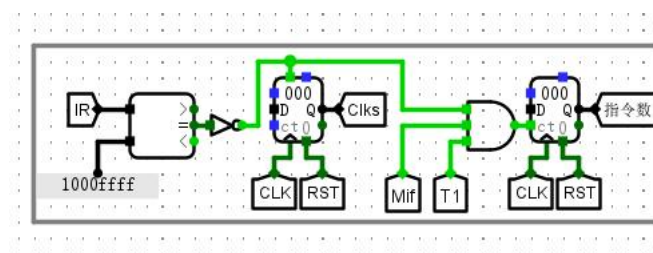
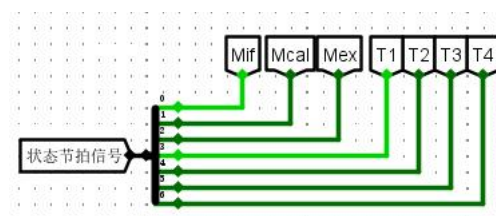
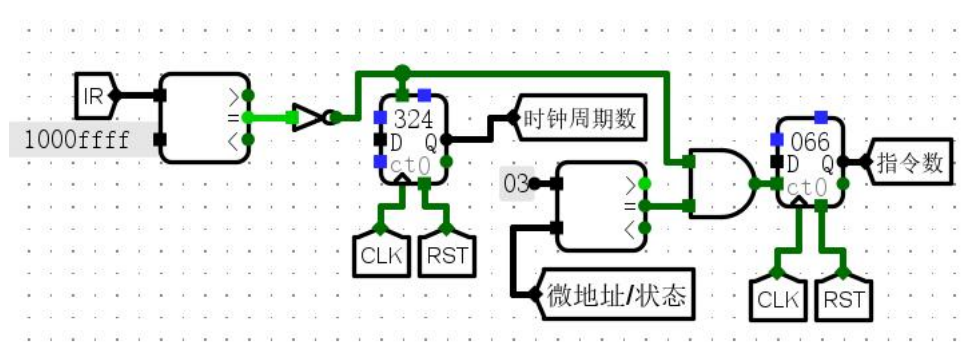
b) 判别测试逻辑



P0	P1	EQUAL	M1	M0
0	0	0	0	0
0	0	1	0	0

0	1	0	0	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1

(6) 结合工作原理，比较下述微程序控制器和硬布线控制器引脚上的不同



主要区别在于输出的引脚，微程序控制器输出微地址/状态，即当前执行微指令的物理地址，通过判断其是否等于特定的微地址 0x03 来进行计数，硬布线控制器输出状态节拍信号，包含具体的时序步进节拍（T1~T4）和机器周期标志（Mif, Mcal, Mex）。

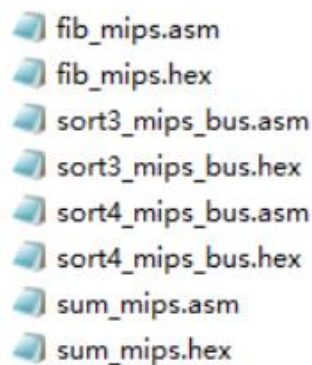
(7) 请在单总线结构 MIPS 处理器（微程序控制器）的数据通路上运行程序：

test1.hex、 sort3_mips_bus.hex、 sort4_mips_bus.hex

观看 RAM 存储器中的结果是否正确？

答：见验证实验，结果均正确。

(8) 针对单周期 MIPS 处理器，测试程序，观察结果（拷屏+文字解释）



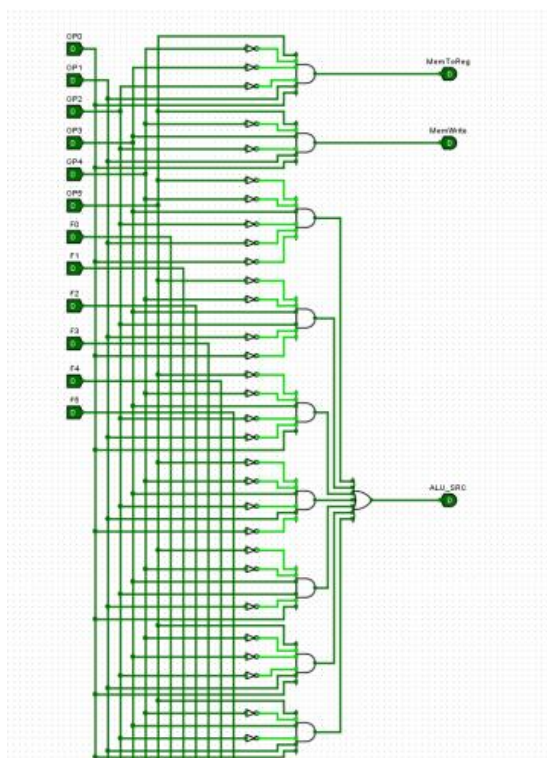
A screenshot of a file explorer window showing a list of files. The files are: fib_mips.asm, fib_mips.hex, sort3_mips_bus.asm, sort3_mips_bus.hex, sort4_mips_bus.asm, sort4_mips_bus.hex, sum_mips.asm, and sum_mips.hex. Each file has a small icon to its left, representing a document or file.

答：见验证实验（3）（4）。

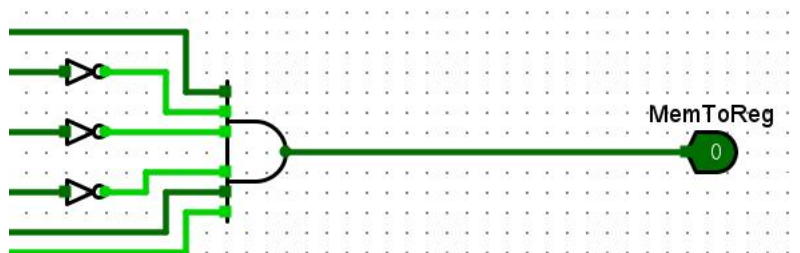
(9) 该单周期 MIPS 处理器的控制器能不能采用微程序控制器的方法设计？为什么？

答：不能。微程序取指令、指令译码、取数过程就要消耗一个时钟周期，而单周期处理器的所有指令在一个时钟周期内完成。

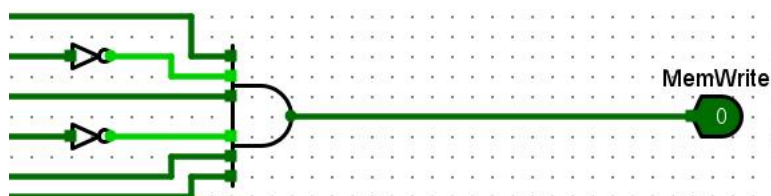
(10) 分析单周期 MIPS 处理器控制信号的生成电路原理（输入输出的逻辑，拷屏+文字解释）



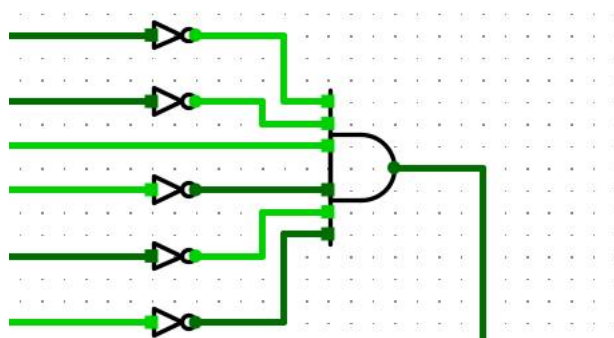
答:



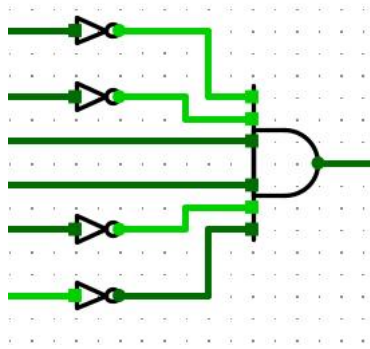
$$\text{MemToReg} = \text{OP0} \wedge \text{OP1} \wedge (\neg \text{OP2}) \wedge (\neg \text{OP3}) \wedge (\neg \text{OP4}) \wedge \text{OP5}$$



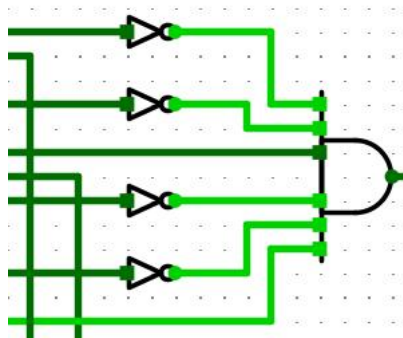
$$\text{MemWrite} = \text{OP0} \wedge \text{OP1} \wedge (\neg \text{OP2}) \wedge \text{OP3} \wedge (\neg \text{OP4}) \wedge \text{OP5}$$



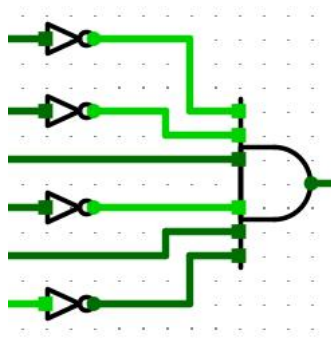
$$\text{中间节点 M1} = (\neg \text{OP0}) \wedge (\neg \text{OP1}) \wedge (\neg \text{OP2}) \wedge \text{OP3} \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5})$$



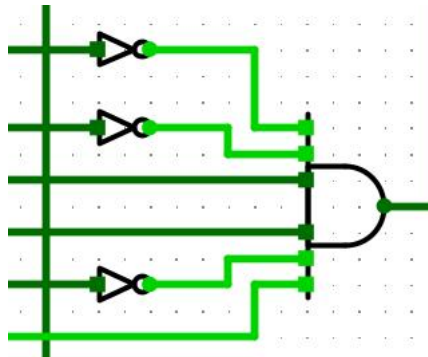
中间节点 $M2 = (\neg OP0) \wedge (\neg OP1) \wedge OP2 \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



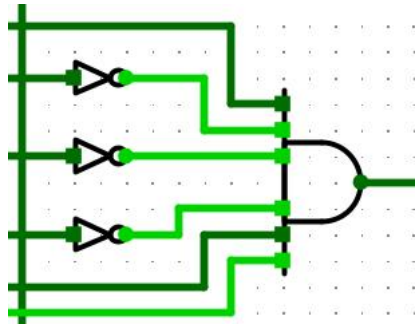
中间节点 $M3 = OP0 \wedge (\neg OP1) \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



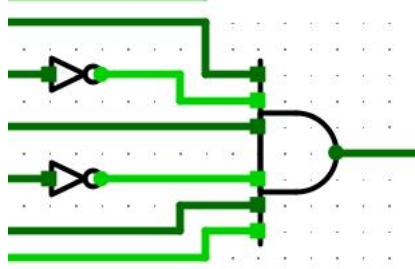
中间节点 $M4 = (\neg OP0) \wedge OP1 \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



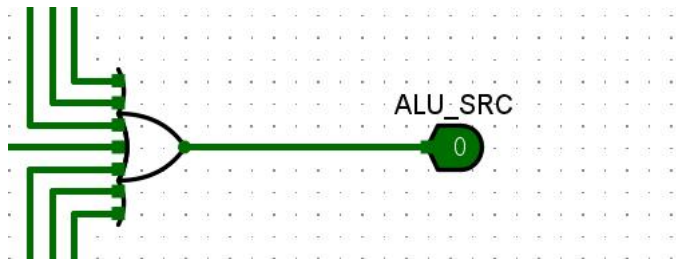
中间节点 $M5 = OP0 \wedge (\neg OP1) \wedge OP2 \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



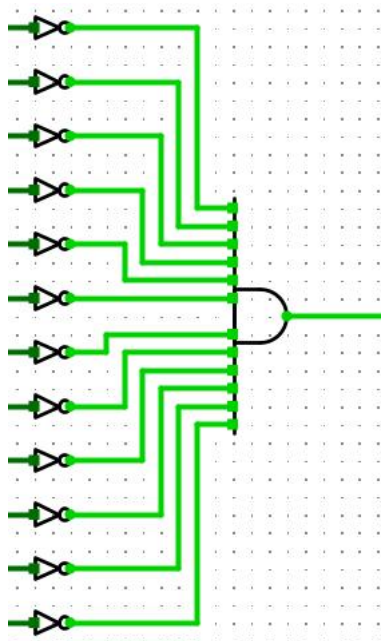
中间节点 $M6=OP0 \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge OP5$



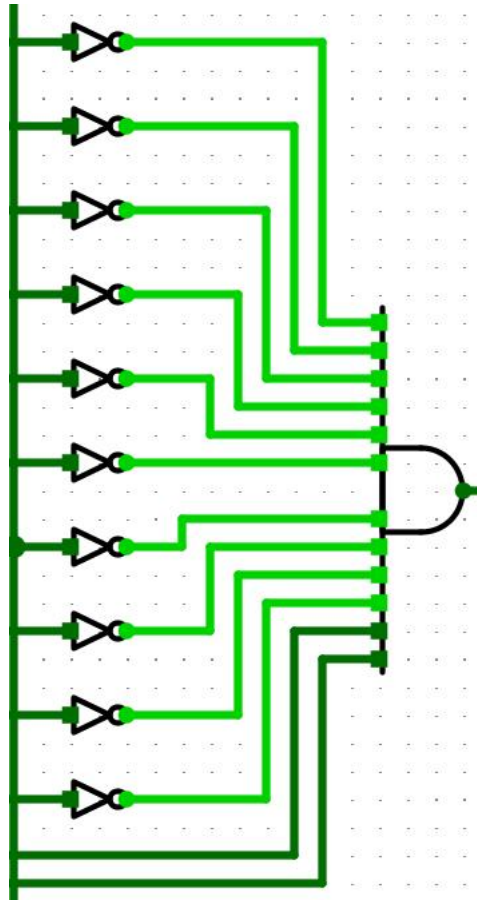
中间节点 $M7=OP0 \wedge OP1 \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge OP5$



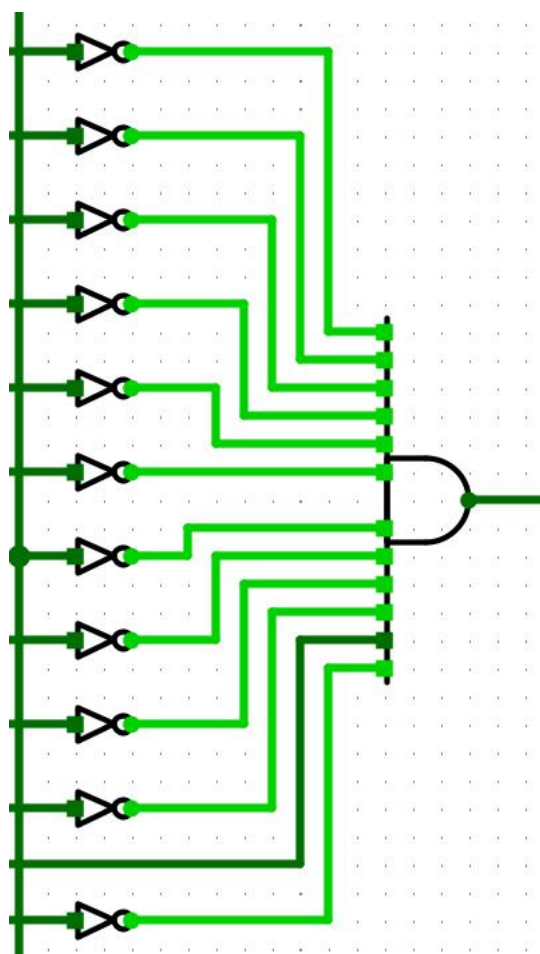
$ALU_SRC=M1 \vee M2 \vee M3 \vee M4 \vee M5 \vee M6 \vee M7$



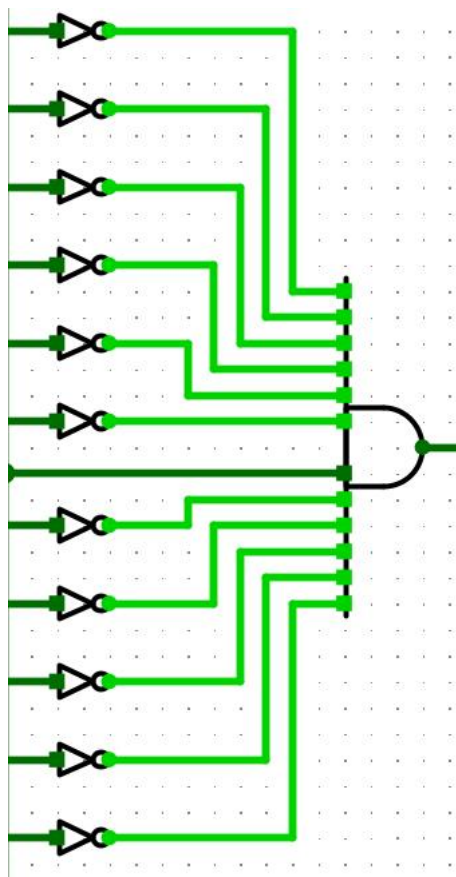
中间节点 $N1 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge (\neg F1) \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge (\neg F5)$



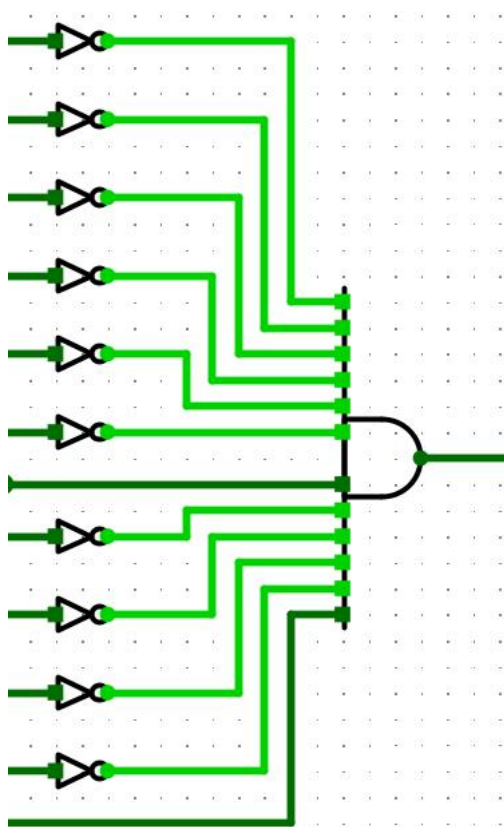
中间节点 $N2 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge F0 \wedge F1 \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge (\neg F5)$



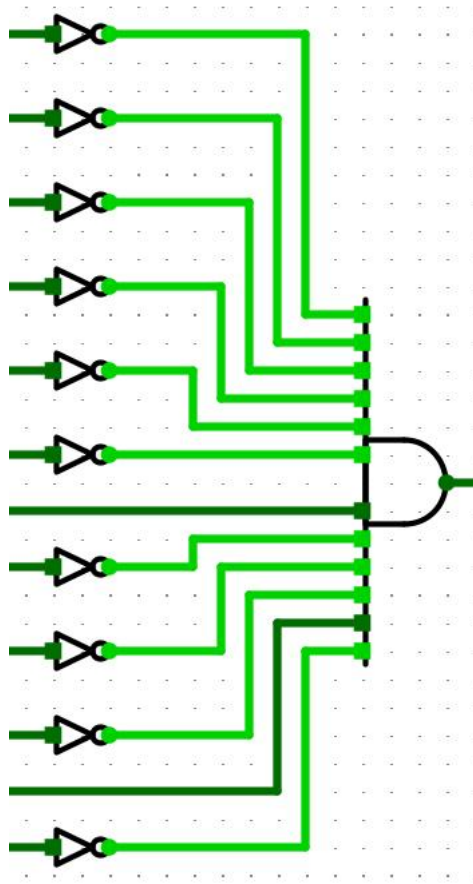
中间节点 $N3 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge$
 $F1 \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge (\neg F5)$



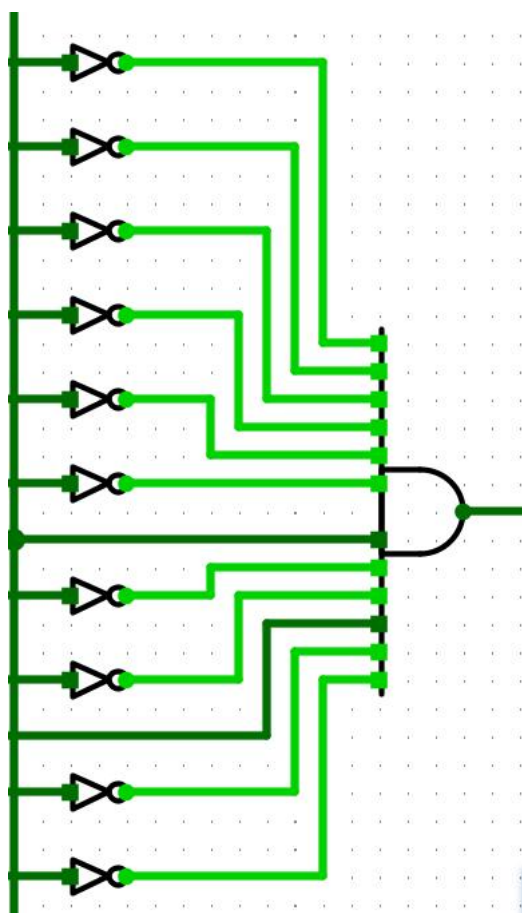
中间节点 $N4 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge$
 $(\neg F1) \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



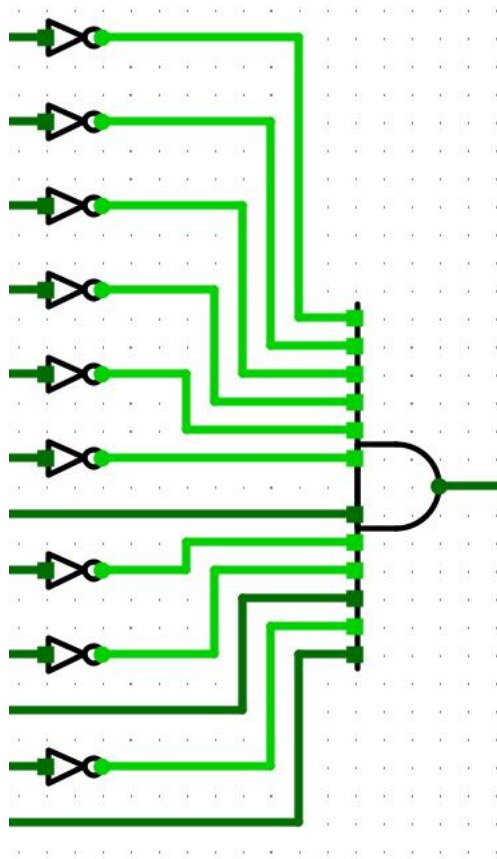
中间节点 $N5 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge F0 \wedge (\neg F1) \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



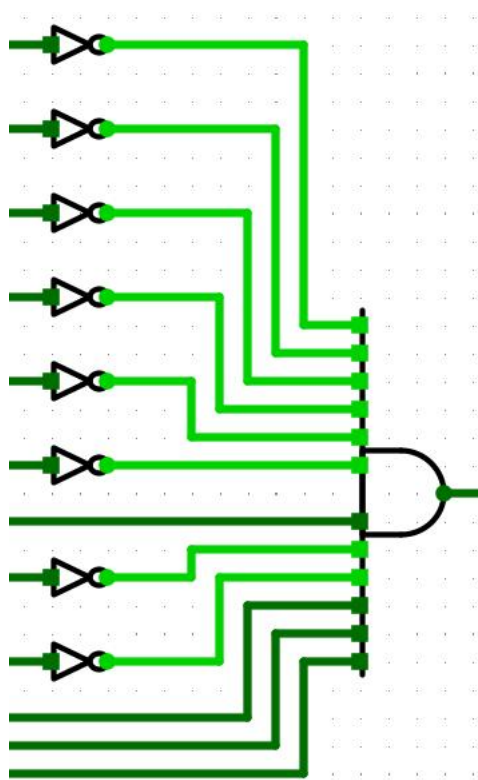
中间节点 $N6 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge F1 \wedge (\neg F2) \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



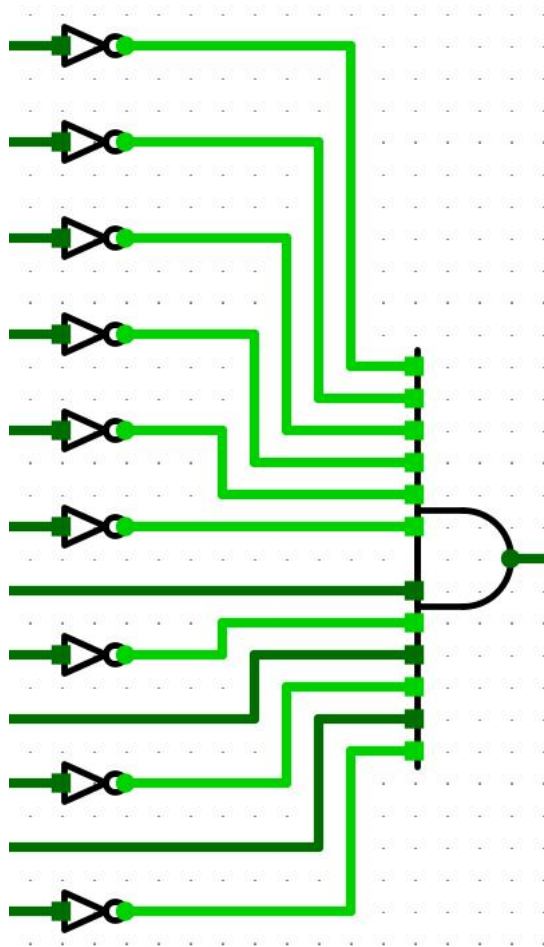
中间节点 $N7 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge (\neg F1) \wedge F2 \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



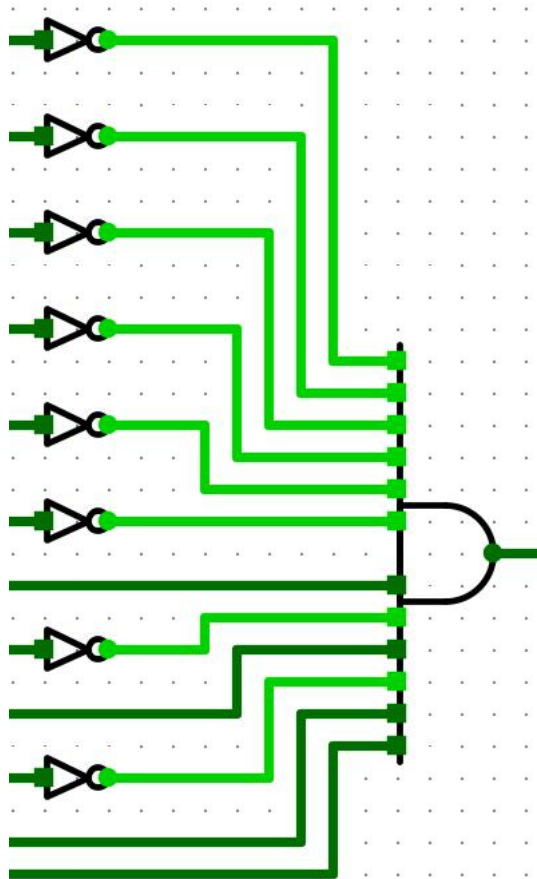
中间节点 $N8 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge F0 \wedge (\neg F1) \wedge F2 \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



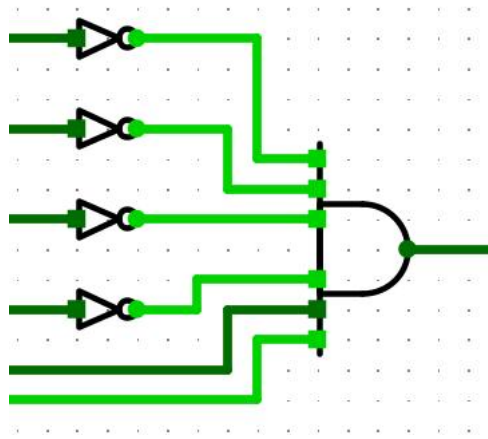
中间节点 $N9 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge F0 \wedge F1$
 $\wedge F2 \wedge (\neg F3) \wedge (\neg F4) \wedge F5$



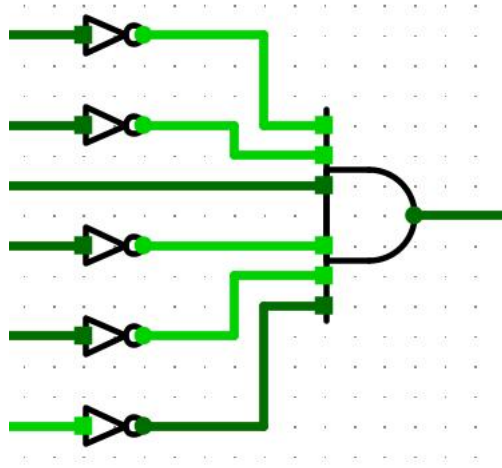
中间节点 $N10 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge$
 $F1 \wedge (\neg F2) \wedge F3 \wedge (\neg F4) \wedge F5$



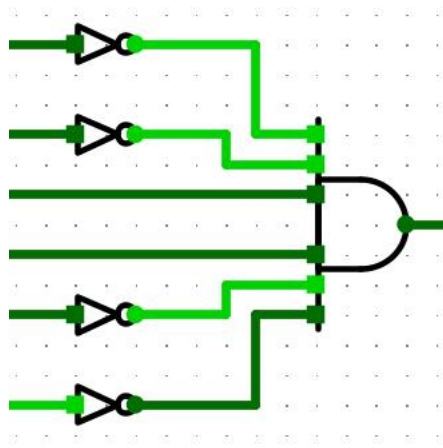
中间节点 $N11 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge F0 \wedge F1$
 $\wedge (\neg F2) \wedge F3 \wedge (\neg F4) \wedge F5$



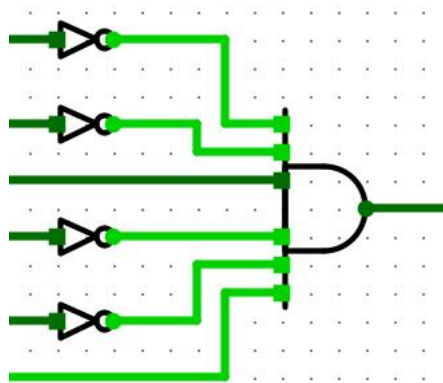
中间节点 $N12 = OP0 \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5)$



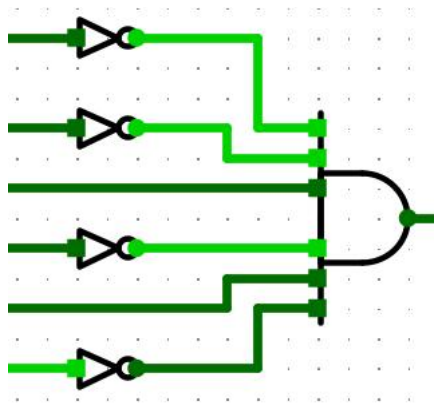
中间节点 $N13 = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



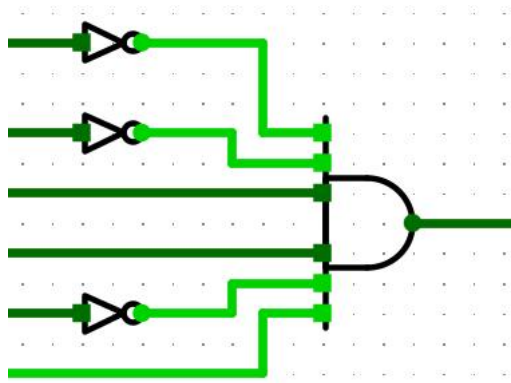
中间节点 $N14 = (\neg OP0) \wedge (\neg OP1) \wedge OP2 \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



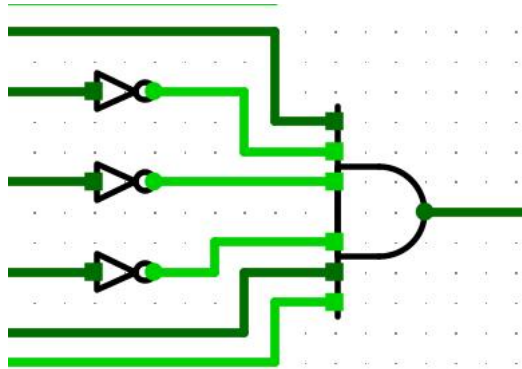
中间节点 $N15 = OP0 \wedge (\neg OP1) \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



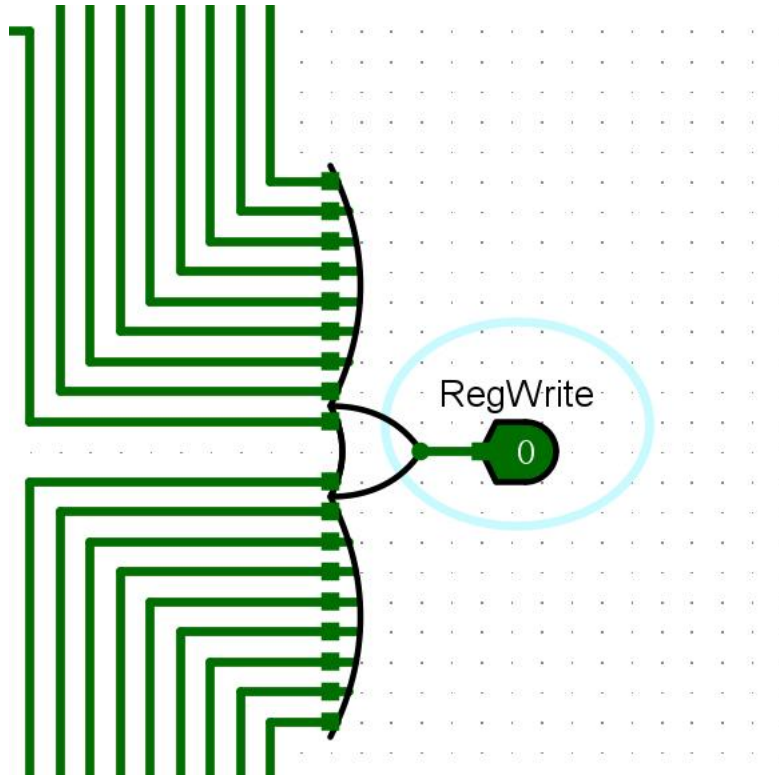
中间节点 $N16 = (\neg OP0) \wedge OP1 \wedge (\neg OP2) \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



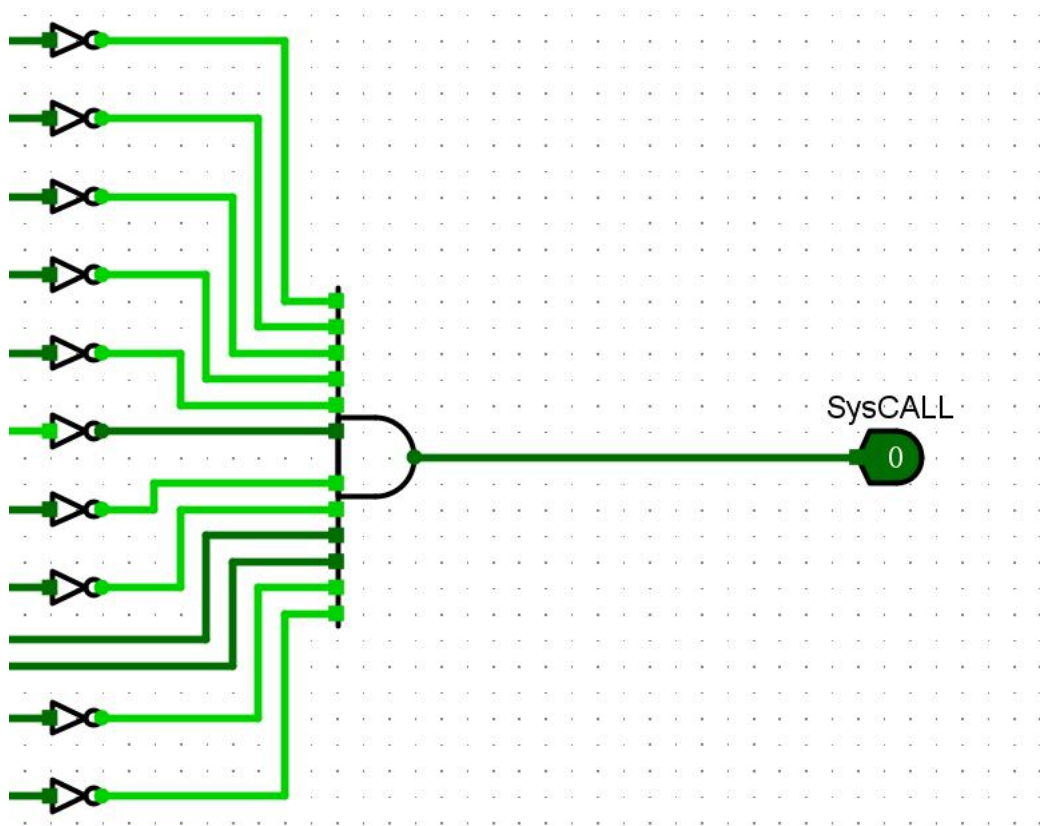
中间节点 $N17 = OP0 \wedge (\neg OP1) \wedge OP2 \wedge OP3 \wedge (\neg OP4) \wedge (\neg OP5)$



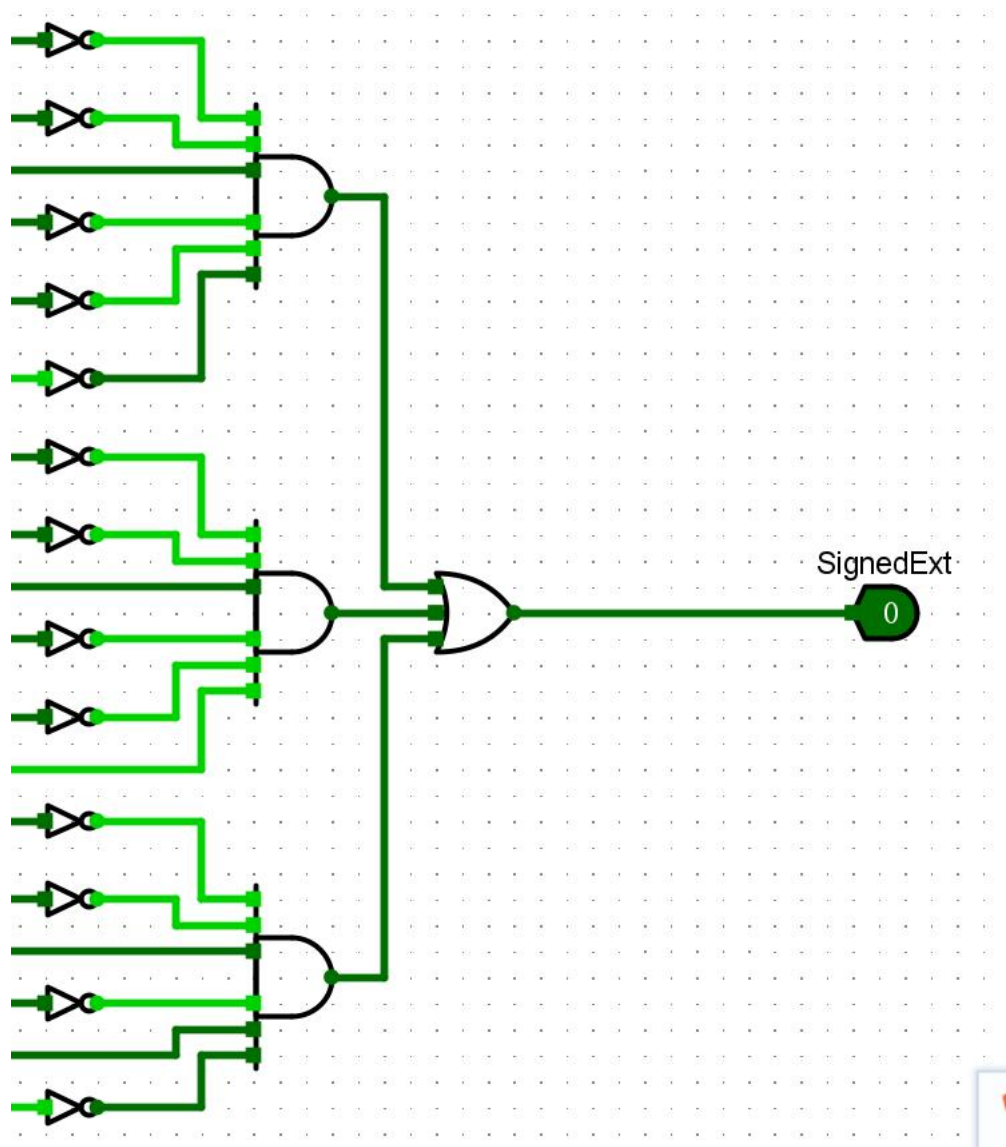
中间节点 $N18 = OP0 \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge OP5$



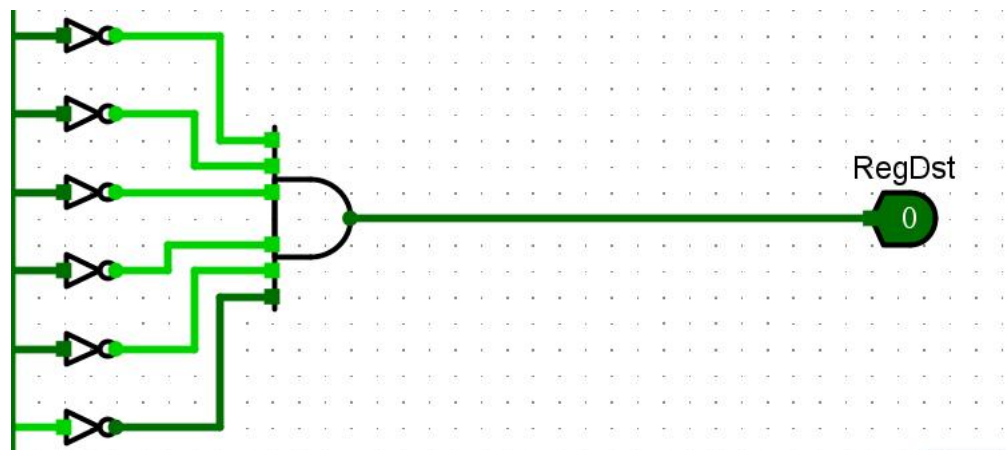
$$\text{RegWrite} = N1 \vee N2 \vee N3 \vee N4 \vee N5 \vee N6 \vee N7 \vee N8 \vee N9 \vee N10 \vee N11 \vee N12 \vee \\ N13 \vee N14 \vee N15 \vee N16 \vee N17 \vee N18$$



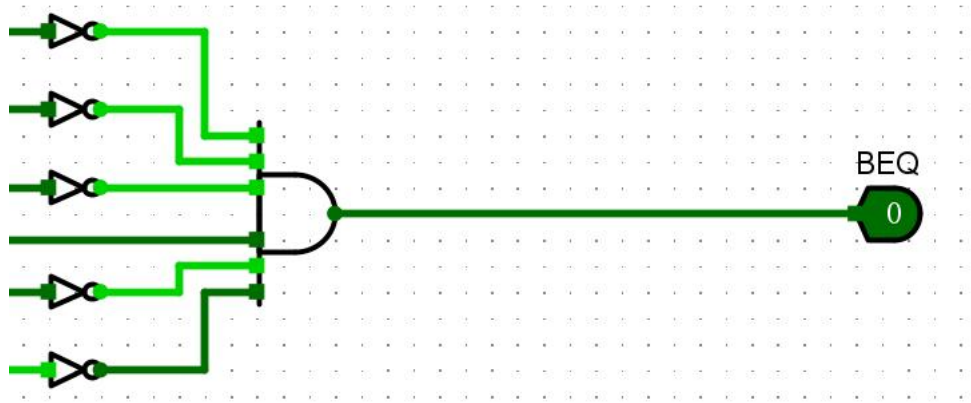
$$\text{SysCALL} = (\neg \text{OP0}) \wedge (\neg \text{OP1}) \wedge (\neg \text{OP2}) \wedge (\neg \text{OP3}) \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5}) \wedge (\neg \text{F0}) \wedge (\neg \text{F1}) \\ \wedge \text{F2} \wedge \text{F3} \wedge (\neg \text{F4}) \wedge (\neg \text{F5})$$



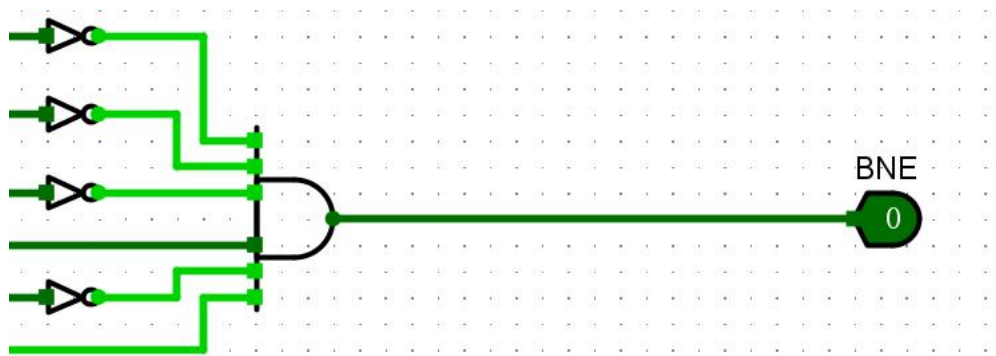
$$\text{SignedExt} = (\neg \text{OP0}) \wedge (\neg \text{OP1}) \wedge (\neg \text{OP2}) \wedge \text{OP3} \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5}) \vee \text{OP0} \wedge (\neg \text{OP1}) \wedge (\neg \text{OP2}) \wedge \text{OP3} \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5}) \vee (\neg \text{OP0}) \wedge \text{OP1} \wedge (\neg \text{OP2}) \wedge \text{OP3} \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5})$$



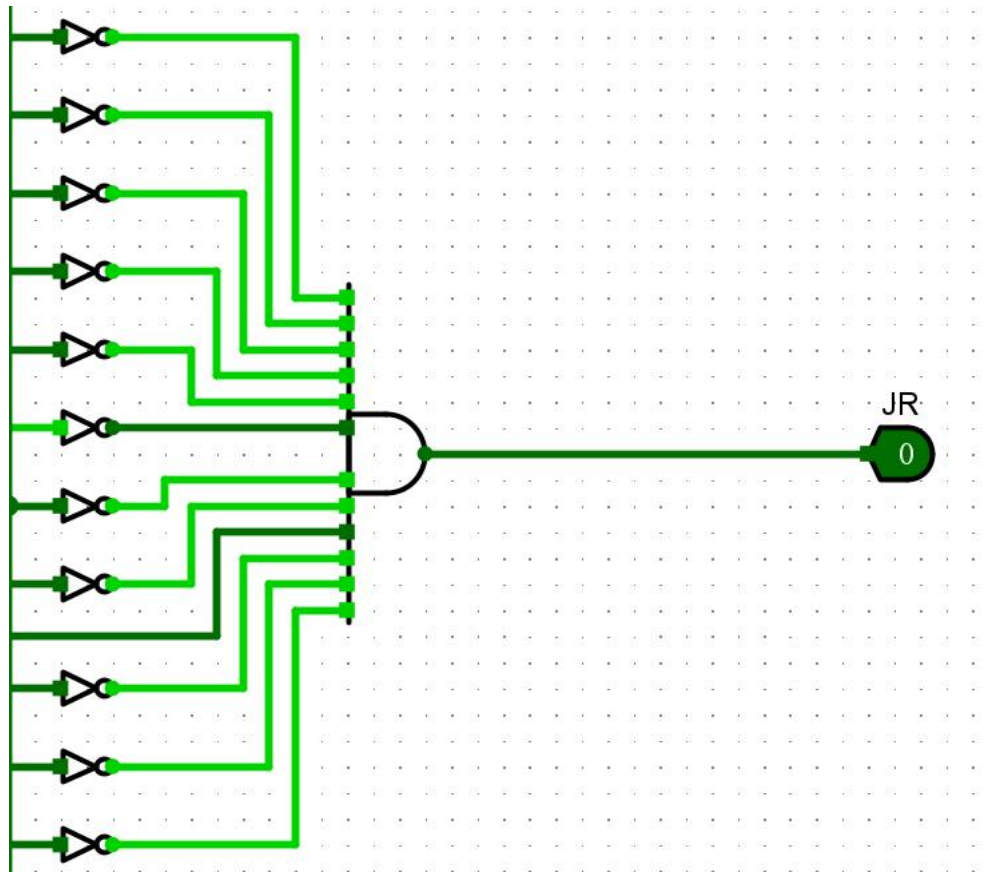
$$\text{RegDst} = (\neg \text{OP0}) \wedge (\neg \text{OP1}) \wedge (\neg \text{OP2}) \wedge (\neg \text{OP3}) \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5})$$



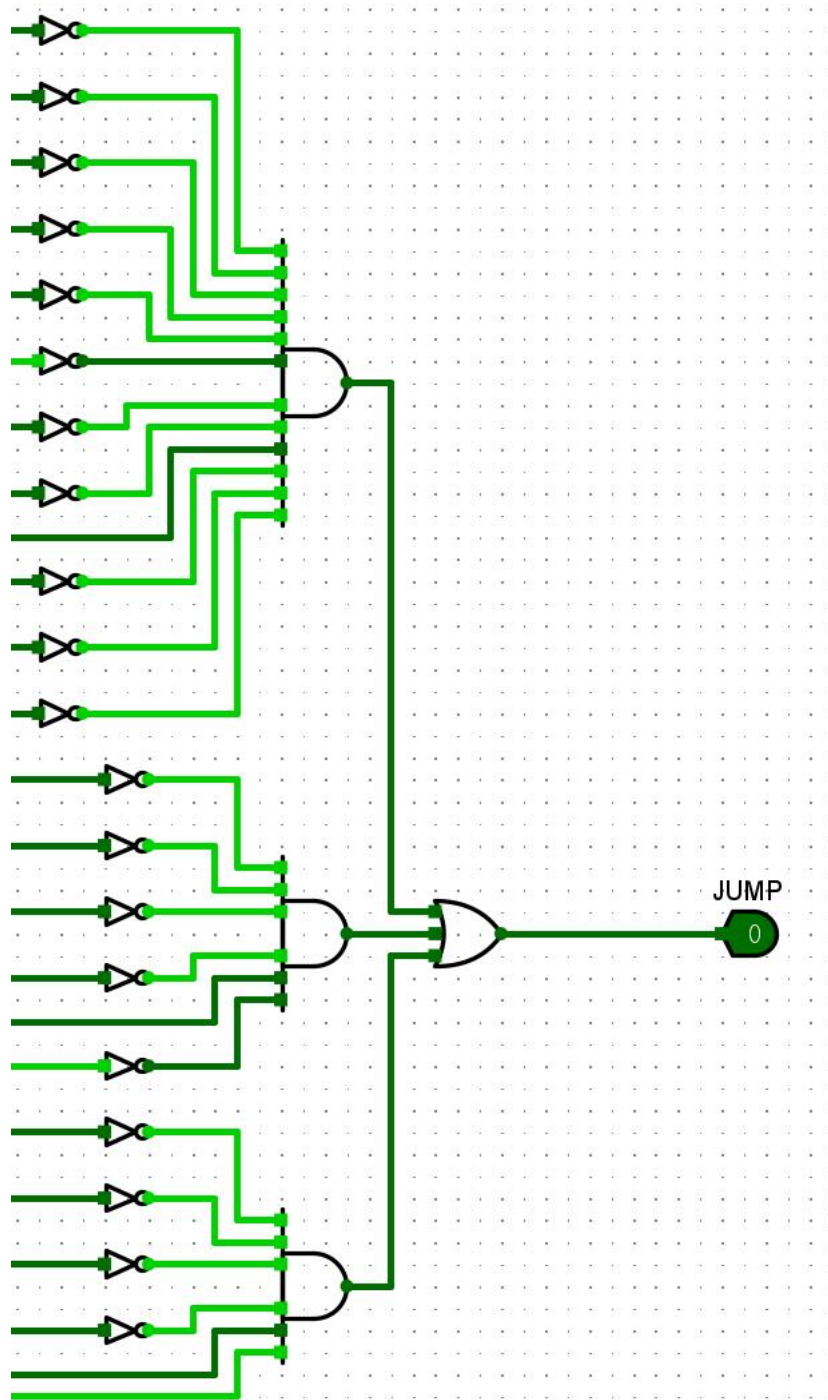
$$\text{BEQ} = (\neg \text{OP0}) \wedge (\neg \text{OP1}) \wedge \text{OP2} \wedge (\neg \text{OP3}) \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5})$$



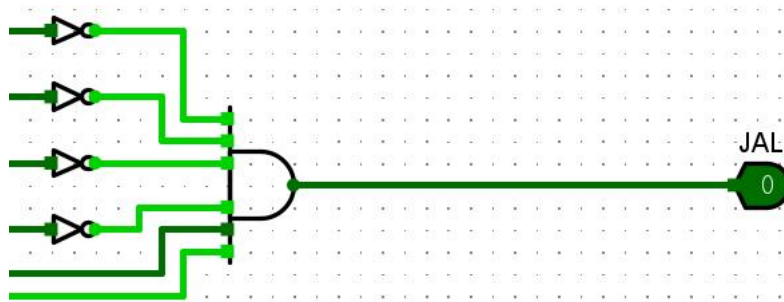
$$\text{BNE} = \text{OP0} \wedge (\neg \text{OP1}) \wedge \text{OP2} \wedge (\neg \text{OP3}) \wedge (\neg \text{OP4}) \wedge (\neg \text{OP5})$$



$$JR = (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge (\neg F1) \wedge (\neg F2) \wedge F3 \wedge (\neg F4) \wedge (\neg F5)$$



$$\begin{aligned} JUMP = & (\neg OP0) \wedge (\neg OP1) \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \wedge (\neg F0) \wedge (\neg F1) \wedge \\ & (\neg F2) \wedge F3 \wedge (\neg F4) \wedge (\neg F5) \vee (\neg OP0) \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \\ & \vee OP0 \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5) \end{aligned}$$



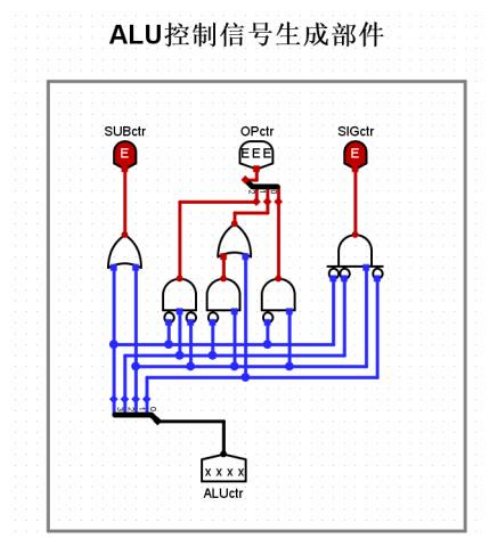
$$JAL = OP0 \wedge OP1 \wedge (\neg OP2) \wedge (\neg OP3) \wedge (\neg OP4) \wedge (\neg OP5)$$

- (11) 单周期 RISC-V 处理器的控制器能不能采用微程序控制器的方法设计？为什么？

答：不能。原因同 mips 处理器类似，使用微程序控制器均会增加时钟周期数。

- (12) 请分析单周期 RISC-V 处理器（硬布线控制器）的下述电路原理：

a) ALU 控制信号生成部件



用 A_i 表示 ALUctr 的第 i 位

$$SUBctr = A1 \vee A3$$

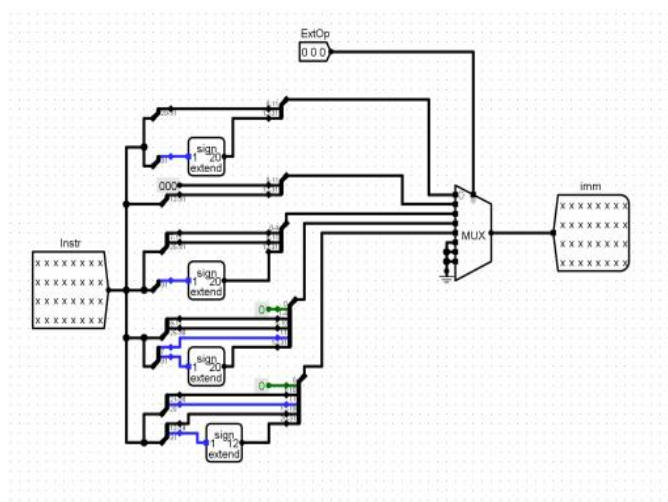
$$OPctr0 = A1 \wedge (\neg A3)$$

$$OPctr1 = A1 \wedge (\neg A2) \vee A0$$

$$OPctr2 = (\neg A1) \wedge A2 \wedge (\neg A3)$$

$$SIGctr = (\neg A1) \wedge A1 \wedge (\neg A2) \wedge (\neg A3)$$

b) 立即数扩展部件



ExtOP=000: 后 12 位变为前 12 位，后补第 31 位符号。

ExtOP=001: 前 12 位补 0，后不变。

ExtOP=010: 第 7-11 位改为第 0-4 位，第 25-31 位改为第 5-11 位，后补第 31 位符号。

ExtOP=011: 第 0 位为 0，第 8-11 位改为第 1-4 位，第 25-30 位改为第 5-10 位，第 7 位改为第 11 位，后补第 31 位符号。

ExtOP=100: 第 0 位为 0，第 21-30 位改为第 1-10 位，第 20 位改为第 11 位，第 12-19 位不变，后补第 31 位符号。

建议：针对电路分析题目，可以给出组合逻辑表达式或结合具体实例分析输入输出关系，从而支持其功能实现。

3.2 设计实验

单总线结构 MIPS 处理器（微程序控制器）（增加 1 条 add 指令，共 6 条指令）

（1）按照实验课件的要求，完成设计任务，提交设计文件单总线 MIPS 处理器-微程序.circ;

填写“微程序入口查找逻辑自动生成（6 条指令）（空表）.xlsx”，并生成 6 条指令的微程序入口查找逻辑公式。

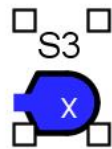
由“微程序自动生成（add 指令）（空表）.xlsx”知 ADD 指令的入口地址为 25。

机器指令译码信号						微程序入口地址					
LW	SW	BEQ	SLT	ADDI	ADD	入口地址 10进制	S4	S3	S2	S1	S0
1						4	0	0	1	0	0
	1					9	0	1	0	0	1
		1				14	0	1	1	1	0
			1			19	1	0	0	1	1
				1		22	1	0	1	1	0
					1	25	1	1	0	0	1

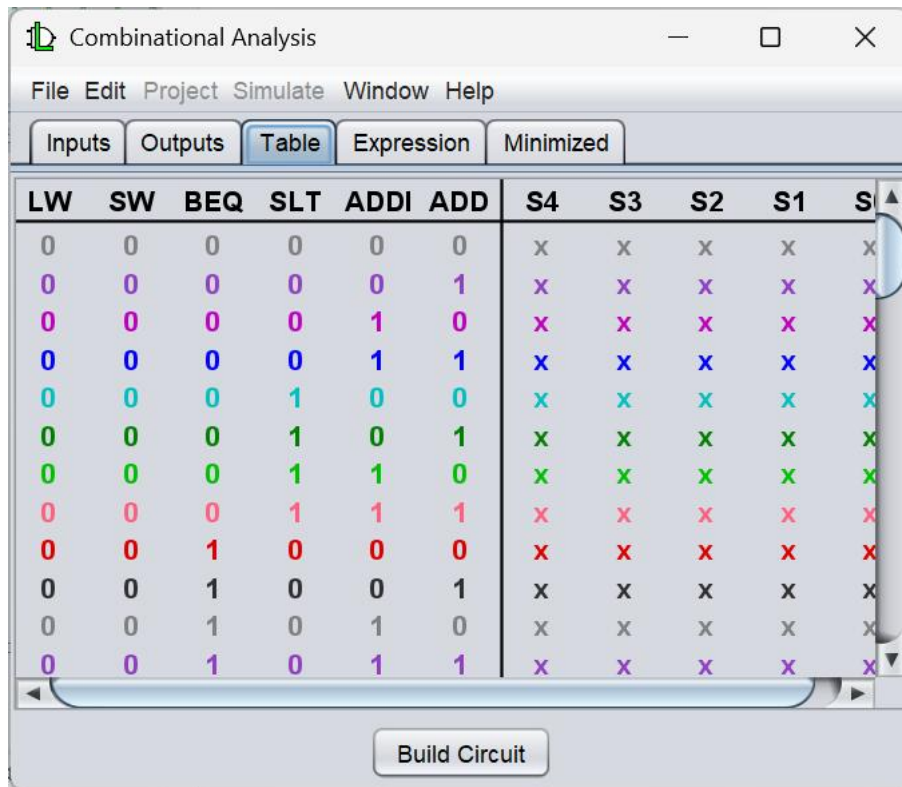
LW	SW	BEQ	SLT	ADDI	ADD	最小项表达式	S4	S3	S2	S1	S0
LW&						LW			LW+		
	SW&					SW		SW+			SW+
		BEQ&				BEQ		BEQ+	BEQ+	BEQ+	
			SLT&			SLT	SLT+			SLT+	SLT+
				ADDI&		ADDI	ADDI+		ADDI+	ADDI+	
					ADD&	ADD	ADD+	ADD+			ADD+
							SLT+ADDI+ADD	SW+BEQ+ADD	LW+BEQ+ADDI	BEQ+SLT+ADDI	SW+SLT+ADD

在 Logisim 上生成逻辑电路。

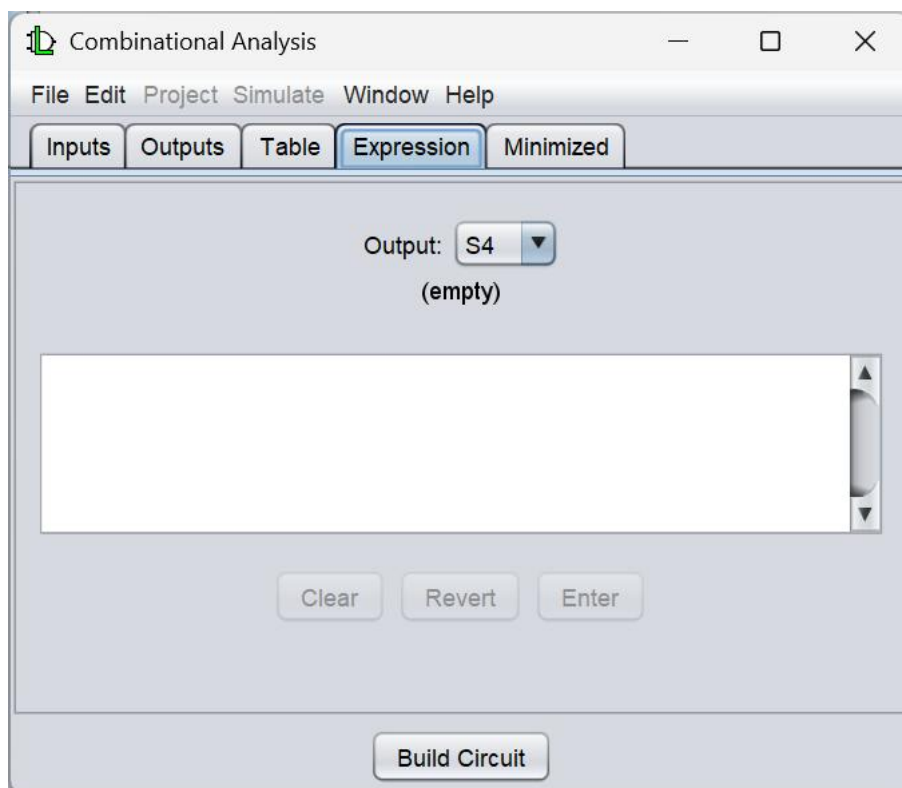
第一步：新建一个电路（6 个输入、 5 个输出）



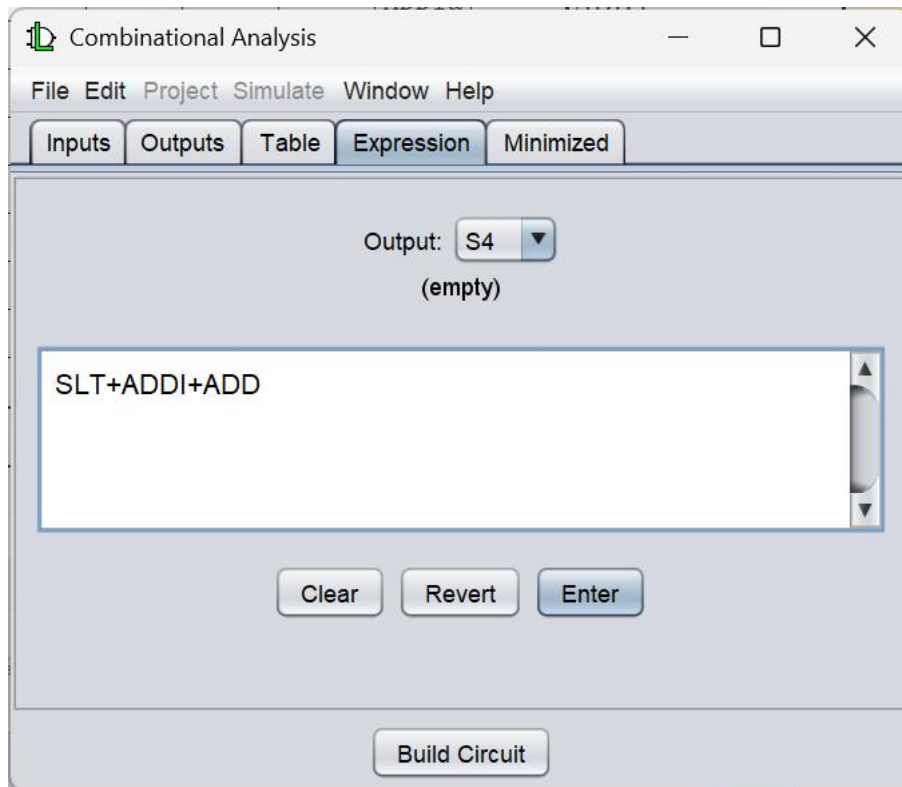
第二步：点击这个菜单



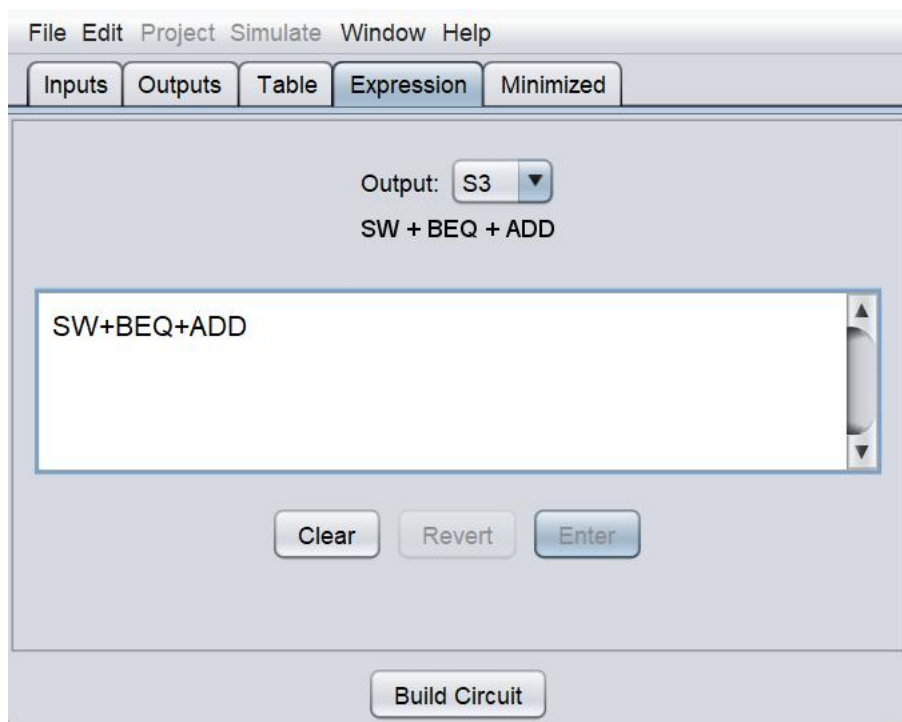
第三步：点击“Expression”



第四步：输入 S4 的逻辑公式，点击“Enter”



第五步：输入 S3、 S2、 S1、 S0 的逻辑公式



File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: S2 ▼

ADDI + BEQ + LW

ADDI + BEQ + LW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

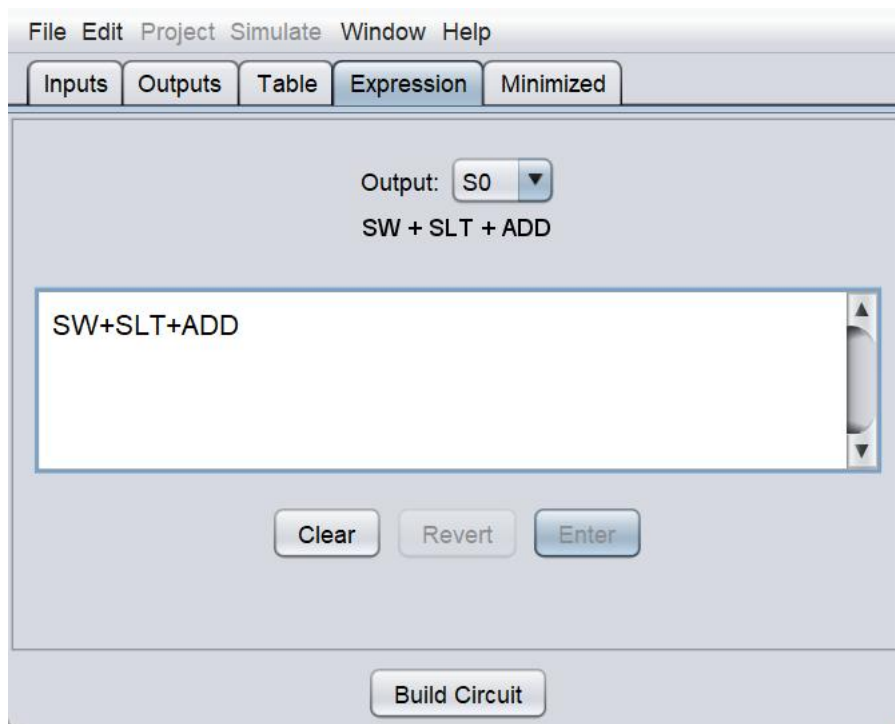
Output: S1 ▼

BEQ + SLT + ADDI

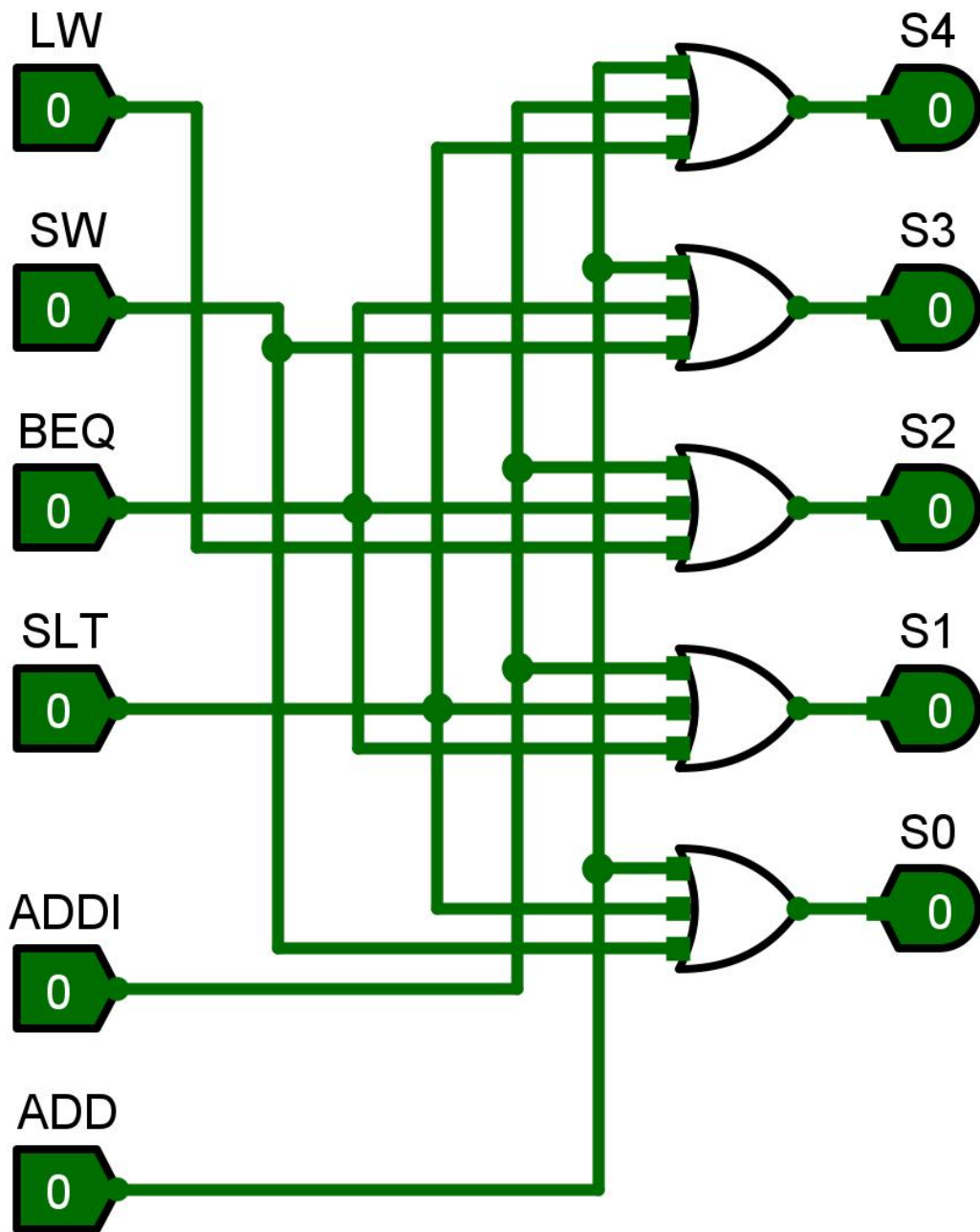
BEQ+SLT+ADDI

Clear Revert Enter

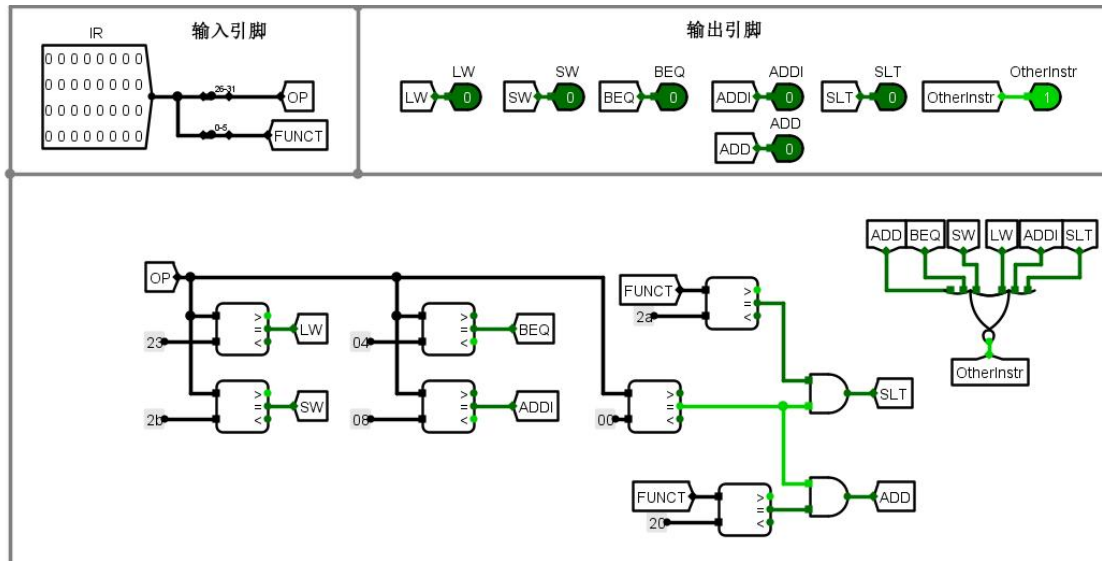
Build Circuit



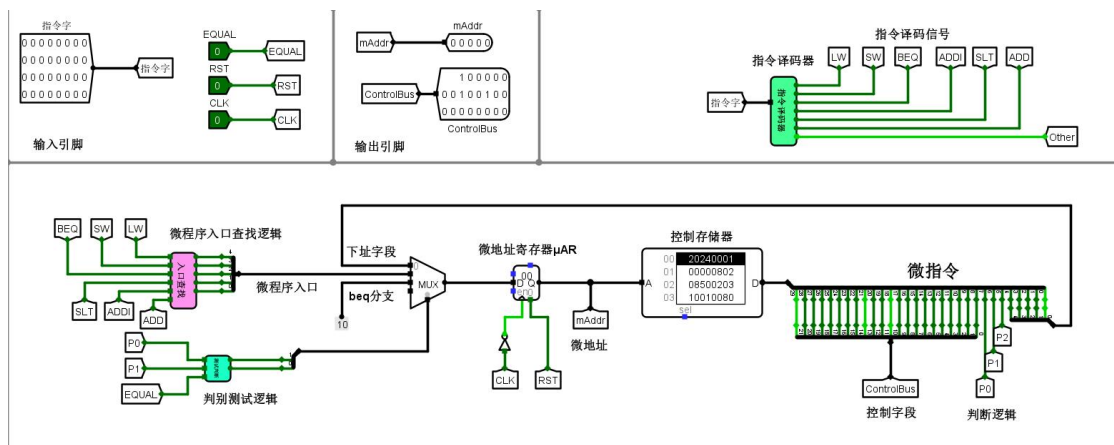
第六步：点击“Build Circuit”， OK， Yes；即可生成电路



测试生成的“微程序入口查找逻辑电路”的正确性。



修改微程序控制器电路，增加 ADD 指令



(2) 通过测试程序验证设计电路的正确性，将运行结果 copy 到实验报告中，并给出相关文字说明。

使用测试程序 test2.hex，对设计好的单总线 MIPS 处理器（微程序控制器，支持 6 条指令）进行测试。

```

test.hex  sum_riscv.asi  fib_riscv.asm  sort3_riscv.asi  sort4_riscv.asi  test2.  X  +
文件  编辑  查看

#单总线结构MIPS处理器测试程序                                test2.asm
#测试6条指令 (lw、sw、beq、addi、slt、add)
#程序执行完 RAM存储器的100、101、102单元的内容分别为：8、9、a

main:
    addi $s0,$zero,8          #s0=8
    sw $s0,1024($zero)        #s0=8 保存到(1024, 即100单元)

    addi $s0,$zero,9          #s0=9
    sw $s0,1028($zero)        #s0=9 保存到(1028, 即101单元)

    lw $s1,1024($zero)         #s1=8
    lw $s2,1028($zero)         #s2=9

    slt $s3,$s1,$s2           # s1 < s2    s3=1

    beq $s1,$s2,loop1         #s1不等于s2  执行下一条指令

    add $s4,$s3,$s2            # s4=1+9=10=a

    sw $s4,1032($zero)         #s4=10=a  保存到(1032, 即102单元)

    beq $zero,$zero,loop2      #转loop2

loop1:
    addi $s4,$zero,2           #s4=2
    sw $s4,1032($zero)         #s4=2 保存到(1032, 即102单元)
                                这条指令不会执行到
                                这条指令不会执行到

loop2:
    beq $zero,$zero,loop2      #转loop2    死循环

it Project Simulate Window Help
000 [20100008] ac100400 20100009 ac100404 8c110400 8c120404 0232982a 12320003 0272a020 ac140408 10000002 20140002 ac140408 1000ffff 000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 test2.hex

```

000 00000000 00000000 00000000 00000000 0000
0e0 00000000 00000000 00000000 00000000 0000
0f0 00000000 00000000 00000000 00000000 0000
100 00000008 00000009 0000000a 00000000 0000
110 00000000 00000000 00000000 00000000 0000
120 00000000 00000000 00000000 00000000 0000

```

运行后存储器 100、101、102 单元的内容分别为 8、9、a

在单总线 MIPS 处理器（微程序控制器，支持 6 条指令）的数据通路上运行求累加和程序、 计算费波那契数列程序、 排序程序（降序排序、升序排序） 。

```

sum_riscv.asi  fib_riscv.asm  sort3_riscv.asi  sort4_riscv.asi  test2.asm  sum_i  X  ▶  +
文件  编辑  查看

#MIPS程序 求累加和      sum mips bus.asm
#求累加和: 1+2+.....+n, n的值为10 (可以改变)
#累加和的结果存放到地址为1024的RAM存储器中 (即RAM存储器的100单元中)

main:
    addi $s0,$zero,10      # n=10 -> s0
    addi $s1,$zero,1       # 1 -> s1
    addi $s2,$zero,1       # 1 -> s2
    addi $s3,$zero,0       # 0 -> s3
loop:
    add $s3,$s3,$s1        # s3+s1 -> s3
    beq $s1,$s0,finish     # 如果s1=s0, 则转finish
    add $s1,$s1,$s2        # s1+s2 -> s1
    beq $zero,$zero,loop   # goto loop
finish:
    sw $s3,1024($zero)     # s3 存到地址为0的存储单元中
end:
    beq $zero,$zero,end    # goto end

```

sum_mips_bus

```

Hex Editor
Project Simulate Window Help

000 2010000a 20110001 20120001 20130000 02719820 12300002 02328820 1000fffc ac130400 1000ffff 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 sum_mips_bus.hex

```

-----
0f0 00000000 00000000
100 00000037 00000000
110 00000000 00000000

```

运行后累加和结果存放在存储器 100 单元中

```

#MIPS程序 计算费波那契数列      fib_mips_bus.asm      n=10 (可以改变)
#计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 保存在地址为1024、1028、1032、1036、...的RAM存储器
#中
#即RAM存储器100、101、102、.....单元中

main:
    addi $s2,$zero,10            # $s2=n=10

    addi $s1,$zero,0            # $s1=0
    sw $s1,1024($zero)          # 0 -> (1024)

    addi $s1,$zero,1            # $s1=1
    sw $s1,1028($zero)          # 1 -> (1028)

    sw $s1,1032($zero)          # 1 -> (1032)

    addi $s3,$zero,2            # $s3 = 2
    addi $s4,$zero,1            # $s4 = 1
    addi $s5,$zero,1            # $s5 = 1

loop:
    add $s6,$s4,$s5             # $s4+$s5 -> $s6
    addi $s4,$s5,0              # $s5 -> $s4
    addi $s5,$s6,0              # $s6 -> $s5
    addi $s3,$s3,1              # $s3+1 -> $s3
    add $s7,$s3,$s3              # $s3+$s3 -> $s7
    add $s0,$s7,$s7              # $s7+$s7 -> $s0
    sw $s6,1024($s0)             # $s6 -> 1024($s0)
    beq $s2,$s3,end              # if $s2 = $s3 goto end
    beq $zero,$zero,loop         # goto loop

end:
    beq $zero,$zero,end         # goto end

```

fib_mips_bus

```

Edit Project Simulate Window Help
000 2012000a 20110000 ac110400 20110001 ac110404 ac110408 20130002 20140001 20150001 0295b020 22b40000 22d50000 22730001 0273b820 02778020 ae160400
010 12530001 1000fff7 1000fff0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
...
```

载入 fib_mips_bus.hex

```

000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
100 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037 00000000 00000000 00000000 00000000 00000000
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
120 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

运行后计算好的 11 个数（0、1、1、2、3、5、8、13、21、34、……）保存在 RAM 存储器 100、101、102、.....单元中

```

#MIPS程序 sort3 mips bus.asm 排序程序（降序排列，从大到小） 仅使用5条指令（lw、sw、beq、addi、slt）
#先将5个数（3、1、5、2、4）存放到地址为1024开始的RAM存储器中，然后对这5个数进行排序。
#排序结果仍然在地址为1024开始的RAM存储器中（即RAM存储器的100、101、102、103、104单元中）

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改）保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改）保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改）保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改）保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改）保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024       #s0=1024          排序区间开始地址
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4 排序区间结束地址    5个数

sort loop:
    lw $s3,0($s0)             #s3=(s0)
    lw $s4,0($s1)             #s4=(s1)
    slt $t0,$s3,$s4           #如果s3<s4, 则置t0=1; 否则, 置t0=0    降序排序    从大到小
    beq $t0,$zero,sort next   #如果t0=0, 则转sort next
    sw $s3,0($s1)             #交换(s0)和(s1)
    sw $s4,0($s0)             #交换(s0)和(s1)

sort next:
    addi $s1,$s1,-4           #s1-4 -> $s1
    beq $s0,$s1,loop1         #如果s0=$s1, 则转loop1
    beq $zero,$zero,sort loop #转sort loop

loop1:
    addi $s0,$s0,4            #s0+4 -> $s0
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4 排序区间结束地址    5个数
    beq $s0,$s1,loop2         #如果s0=$s1, 则转loop2
    beq $zero,$zero,sort loop #转sort loop

loop2:
    beq $zero,$zero,loop2     #转loop2    死循环

```

sort3_mips_bus

Project	Simulate	Window	Help
000	20100003	ac100400	20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0274402a 11000002
010	ae330000	ae140000	2231fffc 12110001 1000ff7f 22100004 20110410 12110001 1000ff7f 1000ff7f 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020	00000000	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

载入 sort3_mips_bus.hex

```

0e0 00000000 00000000 00000000 00000000 00000000 000000
0f0 00000000 00000000 00000000 00000000 00000000 000000
100 00000005 00000004 00000003 00000002 00000001 000000
110 00000000 00000000 00000000 00000000 00000000 000000
120 00000000 00000000 00000000 00000000 00000000 000000
130 00000000 00000000 00000000 00000000 00000000 000000

```

运行后 RAM 存储器的 100、101、102、103、104 单元中五个数据降序排序

```

#MIPS程序 sort4 mips bus.asm 排序程序（升序排列，从小到大） 仅使用5条指令（lw, sw, beq, addi, slt）
#先将5个数（3、1、5、2、4）存放到地址为1024开始的RAM存储器中，然后对这5个数进行排序。
#排序结果仍然在地址为1024开始的RAM存储器中（即RAM存储器的100、101、102、103、104单元中）

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改）保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改）保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改）保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改）保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改）保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024        #s0=1024
    addi $s1,$zero,1040        #s1=1040=1024+5*4-4 排序区间开始地址 排序区间结束地址 5个数

sort loop:
    lw $s3,0($s0)              #s3=(s0)
    lw $s4,0($s1)              #s4=(s1)
    slt $t0,$s4,$s3            #如果s4<s3, 则置t0=1; 否则, 置t0=0 升序排序 从小到大
    beq $t0,$zero,sort next    #如果t0=0, 则转sort next
    sw $s3,0($s1)              #交换(s0)和(s1)
    sw $s4,0($s0)              #交换(s0)和(s1)

sort next:
    addi $s1,$s1,-4            #s1-4 -> s1
    beq $s0,$s1,loop1          #如果s0=s1, 则转loop1
    beq $zero,$zero,sort loop  #转sort loop

loop1:
    addi $s0,$s0,4             #s0+4 -> s0
    addi $s1,$zero,1040        #s1=1040=1024+5*4-4 排序区间结束地址 5个数
    beq $s0,$s1,loop2          #如果s0=s1, 则转loop2
    beq $zero,$zero,sort loop  #转sort loop

loop2:
    beq $zero,$zero,loop2      #转loop2 死循环

```

sort4_mips_bus

```

Project Simulate Window Help
000 [20100003] ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0293402a 11000002
010 ae330000 ae140000 2231fffc 12110001 1000ffff 22100004 20110410 12110001 1000ffff 1000ffff 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 sort4_mips_bus.hex

```

000 00000000 00000000 00000000 00000000 00000000 00000000 000
0e0 00000000 00000000 00000000 00000000 00000000 00000000 000
0f0 00000000 00000000 00000000 00000000 00000000 00000000 000
100 00000001 00000002 00000003 00000004 00000005 00000000 000
110 00000000 00000000 00000000 00000000 00000000 00000000 000

```

运行后 RAM 存储器的 100、101、102、103、104 单元中五个数据升序排序

3.3 挑战性实验 +8 分

单总线结构 MIPS 处理器（硬布线控制器）（增加 1 条 add 指令，共 6 条指令）

(1) 按照实验课件的要求，完成设计任务，提交设计文件单总线 MIPS 处理器-硬布线.circ;

填写“状态机自动生成（6 条指令）（空表）.xlsx”，并生成状态机逻辑公式，然后在 Logisim 上生成逻辑电路。

ADD 状态跳转和 ADDI 相同，从 S3 直接跳转到 S6

当前状态(现态)						输入信号						下一状态 (次态)				
S3	S2	S1	S0	现态 10进制		LW	SW	BEQ	SLT	ADDI	ADD	次态 10进制	N3	N2	N1	N0
0	0	0	0	0								1	0	0	0	1
0	0	0	1	1								2	0	0	1	0
0	0	1	0	2								3	0	0	1	1
0	0	1	1	3	1							4	0	1	0	0
0	0	1	1	3		1						4	0	1	0	0
0	0	1	1	3				1				4	0	1	0	0
0	0	1	1	3					1			6	0	1	1	0
0	0	1	1	3						1		6	0	1	1	0
0	1	0	0	4							1	6	0	1	1	0
0	1	0	1	5								5	0	1	0	1
0	1	0	1	5								6	0	1	1	0
0	1	1	0	6								7	0	1	1	1
0	1	1	1	7								8	1	0	0	0
1	0	0	0	8								0	0	0	0	0

S3	S2	S1	S0	LW	SW	BEQ	SLT	ADDI	ADD	最小项表达式	N3	N2	N1	N0
~S3&~S2&~S1&~S0										~S3&~S2&~S1&~S0				~S3&~S2&~S1&~S0+
~S3&~S2&~S1&S0										~S3&~S2&~S1&S0			~S3&~S2&~S1&S0+	
~S3&~S2&S1&~S0										~S3&~S2&S1&~S0			~S3&~S2&S1&~S0+	~S3&~S2&S1&~S0+
~S3&~S2&S1&S0	LW&									~S3&~S2&S1&S0&LW		~S3&~S2&S1&S0&LW+		
~S3&~S2&S1&S0		SW&								~S3&~S2&S1&S0&SW		~S3&~S2&S1&S0&SW+		
~S3&~S2&S1&S0			BEQ&							~S3&~S2&S1&S0&BEQ		~S3&~S2&S1&S0&BEQ+		
~S3&~S2&S1&S0				SLT&						~S3&~S2&S1&S0&SLT		~S3&~S2&S1&S0&SLT+	~S3&~S2&S1&S0&SLT+	
~S3&~S2&S1&S0					ADDI&					~S3&~S2&S1&S0&ADDI		~S3&~S2&S1&S0&ADDI+	~S3&~S2&S1&S0&ADDI+	
~S3&~S2&S1&S0						ADD&				~S3&~S2&S1&S0&ADD		~S3&~S2&S1&S0&ADD+	~S3&~S2&S1&S0&ADD+	
~S3&S2&~S1&~S0										~S3&S2&~S1&~S0		~S3&S2&~S1&~S0+	~S3&S2&~S1&~S0+	~S3&S2&~S1&~S0+
~S3&S2&~S1&S0										~S3&S2&~S1&S0		~S3&S2&~S1&S0+	~S3&S2&~S1&S0+	~S3&S2&~S1&S0+
~S3&S2&S1&~S0										~S3&S2&S1&~S0		~S3&S2&S1&~S0+	~S3&S2&S1&~S0+	~S3&S2&S1&~S0+
~S3&S2&S1&S0										~S3&S2&S1&S0	~S3&S2&S1&S0+			
S3&~S2&~S1&~S0										S3&~S2&~S1&~S0				

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: N3 ▼

$\overline{S3} \ S2 \ S1 \ S0$

~S3&S2&S1&S0

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: N1 ▼

$\overline{S3} \ \overline{S2} \ \overline{S1} \ S0 + \overline{S3} \ \overline{S2} \ S1 \ \overline{S0} + \overline{S3} \ \overline{S2} \ S1 \ S0 \text{ SLT} + \overline{S3} \ \overline{S2} \ S1 \ S0 \text{ ADDI} + \overline{S3} \ \overline{S2} \ S1 \ S0 \text{ ADD} + \overline{S3} \ S2 \ \overline{S1} \ S0 + \overline{S3} \ S2 \ S1 \ \overline{S0}$

~S3&~S2&~S1&S0+~S3&~S2&S1&~S0+~S3&~S2&S1&S0&SLT+~S3&~S2&S1&S0&ADDI+~S3&~S2&S1&S0&ADD+~S3&S2&~S1&S0+~S3&S2&S1&~S0

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

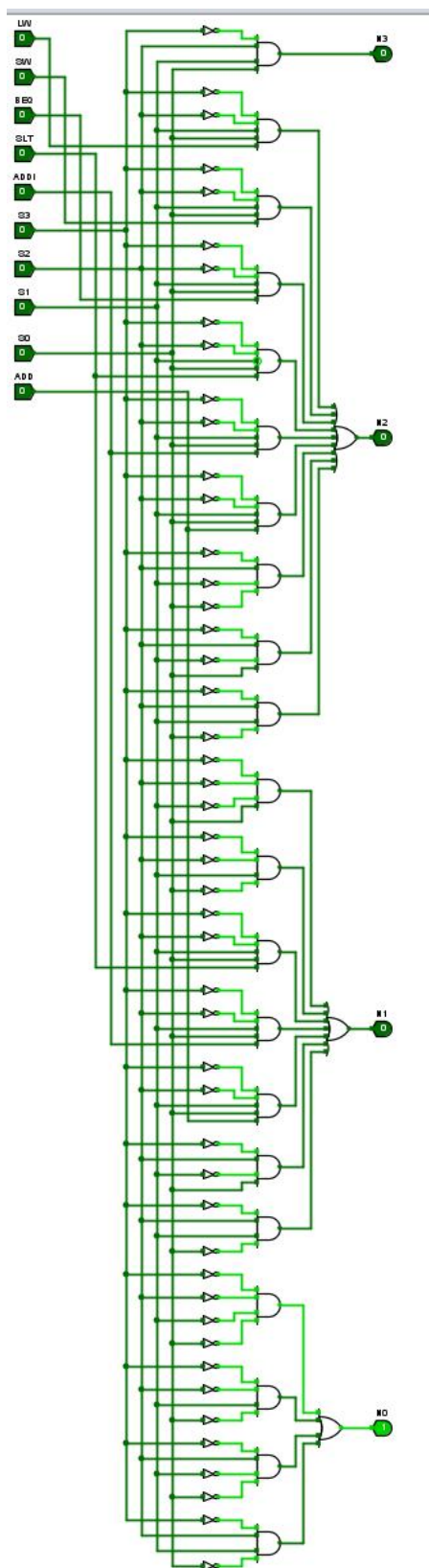
Output: N0 ▼

$\overline{S3} \overline{S2} \overline{S1} \overline{S0} + \overline{S3} \overline{S2} S1 \overline{S0} + \overline{S3} S2 \overline{S1} \overline{S0} + \overline{S3} S2 S1 \overline{S0}$

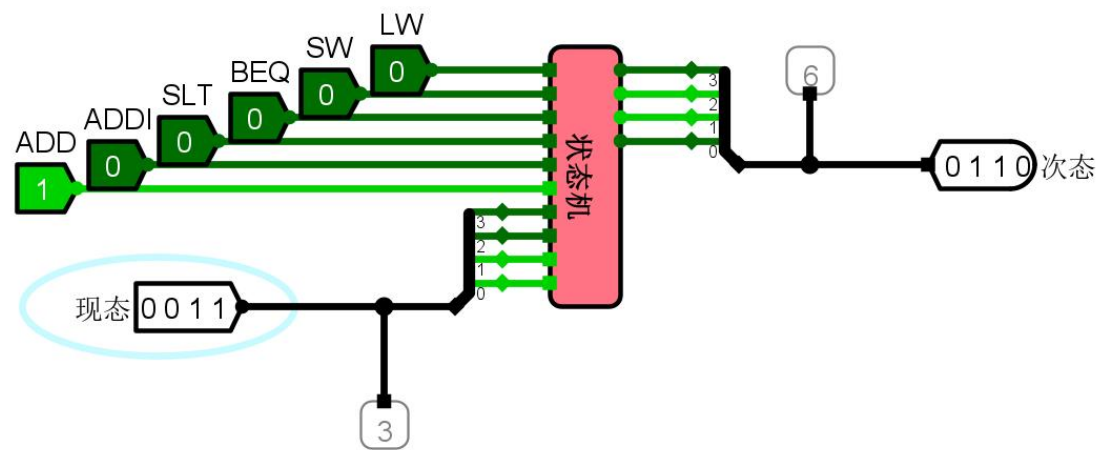
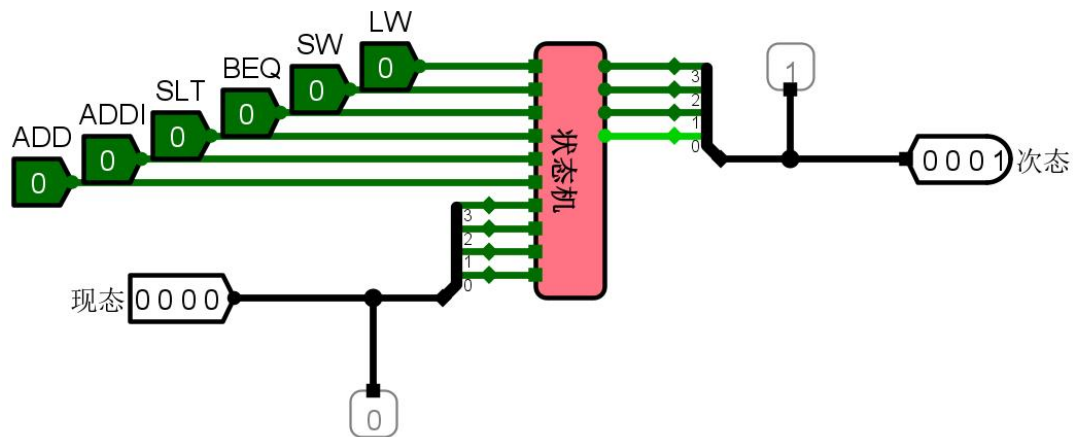
~S3&~S2&~S1&~S0+~S3&~S2&S1&~S0+~S3&S2&~S1&~S0
+~S3&S2&S1&~S0

Clear Revert Enter

Build Circuit



测试生成的“状态机”电路的正确性。



填写“硬布线控制器组合逻辑自动生成（6条指令）（空表）.xlsx”中的 ADD 指令部分，并生成硬布线控制器组合逻辑的逻辑公式，然后在 Logisim 上生成逻辑电路

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Pcout ▼

Mif T1 + Mex T1 BEQ

Mif&T1+Mex&T1&BEQ

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: DRout ▼

Mif T4 + Mex T3 LW

Mif&T4+Mex&T3&LW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Zout ▼

Mif T3 + Mex T1 LW + Mex T1 SW + Mex T3 BEQ EQUAL + Mex T3 SLT
+ Mex T3 ADDI + Mex T3 ADD

Mif&T3+Mex&T1&LW+Mex&T1&SW+Mex&T3&BEQ&EQUAL+
Mex&T3&SLT+Mex&T3&ADDI+Mex&T3&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Rout ▼

Mcal T1 LW + Mcal T1 SW + Mex T2 SW + Mcal T1 BEQ + Mcal T2 BEQ
+ Mex T1 SLT + Mex T2 SLT + Mex T1 ADDI + Mex T1 ADD + Mex T2 ADD

Mcal&T1&LW+Mcal&T1&SW+Mex&T2&SW+Mcal&T1&BEQ+
Mcal&T2&BEQ+Mex&T1&SLT+Mex&T2&SLT+Mex&T1&ADDI
+Mex&T1&ADD+Mex&T2&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output:

Mcal T2 LW + Mcal T2 SW + Mex T2 ADDI

Mcal&T2&LW+Mcal&T2&SW+Mex&T2&ADDI

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output:

Mex T2 BEQ

Mex&T2&BEQ

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: DRE... ▼

Mex T3 SW

Mex&T3&SW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Pcin ▼

Mif T3 + Mex T3 BEQ EQUAL

Mif&T3+Mex&T3&BEQ&EQUAL

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: ARin ▼

Mif T1 + Mex T1 LW + Mex T1 SW

Mif&T1+Mex&T1&LW+Mex&T1&SW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: DREin ▼

Mif T3 + Mex T2 LW

Mif&T3+Mex&T2&LW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: DRin ▼

Mex T2 SW

Mex&T2&SW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Xin ▼

Mif T1 + Mcal T1 LW + Mcal T1 SW + Mcal T1 BEQ + Mex T1 BEQ + Mex T1 SLT + Mex T1 ADDI + Mex T1 ADD

Mif&T1+Mcal&T1&LW+Mcal&T1&SW+Mcal&T1&BEQ+Mex&T1&BEQ+Mex&T1&SLT+Mex&T1&ADDI+Mex&T1&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Rin ▼

Mex T3 LW + Mex T3 SLT + Mex T3 ADDI + Mex T3 ADD

Mex&T3&LW+Mex&T3&SLT+Mex&T3&ADDI+Mex&T3&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: IRin ▼

Mif T4

Mif&T4

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: PSWin ▼

Mcal T2 BEQ

Mcal&T2&BEQ

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: RegTgt ▼

Mex T2 SW + Mcal T2 BEQ + Mex T2 SLT + Mex T2 ADD

Mex&T2&SW+Mcal&T2&BEQ+Mex&T2&SLT+Mex&T2&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: RegDst ▼

Mex T3 SLT + Mex T3 ADD

Mex&T3&SLT+Mex&T3&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Add ▼

Mcal T2 SW + Mex T2 BEQ + Mex T2 ADDI + Mex T2 ADD

Mcal&T2&SW+Mex&T2&BEQ+Mex&T2&ADDI+Mex&T2&ADD

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Add4 ▼

Mif T2

Mif&T2

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: Sit ▼

Mex T2 SLT

Mex&T2&SLT

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

Output: READ ▼

Mif T3 + Mex T2 LW

Mif&T3+Mex&T2&LW

Clear Revert Enter

Build Circuit

File Edit Project Simulate Window Help

Inputs Outputs Table Expression Minimized

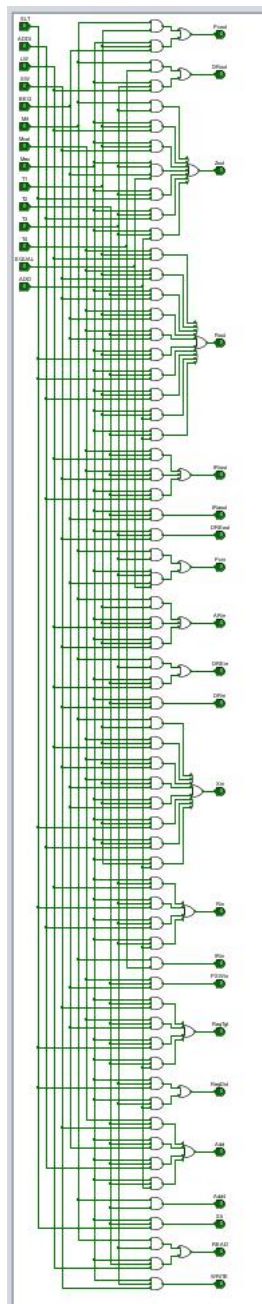
Output: WRITE ▼

Mex T3 SW

Mex&T3&SW

Clear Revert Enter

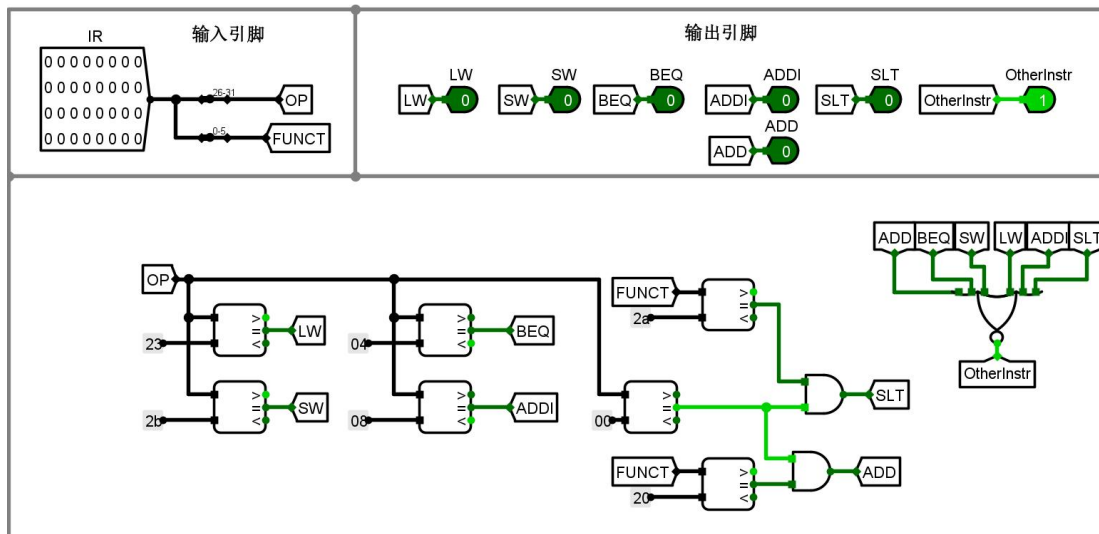
Build Circuit



修改指令译码器

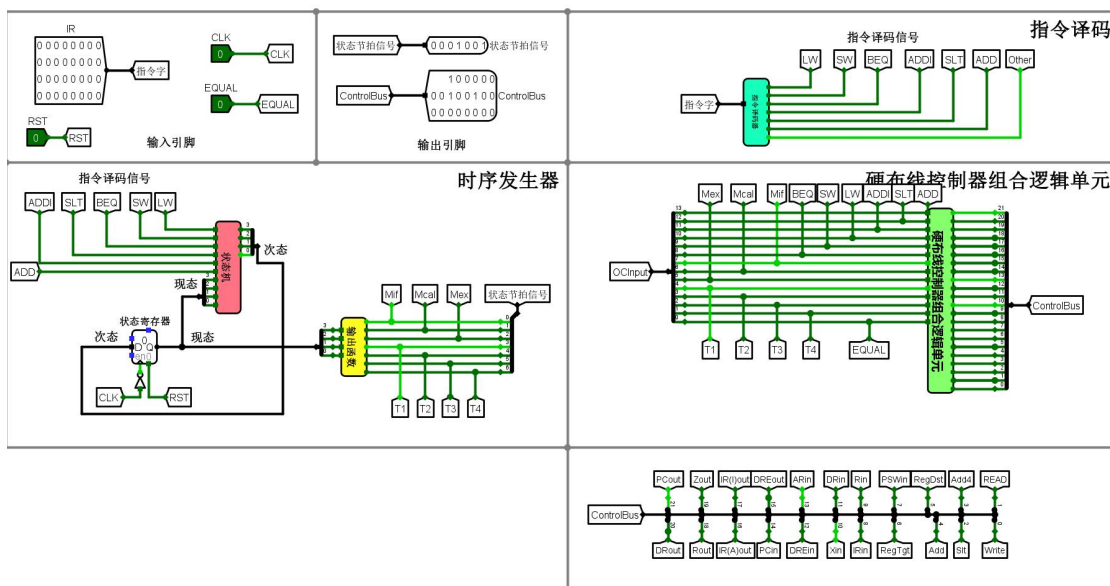
同微程序控制器

指令译码器 (ID)



修改硬布线控制器，增加 ADD 指令

硬布线控制器



(2) 通过测试程序验证设计电路的正确性，将运行结果 copy 到实验报告中，并给出相关文字说明。

使用测试程序 test2.hex，对设计好的单总线 MIPS 处理器（硬布线控制器，支持 6 条指令）进行测试。


```

sum_mips_bi  fib_mips_bus  sort3_mips_t  sort4_mips_t  test2.asm  sum_i  ×  ▶  +
文件  编辑  查看

#MIPS程序 求累加和          sum_mips_bus.asm
#求累加和: 1+2+.....+n, n的值为10 (可以改变), 累加和的结果存放到地址为1024的数据存储器中

main:
    addi $s0,$zero,10          # n=10 -> s0
    addi $s1,$zero,1          # 1 -> s1
    addi $s2,$zero,1          # 1 -> s2
    addi $s3,$zero,0          # 0 -> s3
loop:
    add $s3,$s3,$s1            # s3+s1 -> s3
    beq $s1,$s0,finish         # 如果s1=s0, 则转finish
    add $s1,$s1,$s2            # s1+s2 -> s1
    beq $zero,$zero,loop       # goto loop
finish:
    sw $s3,1024($zero)         # s3 存到地址为0的存储单元中
end:
    beq $zero,$zero,end       # goto end

```

sum_mips_bus

t Project Simulate Window Help											
000	2010000a	20110001	20120001	20130000	02719820	12300002	02328820	1000fffc	ac130400	1000ffff	00000000 000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000 000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000 000

载入 sum_mips_bus.hex

```

0f0 00000000 00000000
100 00000037 00000000
110 00000000 00000000
120 00000000 00000000

```

运行后累加和结果存放在存储器 100 单元中

```

#MIPS程序 计算斐波那契数列          fib mips bus.asm
#n=10 (可以改变), 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 保存在地址为
1024、1028、1032、1036、...的数据存储器中

main:
    addi $s2,$zero,10                # $s2=n=10

    addi $s1,$zero,0                 # $s1=0
    sw $s1,1024($zero)               # 0 -> (1024)

    addi $s1,$zero,1                 # $s1=1
    sw $s1,1028($zero)               # 1 -> (1028)

    sw $s1,1032($zero)               # 1 -> (1032)

    addi $s3,$zero,2                 # $s3 = 2
    addi $s4,$zero,1                 # $s4 = 1
    addi $s5,$zero,1                 # $s5 = 1
loop:
    add $s6,$s4,$s5                  # $s4+$s5 -> $s6
    addi $s4,$s5,0                   # $s5 -> $s4
    addi $s5,$s6,0                   # $s6 -> $s5
    addi $s3,$s3,1                   # $s3+1 -> $s3
    add $s7,$s3,$s3                  # $s3+$s3 -> $s7
    add $s0,$s7,$s7                  # $s7+$s7 -> $s0
    sw $s6,1024($s0)                 # $s6 -> 1024($s0)
    beq $s2,$s3,end                  # if $s2 = $s3 goto end
    beq $zero,$zero,loop             # goto loop
end:
    beq $zero,$zero,end              # goto end

```

fib_mips_bus

```

object Simulate Window Help
000 2012000a 20110000 ac110400 20110001 ac110404 ac110408 20130002 20140001 20150001 0295b020 22b40000 22d50000 22730001 0273b820 02f78020 ae160400
010 12530001 1000fff7 1000ffff 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 fib_mips_bus.hex

```

000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00
100 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037 00000000 00000000 00000000 00
110 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00
120 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00

```

运行后计算好的 11 个数（0、1、1、2、3、5、8、13、21、34、……）保存在 RAM

存储器 100、101、102、.....单元中

```

#MIPS程序 sort3_mips_bus.asm 排序程序（降序排列，从大到小） 仅使用5条指令（lw、sw、beq、addi、slt）

#先将5个数（3、1、5、2、4）存放到地址为1024开始的数据存储器中，然后对这5个数进行排序。

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改）保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改）保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改）保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改）保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改）保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024       #s0=1024          排序区间开始地址
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4 排序区间结束地址 5个数

sort loop:
    lw $s3,0($s0)             #s3=(s0)
    lw $s4,0($s1)             #s4=(s1)
    slt $t0,$s3,$s4           #如果s3<s4, 则置t0=1; 否则, 置t0=0 降序排序 从大到小
    beq $t0,$zero,sort next   #如果t0=0, 则转sort next
    sw $s3,0($s1)             #交换(s0)和(s1)
    sw $s4,0($s0)             #交换(s0)和(s1)

sort next:
    addi $s1,$s1,-4           #s1-4 -> s1
    beq $s0,$s1,loop1         #如果s0=s1, 则转loop1
    beq $zero,$zero,sort loop  #转sort loop

loop1:
    addi $s0,$s0,4            #s0+4 -> s0
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4 排序区间结束地址 5个数
    beq $s0,$s1,loop2         #如果s0=s1, 则转loop2
    beq $zero,$zero,sort loop  #转sort loop

loop2:
    beq $zero,$zero,loop2     #转loop2 死循环

```

sort3_mips_bus

```

object Simulate Window Help
000 20100003 ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0274402a 11000002
010 ae330000 ae140000 2231fff0 12110001 1000fff7 22100004 20110410 12110001 1000fff3 1000fff0 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 sort3_mips_bus.hex

```

0e0 00000000 00000000 00000000 00000000 00000000 000
0f0 00000000 00000000 00000000 00000000 00000000 000
100 00000005 00000004 00000003 00000002 00000001 000
110 00000000 00000000 00000000 00000000 00000000 000
120 00000000 00000000 00000000 00000000 00000000 000

```

运行后 RAM 存储器的 100、101、102、103、104 单元中五个数据降序排序

```

#MIPS程序  sort4 mips bus.asm  排序程序（升序排列，从小到大）  仅使用5条指令（lw、sw、beq、addi、slt）

#先将5个数（3、1、5、2、4）存放到地址为1024开始的数据存储器中，然后对这5个数进行排序。

.text

main:
    addi $s0,$zero,3          #第1个数=3（可以修改）  保存到(1024+0)
    sw $s0,1024($zero)
    addi $s0,$zero,1          #第2个数=1（可以修改）  保存到(1024+4)
    sw $s0,1028($zero)
    addi $s0,$zero,5          #第3个数=5（可以修改）  保存到(1024+8)
    sw $s0,1032($zero)
    addi $s0,$zero,2          #第4个数=2（可以修改）  保存到(1024+12)
    sw $s0,1036($zero)
    addi $s0,$zero,4          #第5个数=4（可以修改）  保存到(1024+16)
    sw $s0,1040($zero)

    addi $s0,$zero,1024       #s0=1024          排序区间开始地址
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4  排序区间结束地址    5个数

sort loop:
    lw $s3,0($s0)             #s3=(s0)
    lw $s4,0($s1)             #s4=(s1)
    slt $t0,$s4,$s3           #如果s4<s3, 则置t0=1; 否则, 置t0=0    升序排序    从小到大
    beq $t0,$zero,sort next   #如果t0=0, 则转sort next
    sw $s3,0($s1)             #交换(s0)和(s1)
    sw $s4,0($s0)             #交换(s0)和(s1)

sort next:
    addi $s1,$s1,-4           #s1-4 -> s1
    beq $s0,$s1,loop1         #如果s0=s1, 则转loop1
    beq $zero,$zero,sort loop #转sort loop

loop1:
    addi $s0,$s0,4            #s0+4 -> s0
    addi $s1,$zero,1040       #s1=1040=1024+5*4-4  排序区间结束地址    5个数
    beq $s0,$s1,loop2         #如果s0=s1, 则转loop2
    beq $zero,$zero,sort loop #转sort loop

loop2:
    beq $zero,$zero,loop2     #转loop2    死循环

```

sort4_mips_bus

```

Project Simulate Window Help
000 20100003 ac100400 20100001 ac100404 20100005 ac100408 20100002 ac10040c 20100004 ac100410 20100400 20110410 8e130000 8e340000 0293402a 11000002
010 ae330000 ae140000 2231fff0 12110001 1000fff7 22100004 20110410 12110001 1000fff3 1000fff7 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

载入 sort4_mips_bus.hex

```

010 00000000 00000000 00000000 00000000 00000000 00000000
100 00000001 00000002 00000003 00000004 00000005 00000006
110 00000000 00000000 00000000 00000000 00000000 00000000

```

运行后 RAM 存储器的 100、101、102、103、104 单元中五个数据升序排序